
ON THE (*Generative*) LINEAR SKETCHING PROBLEM*

Xinyu Yuan 

College of Computer Science and Technology
Zhejiang University
Hangzhou, China
yxy5315@gmail.com

Yan Qiao 

School of Computer Science and Information Engineering
Hefei University of Technology
Hefei, China
qiaoyan@hfut.edu.cn

Zonghui Wang 

College of Computer Science and Technology
Zhejiang University
Hangzhou, China
zhwang@zju.edu.cn

Wenzhi Chen 

College of Computer Science and Technology
Zhejiang University
Hangzhou, China
chenwz2@zju.edu.cn

ABSTRACT

Sketch techniques have been extensively studied in recent years and are especially well-suited to data streaming scenarios, where the sketch summary is updated quickly and compactly. However, it is challenging to recover the current state from these summaries in a way that is accurate, fast, and real. In this paper, we seek a new solution that reconciles this tension, aiming for near-perfect recovery with lightweight computational procedures. Focusing on linear sketching problems of the form $\Phi f \rightarrow f$, our study proceeds in three phases. First, we revisit existing techniques and show the root cause of the dilemma: an orthogonal information loss during the summarization process. Second, we investigate how generative priors can be leveraged to bridge the gap and thereby allow theoretically superior recovery. Third, we present FLORE, the first generative sketching paradigm to embrace these analyses. At its core, FLORE integrates invertible flows with efficient sketch instances to achieve the best of two worlds. More importantly, FLORE can be trained online without access to ground-truth data, which is readily deployable in practice. Comprehensive experiments demonstrate FLORE’s ability to provide high-quality recovery, and support summary with low computing overhead, outperforming previous methods by up to $10^3 \times$ in error reduction and $10^2 \times$ in processing speed compared to learning-based solutions. To aid future research, our experimental codebase is available at  [ANONYMOUS GITHUB](#), and datasets are available at  [GOOGLE DRIVE](#).

Keywords Sketch · Stream Processing · Machine Learning · Generative Model

Contents

1	Introduction	3
2	Preliminaries	4
2.1	Problem Formulation	4
2.2	Generative Models	5
2.3	Notations & Definitions	5
3	Motivating Analyses	6

* This work is licensed under a [Creative Commons Attribution 4.0 International \(CC BY 4.0\) License](#).

3.1	Limitations of Existing Techniques (Proposition 1 & Theorem 1)	6
3.2	The Anatomy of Failure (Proposition 2)	10
4	Toward Generative Linear Sketching	11
4.1	From Lessons to Motivation	11
4.2	Integrating Generative Priors (Theorem 2)	11
4.3	On the Choice of GMs	12
4.3.1	Unconditional Distribution Modeling Capability	12
4.3.2	Conditional Generation for Stream Recovery	12
5	FLORE: A Flow-based Sketching Framework	15
5.1	System Overview	15
5.2	Model Architecture	15
5.3	Training Objective (Theorem 3)	17
5.4	GT-free Learning (Theorem 4)	18
5.5	Data Structure	22
5.5.1	Key Tracking Mechanism	22
5.5.2	Stream Filtering Mechanism	22
5.6	Other Details	23
5.7	End-to-End View	24
6	Evaluation	24
6.1	Experimental Setup	25
6.1.1	Datasets	25
6.1.2	Baselines	25
6.1.3	Hyperparameters	27
6.1.4	Metrics	27
6.2	Experiments on Real Data	27
6.3	Experiments on Synthetic Data	29
6.4	Results on FLORE's Efficiency	30
6.4.1	Stream Processing Speed	30
6.4.2	Convergence Time	31
6.5	Robustness to Stream Changes	32
6.6	Interpreting FLORE's Effectiveness	32
6.6.1	Effect of Generative Model	32
6.6.2	Effect of Training Objectives	33
6.6.3	Effect of Stream Filtering	35
7	Related Works	35
8	Conclusion	36

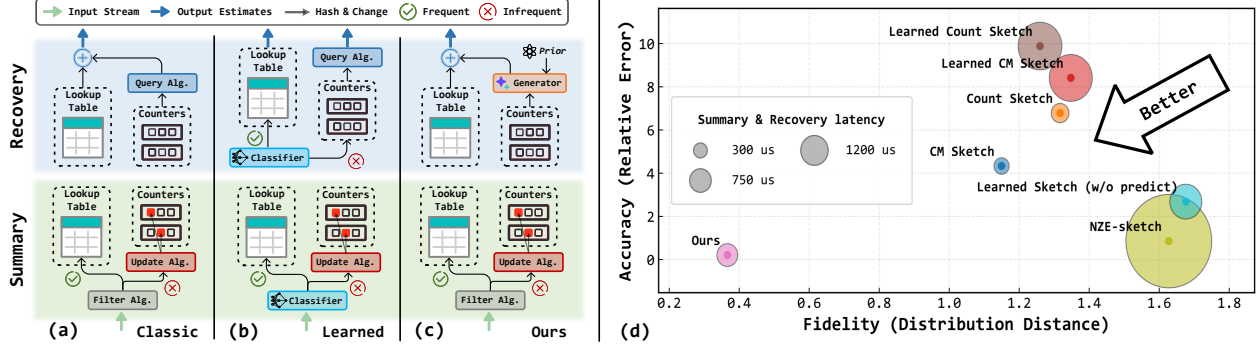


Figure 1: **Comparison of our generative solution and existing solutions.** From (a) to (c), we present an overview of the full pipelines of three sketching paradigms. In (d), our proposal achieves the best trade-off among accuracy, fidelity and speed under a 1MB sketch memory budget on CAIDA-2018 dataset.

A Background	44
A.1 Variational Autoencoder	44
A.2 Generative Adversarial Network	44
A.3 Flow-based Generative Model	45
A.4 Diffusion Model	46
B Useful Lemmas	47
C Visualizations	47
C.1 Learned Generative Flow	47
C.2 Recovery (distributional) Results	48

1 Introduction

Accurately measuring the volume of water from a fire hose while being directly hit by it is nearly an impossible feat. In essence, this captures the central difficulty of streaming data analysis: the data item arrives in a continuous, unrelenting torrent, so instead we need to process each observation quickly to create a summary of the current state. Here’s a simple example—for a stream of numbers, obtaining the cumulative sum requires storing only a single running total rather than all past inputs. Such a summary is referred to as a *sketch* of the data, which has been extensively studied [1, 2, 3], since it is very helpful in any situation where people are monitoring a database or network that is being updated continuously. However, due to some (hashing) collisions, coping with the vast scale of information means making compromises—the *description of the world via data sketching is approximate rather than exact*. A particular challenge has been to design algorithms that navigate tradeoffs effectively, like sacrificing speed for accuracy or memory consumption for reliability [4]. The challenge gets harder, though, when the questions about the data stream get more complicated, e.g., heavy hitters [5], entropy [6], and graph summarization [7].

Drawing the mathematical form from compressive sensing (CS) [8], this work interprets sketch as a coding problem, i.e., it is encoding sparse information, such as heavy-tailed traffic, down to compact summary and trying to extract information that lets them recover what was “squeezed in” initially (§2.1). Further, if the coding procedure is able to be represented as a linear transform of the input, i.e., $b = \Phi f$, it results in what we call a linear sketching problem [9, 10]. From this perspective, we start by analyzing a fundamental tension between sketching cost and provable ac-

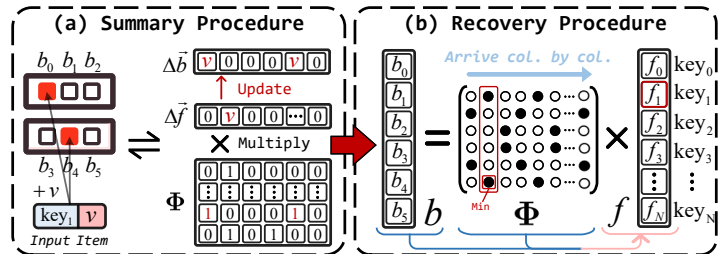


Figure 2: **Matrix formulation of Count-Min.** The linear sketching problem can be analyzed via simple matrix operations.

curacy for existing techniques (§3.1). And we reveal that the root cause lies in an inherent *deficiency of orthogonal information* that cannot be recovered without any prior knowledge (§3.2). As a result, the central challenge shifts toward compensating for the missing stream component that meets both realness and efficiency requirements.

In order to overcome the above challenge, this work turns to a brand-new machine learning (ML) paradigm empowered by *generative models* (GMs). Our key insight here is two-fold: (i) Theoretically, *by enforcing alignment with the learned distributional prior knowledge, GMs may help mitigate the aforementioned information loss* from aggressive summarization. By formulating the learned prior assumptions as a probability distribution, the recovery procedure can be reformulated as the problem of approximating the posterior distribution $p(f | b)$. (ii) Systematically, since data-plane summary is on the critical path, as shown in Fig. 1(c), *deploying GMs for only recovery in the control plane may avoid burdening the overall system*.

We study the generative linear sketching problem along two lines. In the *first line* (§4.2), we attempt to examine prevalent GMs on synthetic but representative streaming scenarios. Our preliminary experiments show that all GMs fit low-cardinality stream distributions (<10K unique keys) well; however, it is non-trivial to scale these GMs other than flow-based generative models (FGMs) to real-world applications, due to limitations of their own (e.g., unstable training, slow inference, and poor expressiveness). This motivates the *second line* (§5) of our work that leverages FGM to develop a novel and practical methodology for solving problems of interest. Specifically, we propose FLORE, the *first* deep generative sketching framework for fast summarization and accurate recovery while preserving fidelity. As shown in Figure 1(d), experiments on CAIDA-2018 dataset (d) show that, compared with both existing learning-based and classical approaches, FLORE simultaneously achieves high speed (throughput), fidelity (WMRE), and accuracy (ARE).

To realize FLORE, there are the following four core design components. (i) We adopt the data-control plane separation design to build a deployable ML system for FLORE. (ii) Instead of designing from scratch, we utilize the analysis results as a guideline: FLORE is purposefully modeled as an invertible solver to separate and compensate for the lost information from orthogonal subspaces. (iii) Since it is nearly impossible to collect ground-truth (GT) labels [11], we demonstrate that FLORE can be trained by accessing only aggregated counters, based on a well-designed Expectation Maximization (EM) algorithm. (iv) FLORE can easily support long-term, lightweight operation for high-speed, high-volume data streams without frequent model changes (see simulations in §??), through various techniques such as stream filtering mechanism and scalable invertible neural network (INN) architecture.

Contributions. We summarize our contributions in this work as follows:

- We dissect existing sketch techniques from a perspective of matrix analysis based on CS theory. As a lesson, we reveal the orthogonal irrecoverability that precludes perfect recovery from compact summary of the data stream.
- We investigate the feasibility of applying GMs to enhance data sketching. By testing representative GM families, we find that, except for FGMs, they either scale poorly or fall short in the desired efficiency or accuracy.
- We design a new linear sketching solution, FLORE, that efficiently utilizes the distribution modeling capability of FGMs to embrace our analyses seamlessly. First, FLORE theoretically supports faithful recovery, even without knowledge about the GT data. Second, FLORE is scalable and generic to real-world applications. To our best knowledge, the deep generative stream processing paradigm has never been explored before.
- We conduct a comprehensive evaluation of FLORE on streams simulated using 10 datasets with diverse sizes and patterns. The results demonstrate FLORE outperforms 10 state-of-the-art (SOTA) solutions, achieving up to a $10^3 \times$ reduction in per-element frequency estimation error, a $10^2 \times$ improvement in accurately estimating entropy and distributions, and up to an 80+ point gain in heavy-hitter detection. It also attains a higher sketching throughput than prior art.

Organization. The problem formulation and preliminaries are presented in §2, followed by the analysis of existing sketch techniques in §3. Next, we benchmark mainstream generative models in §4, then present our method and system design in §5. Finally, §6 conducts simulations and reports the experimental results.

2 Preliminaries

2.1 Problem Formulation

A data stream is a sequence of n tuples arriving in order. The t -th tuple is denoted as (key_t, v_t) , where $key_t \in \mathcal{I}$ is a data-item key used for hashing and v_t is a value associated with the item. These n tuples can have duplicate keys, but algorithms running on the data stream must be *one-pass*. Sketch techniques utilize a compact data structure to

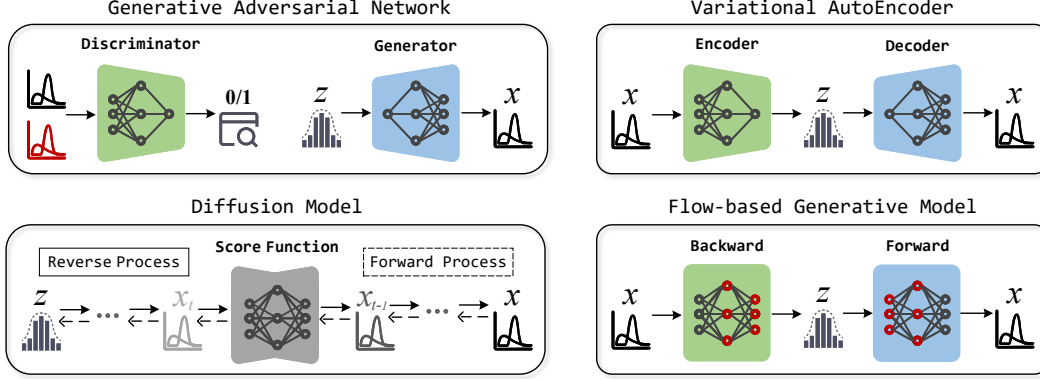


Figure 3: Overview of four types of GMs considered in this work.

summarize each incoming item with constant-time complexity, e.g., $\mathcal{O}(k)$ when applying k independent hash functions to (flattened) counters $b \in \mathbb{R}^m$. Upon receiving a query, the goal is to recover the required statistics from the sketch summary, most notably frequencies $f \in \mathbb{C}^N$ in this paper.

Tailored problem. Our study focuses on the *linear sketching* problem, where the summary is defined to have a linear part such that each counter is updated linearly by incoming items. If we consider f to be sparse or heavy-tailed, a property justified and utilized in many prior works [12, 13], the problem falls into a classical CS framework: $\min_{\Phi f=b} \|f\|_1$, where Φ is an indicator matrix produced by multiple hashing operations, and its columns are presented one by one over time. In Figure 2, we take the Count-Min Sketch as an example to illustrate the linear matrix formulation with two procedures: summary and recovery. In this setting, $\Phi_{i,j} = 1$ if and only if key_j is hashed to counter i ; otherwise, $\Phi_{i,j} = 0$. This problem is NP-hard [14] and often highly ill-posed, i.e., $m \ll N$.

◇ **Note:** We assume that the possible keys set $\mathcal{I}$ is known at the start of data streams, following the recent study [15, 16]. In fact, the online collection of $\mathcal{I}$ can be easily achieved by employing a Bloom Filter as detailed in [17, 18, 19] (see §5.7 also). For untracked keys, we can report zeros as their final estimates^a, and extend our formulation to $\min_{\Phi f=b+\epsilon} \|f\|_1$ where ϵ captures the error induced by them. We will soon show in §6 that the considered framework maintains overall accuracy under incomplete $\mathcal{I}$.

^aAamand et al.[20] proved that truncating low frequency estimates to zeros will not impact the sketch’s overall accuracy.

2.2 Generative Models

Deep generative models are a class of ML models designed to learn the underlying distribution of data and generate new samples from it. The key idea of them is to develop a latent vector space, where any point satisfies a fixed prior distribution, e.g., an isotropic Gaussian $\mathcal{N}(0, \mathbf{I})$, can be mapped to a “valid” sample.

There is a range of different strategies for learning such latent spaces, each with its own characteristics. As illustrated in Figure 3, the most common types of GMs are ① Variational Autoencoders (VAEs) [21], which learn probabilistic latent representations by optimizing a variational lower bound; ② Generative Adversarial Networks (GANs) [22], which synthesize data through an adversarial game between a generator and a discriminator; ③ Diffusion Models (DMs) [23], which generate data by iteratively reversing a noise injection process; and ④ Flow-based Generative Models (FGMs) [24], which establish exact and invertible mappings between data and latent variables, enabling tractable likelihood evaluation and efficient sampling under suitable architectural constraints. In this work, their underlying principles are explained in detail in Appendix A, and we will restrict our analysis to these four GMs.

2.3 Notations & Definitions

Notations. In Table 1, we list the notational conventions used in this paper.

Definition 1. (Support) For a vector $\mathbf{x} \in \mathbb{R}^n$, the set of all indices of non-zero elements in $\mathbf{x}$ is called the support of x , denoted as $\text{supp}(\mathbf{x})$: $\text{supp}(\mathbf{x}) = \{i : \mathbf{x}_i \neq 0\}$.

Definition 2. (Sparsity) The sparsity of $\mathbf{x}$ = the number of non-zero element in $\mathbf{x}$ = the cardinality of $\text{supp}(\mathbf{x})$. It can be denoted as $|\text{supp}(\mathbf{x})|$ or $\|\mathbf{x}\|_0$. A vector x is s -sparse if $\|\mathbf{x}\|_0 \leq s$.

Table 1: List of Main Symbols

Symbol	Description	Symbol	Description
f	Ground-truth (frequency) vector	Φ	Linear sketching matrix
b	Measurement (counters) vector	k	Number of hash functions in sketch
z	Prior (Gaussian) vector	N	Number of distinct items
$\mathcal{I}$	Key set of data stream	m	Number of total counters in sketch
T_i	i -th invertible transformation function	(ε, ς)	Coefficient & error probability of sketch
ϵ	Error introduced by missing elements	m_{bf}	Number of bits in Bloom Filter
s	Non-zero count upper bound	k_{bf}	Number of hash functions in Bloom Filter
$p_B(\cdot)$	Distribution of counters	$p_Z(\cdot)$	Distribution of priors
$p_F(\cdot)$	Distribution of ground truth	$q(\cdot)$	Joint distribution of counters and priors

Definition 3. (*s-term Approximation*) For $p > 0$, the ℓ_p -error of best s -term approximation to a vector $\mathbf{x} \in \mathbb{C}^N$ is defined by $\sigma_s(\mathbf{x})_p := \inf \left\{ \|\mathbf{x} - \mathbf{z}\|_p, \mathbf{z} \in \mathbb{C}^N \text{ is } s\text{-sparse} \right\}$.

Definition 4. (*Restricted Isometry Constant*) The s -th restricted isometry constant $\delta_s = \delta_s(\mathbf{A})$ of a matrix $\mathbf{A} \in \mathbb{C}^{m \times N}$ is the smallest number $\delta_s \geq 0$, such that $(1 - \delta_s)\|\mathbf{x}\|_2^2 \leq \|\mathbf{A}\mathbf{x}\|_2^2 \leq (1 + \delta_s)\|\mathbf{x}\|_2^2$ holds for all s -sparse vectors $\mathbf{x} \in \mathbb{C}^N$. Equivalently, it is given by $\delta_s = \max_{S \subseteq [N], \text{card}(S) \leq s} \|\mathbf{A}_S^* \mathbf{A}_S - \mathbf{Id}\|_{2 \rightarrow 2}$, where $\mathbf{A}_S^*$ is the hermitian transpose of $\mathbf{A}_S$ and $\text{card}(S)$ is the cardinality of set S .

Definition 5. (*Kullback–Leibler Divergence*) The Kullback–Leibler (KL) divergence is a type of statistical distance: a measure of how much an approximating probability distribution q is different from a true probability distribution p . Mathematically, it is defined as $\mathcal{D}_{\text{KL}}(p \parallel q) = \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)}$. Obviously, $\mathcal{D}_{\text{KL}}(p \parallel q) \geq 0$ and equals zero if and only if $p = q$ as measures.

Definition 6. (*Maximum Mean Discrepancy*) Let x and y be random variables defined on a topological space X , with respective Borel probability measures p and q . And let $\mathcal{F}$ be a class of functions $F : \mathcal{X} \rightarrow \mathbb{R}$. We define the maximum mean discrepancy (MMD) as $\text{MMD}[\mathcal{F}, p, q] := \sup_{F \in \mathcal{F}} (\mathbb{E}_x[F(x)] - \mathbb{E}_y[F(y)])$.

3 Motivating Analyses

3.1 Limitations of Existing Techniques (Proposition 1 & Theorem 1)

(T1) Randomized approximation. Typically, $\hat{f}_i$ is carried out by extracting information associated with key_i from the sketch summary, which means each element’s estimate is computed individually. Representative sketch techniques include Count-Min Sketch (CM) [25] and Count Sketch (CS²) [26]. Essentially, their recovery of $\hat{f}_i$ is based on observing only k entries of Φ , ignoring the influence of all other keys. While their processes are very fast (requiring only $\mathcal{O}(k)$ time) and aggregation by minimum or mean suppresses partial noise, the per-element accuracy, e.g., $\varepsilon\|f\|_1$ for CM and $\varepsilon\|f\|_2$ for CS where ε denotes the sketch coefficient, is heavily constrained by the space budget, i.e., the number of rows in Φ .

(T2) Disentangled augmentation. The second class splits the sketch into two smaller components: a heavy part for frequent elements and a light part for infrequent elements. The Augmented Sketch (AG) [12] and Elastic Sketch (ES) [27] both fall into this category. With $\|f\| \downarrow$ after stream filtering, this design more effectively handles skewed distributions by mitigating hash collisions. From our perspective, such methods can be interpreted as a decomposition of the sketching matrix Φ , which allocates nonzero entries into separate components. By clearing columns of Φ , the degree of underdetermination is thus decreased. However, when resources are constrained, the capacity of the heavy part is limited. Additionally, the light part continues to suffer from large error bounds across all algorithms, which becomes even worse under a fixed budget.

In particular, the learning-augmented sketches [28, 29], a growing body of research closely related to ours, attempt to optimize partitioning by learning an “oracle” (i.e., a classifier). However, they inherit the aforementioned limitations and are often not system-friendly, which brings painstaking deployment overhead, as performing a neural network forward pass before hashing operations each time may result in substantial stream processing efficiency degradation for various scenarios.

²This abbreviation should be distinguished from the compressive sensing (CS)

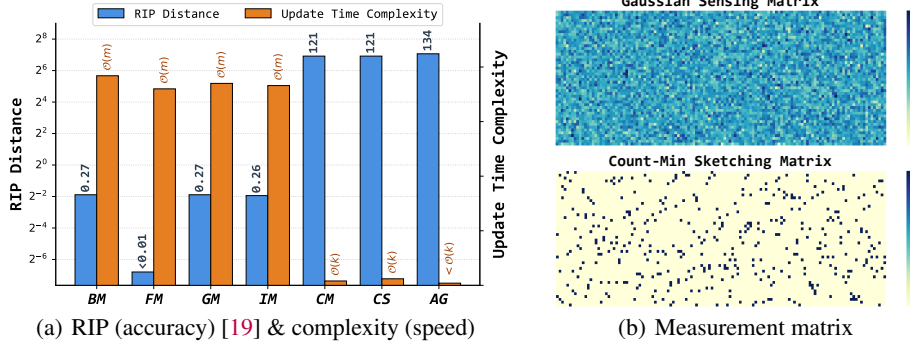


Figure 4: Compare compressive sensing with sketch techniques.

(T3) Compressive sensing. Recently, the sketch techniques have begun to compute estimates for all elements simultaneously rather than independently. They usually decode all information by solving a system of linear equations using CS algorithms. Among these sketches, the PR-sketch (PR) [17] and NZE-sketch (NZE) [19] represent the most mature examples. Before proceeding with our analysis, we need to introduce a notion called restricted isometry property (RIP) [30], which characterizes how well Φ preserves the norm of sparse signals. Since Φ must operate uniformly over all sparse vectors, this property is quantified by a sequence of restricted isometry constants (RICs), denoted by $\{\delta_s\}$, where $\delta_s = \sup \left\{ \frac{\|\Phi x\|_2 - \|x\|_2}{\|x\|_2} : x \text{ is an } s\text{-sparse vector} \right\}$. We say that Φ satisfies the RIP if δ_s is small for reasonably large s . Under this condition, the following error bound of CS-based sketch techniques holds:

Proposition 1. Suppose that for $p > 0$, the error of best s -term approximation is denoted by $\sigma_s(f)_p := \inf_{\|z\|_0 \leq s} \|f - z\|_p$. Then, for any vector $f \in \mathbb{C}^N$, a CS solution f^* with $b = \Phi f + \epsilon$ and $\|\epsilon\|_2 \leq \eta$ approximates f with ℓ_2 -error

$$\|f^* - f\|_2 \leq C s^{-1/2} \sigma_s(f)_1 + 2C\eta \quad (1)$$

where the constant $C = \frac{2(1+\rho)}{1-\rho}$ and $\rho \propto \delta_{2s}$.

Proof. We first partition the vector f into a sequence of vectors $f_{\mathcal{T}_k}$ based on the magnitudes of its entries. Specifically, let $f_{\mathcal{T}_0}$ retain the s largest entries of f in magnitude (with all others set to 0), $f_{\mathcal{T}_1}$ retain the next s largest entries, and so forth. Given $f = f^* + h$, at a high level, our proof separately provides upper bounds for $\|h_{\mathcal{T}_0 \cup 1}\|_2$ and $\|h_{\mathcal{T}_0^c \cup 1}\|_2$, thereby bounding our target $\|f^* - f\|_2 = \|h\|_2$ (i.e., the sum of these two terms). According to the above definition, we can write

$$\|h_{\mathcal{T}_j}\|_2 = \sqrt{\sum_i h_{\mathcal{T}_j}^2(i)} \leq \sqrt{s \max_i h_{\mathcal{T}_j}^2(i)} \leq \sqrt{\frac{1}{s} \sum_i h_{\mathcal{T}_{j-1}}(i)} = \sqrt{\frac{1}{s}} \|h_{\mathcal{T}_{j-1}}\|_1,$$

which holds for $j \geq 2$. Based on this, we have $\sum_{j \geq 2} \|h_{\mathcal{T}_j}\|_2 \leq \frac{1}{\sqrt{s}} \sum_{j \geq 1} \|h_{\mathcal{T}_j}\|_1 = \frac{1}{\sqrt{s}} \|h_{\mathcal{T}_0^c}\|_1$. Then the following useful inequality can be further derived:

$$\|h_{\mathcal{T}_0^c \cup 1}\|_2 = \left\| \sum_{j \geq 2} h_{\mathcal{T}_j} \right\|_2 \leq \sum_{j \geq 2} \|h_{\mathcal{T}_j}\|_2 \leq \frac{1}{\sqrt{s}} \sum_{j \geq 1} \|h_{\mathcal{T}_j}\|_1 = \frac{1}{\sqrt{s}} \|h_{\mathcal{T}_0^c}\|_1 \quad (2)$$

Since f^* minimizes $\|f^*\|_1$ subject to the constraint $\Phi f^* = \Phi f + \epsilon$, i.e., $\min_{\Phi f^* = b} \|f^*\|_p$, it is intuitively clear that $\|f^*\|_1$ cannot exceed $\|f\|_1$. Hence, we have $\|f\|_1 \geq \|f^*\|_1$, which leads to

$$\begin{aligned} \|f_{\mathcal{T}_0}\|_1 + \|f_{\mathcal{T}_0^c}\|_1 &= \|f\|_1 \geq \|f^*\|_1 = \|f - h\|_1 = \|(f - h)_{\mathcal{T}_0}\|_1 + \|(f - h)_{\mathcal{T}_0^c}\|_1 \\ &\geq \|f_{\mathcal{T}_0}\|_1 - \|h_{\mathcal{T}_0}\|_1 + \|h_{\mathcal{T}_0^c}\|_1 - \|f_{\mathcal{T}_0^c}\|_1. \end{aligned}$$

To this end, we get $\|h_{\mathcal{T}_0^c}\|_1 \leq 2\|f_{\mathcal{T}_0^c}\|_1 + \|h_{\mathcal{T}_0}\|_1$. Combining with Ineq. (2), it gives the first upper bound

$$\|h_{\mathcal{T}_0^c \cup 1}\|_2 \leq \frac{1}{\sqrt{s}} \|h_{\mathcal{T}_0^c}\|_1 \leq \frac{1}{\sqrt{s}} \|h_{\mathcal{T}_0}\|_1 + \frac{2}{\sqrt{s}} \|f_{\mathcal{T}_0^c}\|_1 \quad (3)$$

We now continue to derive an upper bound for the other term, $\|h_{\mathcal{T}_{0 \cup 1}}\|_2$. By applying the Cauchy–Schwarz inequality, we obtain the following inequality:

$$\begin{aligned}\|\Phi(f^* - f)\|_2^2 &= \langle \Phi f^* - b + b - \Phi f, \Phi f^* - b + b - \Phi f \rangle \\ &= \|\Phi f^* - b\|_2^2 + \|b - \Phi f\|_2^2 + 2|\langle \Phi f^* - b, b - \Phi f \rangle| \\ &\leq \|\Phi f^* - b\|_2^2 + \|b - \Phi f\|_2^2 + 2\|\Phi f^* - b\|_2 \|b - \Phi f\|_2 \\ &= (\|\Phi f^* - b\|_2 + \|b - \Phi f\|_2)^2\end{aligned}$$

Therefore,

$$\|\Phi h\|_2 = \|\Phi(f^* - f)\|_2 \leq \|\Phi f^* - b\|_2 + \|b - \Phi f\|_2 \leq 2\|\epsilon\|_2 \leq 2\eta$$

Here we again apply the Cauchy–Schwarz inequality together with Definition 4 to get:

$$|\langle \Phi h_{\mathcal{T}_{0 \cup 1}}, \Phi h \rangle| \leq \|\Phi h_{\mathcal{T}_{0 \cup 1}}\|_2 \|\Phi h\|_2 \leq 2\eta\sqrt{1 + \delta_{2s}} \|h_{\mathcal{T}_{0 \cup 1}}\|_2$$

By Lemma 1, we have

$$\begin{cases} |\langle \Phi h_{\mathcal{T}_0}, \Phi h_{\mathcal{T}_j} \rangle| \leq \delta_{2s} \|h_{\mathcal{T}_0}\|_2 \|h_{\mathcal{T}_j}\|_2, & \forall j \in \{1, 2, \dots\}, \\ |\langle \Phi h_{\mathcal{T}_1}, \Phi h_{\mathcal{T}_j} \rangle| \leq \delta_{2s} \|h_{\mathcal{T}_1}\|_2 \|h_{\mathcal{T}_j}\|_2, & \forall j \in \{0, 2, \dots\}. \end{cases}$$

Thus, using the inequality $\|h_{\mathcal{T}_0}\|_2 + \|h_{\mathcal{T}_1}\|_2 \leq \sqrt{2} \|h_{\mathcal{T}_{0 \cup 1}}\|_2$, we obtain

$$\begin{aligned}\|\Phi h_{\mathcal{T}_{0 \cup 1}}\|_2^2 &= \langle \Phi h_{\mathcal{T}_{0 \cup 1}}, \Phi l \rangle - \left\langle \Phi h_{\mathcal{T}_{0 \cup 1}}, \sum_{j \geq 2} \Phi h_{\mathcal{T}_j} \right\rangle \\ &\leq 2\eta\sqrt{1 + \delta_{2s}} \|h_{\mathcal{T}_{0 \cup 1}}\|_2 - \left\langle \Phi h_{\mathcal{T}_0} + \Phi h_{\mathcal{T}_1}, \sum_{j \geq 2} \Phi h_{\mathcal{T}_j} \right\rangle \\ &\leq 2\eta\sqrt{1 + \delta_{2s}} \|h_{\mathcal{T}_{0 \cup 1}}\|_2 + \delta_{2s} (\|h_{\mathcal{T}_0}\|_2 + \|h_{\mathcal{T}_1}\|_2) \sum_{j \geq 2} \|h_{\mathcal{T}_j}\|_2 \\ &\leq 2\eta\sqrt{1 + \delta_{2s}} \|h_{\mathcal{T}_{0 \cup 1}}\|_2 + \sqrt{2}\delta_{2s} \|h_{\mathcal{T}_{0 \cup 1}}\|_2 \sum_{j \geq 2} \|h_{\mathcal{T}_j}\|_2.\end{aligned}$$

Moreover, since $(1 - \delta_{2s}) \|h_{\mathcal{T}_{0 \cup 1}}\|_2^2 \leq \|\Phi h_{\mathcal{T}_{0 \cup 1}}\|_2^2$, revisiting Ineq. (2) yields:

$$\|h_{\mathcal{T}_{0 \cup 1}}\|_2 \leq \frac{2\sqrt{1 + \delta_{2s}}}{1 - \delta_{2s}} \eta + \frac{\sqrt{2}\delta_{2s}}{1 - \delta_{2s}} \sum_{j \geq 2} \|h_{\mathcal{T}_j}\|_2 \leq \frac{2\sqrt{1 + \delta_{2s}}}{1 - \delta_{2s}} \eta + \frac{\sqrt{2}\delta_{2s}}{\sqrt{s}(1 - \delta_{2s})} \|h_{\mathcal{T}_0^c}\|_1$$

Combining this further with inequality $\|h_{\mathcal{T}_0^c}\|_1 \leq 2\|f_{\mathcal{T}_0^c}\|_1 + \|h_{\mathcal{T}_0}\|_1$ established earlier, we have

$$\|h_{\mathcal{T}_{0 \cup 1}}\|_2 \leq \frac{2\sqrt{1 + \delta_{2s}}}{1 - \delta_{2s}} \eta + \frac{\sqrt{2}\delta_{2s}}{\sqrt{s}(1 - \delta_{2s})} \|h_{\mathcal{T}_0}\|_1 + \frac{2\sqrt{2}\delta_{2s}}{\sqrt{s}(1 - \delta_{2s})} \|f_{\mathcal{T}_0^c}\|_1.$$

It should also be noted that $\|h_{\mathcal{T}_0}\|_1 \leq \sqrt{s} \|h_{\mathcal{T}_0}\|_2 \leq \sqrt{s} \|h_{\mathcal{T}_{0 \cup 1}}\|_2$, which also follows from the Cauchy–Schwarz inequality, as shown in the following derivation:

$$\frac{1}{\sqrt{s}} \|h_{\mathcal{T}_0}\|_1 = \sqrt{\left(\sum_i \frac{1}{\sqrt{s}} |h_{\mathcal{T}_0}(i)| \right)^2} \leq \sqrt{\left(\sum_{j=1}^s \frac{1}{s} \right) \sum_i h_{\mathcal{T}_0}^2(i)} = \|h_{\mathcal{T}_0}\|_2 \leq \|h_{\mathcal{T}_{0 \cup 1}}\|_2$$

Thus, further, we have

$$\begin{aligned}\|h_{\mathcal{T}_{0 \cup 1}}\|_2 &\leq \frac{2\sqrt{1 + \delta_{2s}}}{1 - \delta_{2s}} \eta + \frac{\sqrt{2}\delta_{2s}}{1 - \delta_{2s}} \|h_{\mathcal{T}_{0 \cup 1}}\|_2 + \frac{2\sqrt{2}\delta_{2s}}{\sqrt{s}(1 - \delta_{2s})} \|f_{\mathcal{T}_0^c}\|_1 \\ &\leq \frac{2\sqrt{1 + \delta_{2s}}}{1 - (\sqrt{2} + 1)\delta_{2s}} \eta + \frac{2\sqrt{2}\delta_{2s}}{\sqrt{s} - \sqrt{s}(\sqrt{2} + 1)\delta_{2s}} \|f_{\mathcal{T}_0^c}\|_1\end{aligned}\tag{4}$$

Now using the first upper bound as in Ineq. (3), we can write

$$\|f^* - f\|_2 = \|h\|_2 \leq \|h_{T_{0 \cup 1}}\|_2 + \|h_{T_{0 \cup 1}^c}\|_2 \leq 2\|h_{T_{0 \cup 1}}\|_2 + \frac{2}{\sqrt{s}} \|f_{T_0^c}\|_1.$$

Finally, Substituting into the Ineq. (4), we see that

$$\|f^* - f\|_2 = \|h\|_2 \leq \frac{2 + (2\sqrt{2} + 2) \delta_{2s}}{\sqrt{s} - \sqrt{s}(\sqrt{2} + 1) \delta_{2s}} \|f_{T_0^c}\|_1 + \frac{4\sqrt{1 + \delta_{2s}}}{1 - (\sqrt{2} + 1) \delta_{2s}} \eta$$

Let $\rho = (\sqrt{2} + 1) \delta_{2s}$; then the above expression can be rewritten as

$$\|f^* - f\|_2 \leq \frac{2(1 + \rho)}{\sqrt{s}(1 - \rho)} \|f_{T_0^c}\|_1 + \frac{4\sqrt{1 + \delta_{2s}}}{1 - \rho} \eta \leq \frac{2(1 + \rho)}{\sqrt{s}(1 - \rho)} \|f_{T_0^c}\|_1 + \frac{4(1 + \rho)}{1 - \rho} \eta,$$

where the second inequality is by Lemma 2. And by Definition 3, $\sigma_s(f)_1$ is the minimum ℓ_1 -error over all s -sparse approximations of f . Since f_{T_0} retains the s largest entries of f in magnitude, $\sigma_s(f)_1 = \|f - f_{T_0}\|_1 = \|f_{T_0^c}\|_1$. Hence, $\|f^* - f\|_2 \leq C s^{-1/2} \sigma_s(f)_1 + 2C\eta$, where $C = \frac{2(1+\rho)}{1-\rho}$. This concludes the proof. $\square$

Proposition 1 indicates that their per-element accuracy is bounded on the order of around $\mathcal{O}(C s^{-1/2} \sigma_s(f)_1 / N)$, which is tighter than the bounds of previous techniques. However, this comes at a greatly increased query cost [31] and does not imply better estimates in practice. Since the coefficient C is positively correlated with RICs, we measure the RIP distance (i.e., the difference between $\|\Phi f\|_2$ and $\|f\|_2$) of the classical CS matrices [32, 33], i.e., Bernoulli Matrix (BM), Fourier Matrix (FM), Gaussian Matrix (GM), Incoherence Matrix (IM), and the matrices induced by sketch techniques. In Figure 4(a) (borrowed from [19]), we can see that all three sketch techniques suffer from extremely high RIP distance (> 120), while CS matrices achieve much smaller ones (< 0.3). This discrepancy stems from the sparse characteristic of real-world sketching matrices, as visualized in Figure 4(b), because only updating a small number, e.g., $\mathcal{O}(k)$, of counters each time can keep pace with high-speed data streams. Thus, CS is still not enough for efficiently solving the problem. Theorem 1 formally shows that sketching matrices (e.g., CM) require substantially more rows (or counters here) that optimal RIP matrices.

Theorem 1. Let $f \in \mathbb{C}^N$ be s -sparse and Φ be an $m \times N$ matrix that satisfies RIP for perfect recovery of the vector f . Then, the necessary number of counters

$$m \geq \begin{cases} \mathcal{O}(s \log(N/s)), & \text{if } \Phi \text{ is random dense,} \\ \mathcal{O}(s^2), & \text{if } \Phi \text{ is 0-1 sparse.} \end{cases} \quad (5)$$

Proof. Clearly, our proof is divided into two parts, establishing the lower bounds for the 0-1 sparse and random dense cases separately. Without loss of generality, we further emphasize that these two cases specifically refer to the count-min and Gaussian settings; that is, in the 0-1 matrix, each column contains exactly k (determined by the number of hash functions) 1's. Before we prove the first case, we need to introduce the following variant of RIP: there exists $c > 0$ such that for all s -sparse vector $f \in \mathbb{R}^N$, $c\|\Phi f\|_2 \leq \|f\|_2 \leq cD\|\Phi f\|_2$.

① **Φ is 0-1 sparse matrix.** To start, we square the new RIP inequality to obtain $c^2\|\Phi f\|_2^2 \leq \|f\|_2^2 \leq c^2 D^2 \|\Phi f\|_2^2$. We can bound c by considering the 1-sparse vector $f = e_i$, where e_i is a standard basis vector with a 1 in a coordinate corresponding to a column of Φ with k 1's. Then, the inequalities above give $c^2 k \leq 1 \leq c^2 D^2 k$. We will only need the right inequality, which we rewrite as $\frac{1}{c^2} \leq D^2 k$.

Now, consider the vector f with 1's in the first s positions and 0's elsewhere, i.e., $f = \sum_{i=1}^s e_i$. Let w_i denote the number of 1's in row i and in the first s columns of Φ . Then, since the first s columns of Φ contain ks 1's, we can write $\sum_i w_i \geq ks$. Applying the Cauchy-Schwarz inequality, we obtain

$$\|\Phi f\|_2^2 = \sum_{i=1}^m w_i^2 \geq \frac{(\sum_{i=1}^m w_i)^2}{m} \geq \frac{(ks)^2}{m}, \text{ but } \|\Phi f\|_2^2 \leq \frac{\|f\|_2^2}{c^2} \leq D^2 ks,$$

so putting the two inequalities together gives $\frac{(ks)^2}{m} \leq D^2 ks$. Cancelling out ks from both sides gives $\frac{k}{m} \leq \frac{D^2}{s}$. Also, let r be the maximum number of 1's in any row of Φ . By permuting the rows and columns of Φ , we may assume that $\Phi_{1,i} = 1$ for $1 \leq i \leq r$, i.e., that the first row of Φ has a 1 in the first r entries. In the following proof, we assume that this reordering has already been done. We can now obtain a second bound on $\frac{k}{m}$ from the obvious inequality: $kN \leq \text{number of 1's in } \Phi \leq rm$, and thus, $\frac{k}{m} \leq \frac{r}{n}$.

For notational convenience, let $\tau = \min\{r, s\}$. Consider the vector f with 1's in the first τ positions and 0's elsewhere, i.e., $f = \sum_{i=1}^{\tau} e_i$. For this choice of f , we see that $\|\Phi f\|_2^2 \geq \tau^2$ because the first entry of Φf is τ . Note that because $\tau \leq s$, the inequalities defining the new RIP apply to f . Thus, we can apply $\frac{1}{c^2} \leq D^2 k$ to get $\tau^2 \leq \|\Phi f\|_2^2 \leq \frac{\|f\|_2^2}{c^2} \leq D^2 k \tau$.

We now plug in our two bounds on $\frac{k}{m}$. If $r > s$, then $\tau = s$, and we use the bound $\frac{k}{m} \leq \frac{D^2}{s}$. This gives $s^2 \leq D^2 \frac{D^2}{s} m s$, so $m \geq \frac{s^2}{D^4}$. Similarly, if $r \leq s$, then $\tau = r$, so using our second bound on $\frac{k}{m}$ gives $r^2 \leq D^2 \frac{r}{N} m r$. Therefore, in this case $m \geq \frac{N}{D^2}$. Putting the two bounds together, $m \geq \min\left\{\frac{s^2}{D^4}, \frac{N}{D^2}\right\}$, or we can say that $m \geq \mathcal{O}(s^2)$.

② **Φ is random dense matrix.** Based on Lemma 3, we consider the asymptotic regime where $N, s, m \rightarrow \infty$. The left-hand side of Ineq. (25) satisfies $\frac{m^2}{m+1} \approx m$ for large m . On the right-hand side, since N/s is large, we have $\ln(2.34N/s) = \ln(N/s) + O(1) \approx \ln(N/s)$. Furthermore, as s grows, the term $\frac{\ln(\epsilon^{-1})}{s}$ becomes negligible and vanishes in the limit. Substituting these approximations back into the inequality yields $m \gtrsim 2s(\sqrt{\ln(N/s)})^2$, which simplifies to the lower bound $m \geq 2s \ln(N/s)$, or we can say that $\mathcal{O}(s \log(N/s))$, ending the proof. $\square$

3.2 The Anatomy of Failure (Proposition 2)

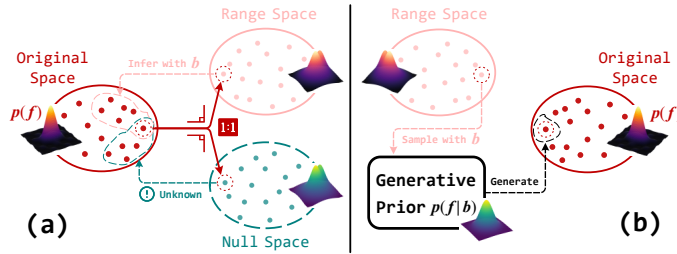


Figure 5: **Illustration of our insights.** (a) The GT f cannot be recovered due to orthogonal information loss in the null space. (b) We leverage GMs to generate the null-space component which is in harmony with the range-space component.

Acknowledging the intrinsic limitations of existing techniques, we next analyze the root causes of their poor performance. Our methodology is to examine whether compressed b admits perfect recovery of f , and range-null space decomposition is at the core of our motivation. Here we first introduce the definitions of null space and range space. A vector f_N is said to lie in the null space of Φ if and only if $\Phi f_N = 0$, and f_Φ lies in the range space of Φ if and only if $\Phi f_\Phi = b$. See related works [16, 34] for more details. For any linear system, an arbitrary signal f can be decomposed into two parts: $f = f_\Phi + f_N$. This decomposition can be obtained in various ways, e.g., the singular value decomposition (SVD) used below. In the context of linear sketching problem in data streams, however, this unfortunately leads to the following negative result.

Proposition 2. *The sketch summary b captures only the range-space component f_Φ , while the null-space component f_N is orthogonal to f_Φ and irrecoverable from b .*

Proof. Before decomposing the vector f , we first perform a singular value decomposition (SVD) of the sketching matrix Φ : $\Phi = U\Sigma V^\top$. Here, $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{N \times N}$ are orthogonal matrices, and $\Sigma \in \mathbb{R}^{m \times N}$ is a rectangular diagonal matrix whose diagonal entries are the singular values of Φ , arranged in descending order. Let the rank of Φ be r . According to the properties of SVD, the orthogonal matrix V can be written as $V = [\psi, \phi]$, where $\psi = \psi_1, \psi_2, \dots, \psi_r \in \mathbb{R}^{N \times r}$ forms an orthonormal basis for the range-space of Φ , and $\phi = \phi_1, \phi_2, \dots, \phi_{N-r} \in \mathbb{R}^{N \times (N-r)}$ forms an orthonormal basis for the null-space of Φ .

Now define $f_\Phi = \psi\psi^\top f$ and $f_N = \phi\phi^\top f$. Note that

$$f_\Phi + f_N = \psi\psi^\top f + \phi\phi^\top f = (\psi\psi^\top + \phi\phi^\top)f = VV^\top f = f$$

Hence, f can be decomposed into two components, f_Φ and f_N . Since $f = f_\Phi + f_N$, left-multiplying both sides by Φ yields $\Phi f = \Phi f_\Phi + \Phi f_N$. And because $f_N = \phi\phi^\top f$ and ϕ is a basis of the null-space of Φ , we have $\Phi f_N = \Phi\phi\phi^\top f = 0$, and therefore $\Phi f_\Phi = \Phi f = b$. Thus, f_Φ and f_N correspond to the components of f lying in the range-space and the null-space of Φ , respectively. Moreover,

$$f_\Phi^\top f_N = (\phi\phi^\top f)^\top, \quad \phi\phi^\top f = f^\top \psi(\psi^\top \phi)\phi^\top f = 0$$

Therefore, f_Φ and f_N are orthogonal and uncorrelated, which completes the proof. $\square$

Figure 5(a) illustrates the implication of Proposition 2, which we term *orthogonal irrecoverability*. To guarantee exact recovery, it basically requires that all f_N are nearly-zero vectors, as they are entirely discarded during compression. This requirement is especially hard to meet for sparse matrices induced by sketch techniques, motivating the usage of external prior knowledge to compensate for the missing information, which we will get to later.

4 Toward Generative Linear Sketching

4.1 From Lessons to Motivation

On generative optimization and why it might do better. As outlined in §3, relying solely on the summary b is insufficient to achieve nearly-zero-error data sketching. To break the “curse of compression”, we seek solutions that conform to two critical constraints: *consistency* and *fidelity*. For the consistency constraint, we can simply resort to the range-space $\Phi f \equiv b$. For the fidelity constraint, as discussed in §3.1, while the sparsity assumption of f is often natural and makes sense in CS theory, it is neither necessary nor sufficient for our goal. An enhanced approach, which avoids presuppositions regarding f and also iterative optimizations of CS, is training a generator on recent realizations of the data stream to directly produce samples that closely follow f ’s distribution, thereby effectively “infilling” the null space. In Figure 5(b), we illustrate the key idea of which we refer to *generative prior* through the maximization of the likelihood $\log p(f | b)$ via sampling. It can outperform existing techniques in scenarios where streaming data is volatile and hard to predict under limited memory budgets. Directly inferring f also obviates the need for solving a lengthy linear program, which significantly accelerates runtimes for large-scale data streams (see §6.6.1). The pre-trained generator is expected to learn the optimum in Figure 5(b). Indeed, our proposed approach, which we will get to later, quickly converges to this outcome.

Applicability of ISR to sketch. For computer vision, image super-resolution (ISR) is a similar and well-studied task [35]. Recent advances [36, 37, 38, 39, 40] in generative AI for ISR also give us hope, but intuitively, not every ISR algorithm makes sense for data sketching, due to fundamentally different domains of data (super-high-dimensional counters v.s. grids of pixels). In addition, translating the measurement system from concept into practice is an open frontier yet, due to a host of challenges such as evolving data streams and training data scarcity. Thus, rather than simply porting ISR into a new domain, this work aims to pioneer a deployable generative sketching paradigm.

4.2 Integrating Generative Priors (Theorem 2)

There are two ways to incorporate distributional prior information into our algorithm design. The first is a straightforward strategy that adds distributional regularization during training, such as Dirichlet process [41] and Zipf’s law [42]. For example, the mapping $b \rightarrow f$ may be pre-trained on Zipfian datasets exhibiting similar skewness [11], or f may be constrained to satisfy prescribed structure [43]. However, this way has a clear shortcoming: the manually imposed, deterministic constraints are often overly restrictive, resulting in a narrow range of applicability. Therefore, here it is not a good fit.

In this work, we adopt the second way to build a generative mapping between $[b, z]$ and f , where z is a latent vector drawn from a gaussian distribution. Unlike the previous strategy, the latent variable z explicitly introduces external information that can be actively leveraged for diverse generation. Moreover, for the guarantee of learning the mapping unbiasedly, we prove the following theorem:

Theorem 2. *For a sketching matrix $\Phi \in \mathbb{R}^{m \times N}$ of rank r , ground-truth vector $f \in \mathbb{C}^N$ and $b = \Phi f$, there exists a gaussian vector $z \in \mathbb{R}^{N-r}$ and learnable mapping $G : \mathbb{R}^{m+N-r} \rightarrow \mathbb{R}^N$ such that $G([b, z])$ uniquely determines f .*

Proof. This proof continues the proof of Proposition 2; thus, there is conceptual overlap, and unless otherwise stated, the used notation is consistent with the preceding discussion.

We first notice that $b = \Phi f_\Phi = U \Sigma V^\top f_\Phi$. Recalling that $V^\top = [\psi, \phi]$, and using the facts that $\psi^\top \psi = \text{Id}$ and $\phi^\top \phi = 0$, this expression can be further expanded as $\begin{pmatrix} \Sigma_{\leq r}^{-1} U_{\leq r}^\top b \\ \mathbf{0} \end{pmatrix} = V^\top f_\Phi$, where $\Sigma_{\leq r} \in \mathbb{R}^{r \times r}$ is the diagonal matrix consisting of the top r singular values of Φ , $\Sigma_{\leq r}^{-1}$ is its inverse, $U_{\leq r}^\top \in \mathbb{R}^{r \times m}$ denotes the first r rows of $U^\top$, and $\mathbf{0}$ is the zero vector. Since $V^\top$ is full rank, there exists a one-to-one correspondence between f_Φ and $\bar{b} = U_{\leq r}^\top b$.

Next, we show that b and $\bar{b}$ are also in one-to-one correspondence. Analogous to V , we can rewrite $U^\top$ as $U^\top = [\psi', \phi']$, where $\psi' = U_{\leq r}^\top$ forms an orthonormal basis for the column space of Φ , and ϕ' spans the null space of $\Phi^\top$. Since for all possible f , the vector b always lies in the column space of Φ , we have $U^\top b =$

$\begin{pmatrix} \psi' b \\ \mathbf{0} \end{pmatrix} = \begin{pmatrix} \bar{b} \\ \mathbf{0} \end{pmatrix}$. Since $U^\top$ is full rank, there exists a one-to-one correspondence between b and $\bar{b}$. To this end, b is uniquely associated with both $\bar{b}$ and f_Φ . Similarly, left-multiplying the identity $f = f_\Phi + f_N$ by $\phi^\top$ yields $\phi^\top f = \phi^\top f_\Phi + \phi^\top f_N = \phi^\top f_N =: Z \in \mathbb{R}^{N-r}$. We then perform an SVD of $\phi^\top$, denoted by $\phi^\top = U' \Sigma' V'^\top$. Since the rank of $\phi^\top$ is $N - r$, its singular value matrix Σ' contains $N - r$ non-zero singular values. Moreover, because the null space of $\phi^\top$ is spanned by $\text{span}(\psi)$, we have $V' = [\phi, \psi]$. Consequently, the above equation can be rewritten as $\begin{pmatrix} \Sigma'^{-1}_{\leq N-r} U'^\top Z \\ \mathbf{0} \end{pmatrix} = V'^\top f_N$, where $\Sigma'_{\leq N-r}$ consists of the first $N - r$ columns of Σ' , and $\Sigma'^{-1}_{\leq N-r}$ denotes its inverse. Since both V' and U' are full-rank matrices, the above equation establishes a one-to-one correspondence between Z and f_N . Since any fixed vector can be regarded as the mean of some Gaussian distribution, we can efficiently approximate Z using a gaussian vector z . The vector z can be standard gaussian if passed through a single-layer MLP by leveraging Universal Approximation Theorem [44]. Therefore, we can say that the pair $[b, z]$ uniquely determines $f = f_\Phi + f_N$. Equivalently, there exists a learnable mapping $G : \mathbb{R}^{m+N-r} \rightarrow \mathbb{R}^N$ such that $G([b, z]) = f$. $\square$

4.3 On the Choice of GMs

Given the high-level view of the mapping, the next challenge is to maximize the log-likelihood of its outputs. To this end, we consider four classes of deep generative paradigms as detailed in §2.2. Further, to support real-world streaming applications, our model selection is guided by three key principles: (i) The selected GM should support **easy adaptation**, because after prolonged usage, the model inevitably encounters generalization issues, and any subsequent fine-tuning must minimize disruption as much as possible; (ii) The selected GM should support **fast inference**, because sketch recovery and downstream analytics are often performed under limited latency constraints in high-speed data stream processing; and (iii) The selected GM should support **high accuracy & realness**, because faithful reconstruction of the underlying data distribution is essential for reliable estimation and decision-making in measurement-based applications.

4.3.1 Unconditional Distribution Modeling Capability

We start with evaluating the GM’s ability to fit the data stream distribution, as this ability forms the foundation of any downstream generative task. To this end, we construct synthetic data streams drawn from two different distributions. Specifically, *Gaussian* streams are used to assess the models’ generalization ability under smooth and light-tailed distributions, while *Zipfian* streams are adopted to emulate natural data stream characteristics with strong skewness and heavy tails. For both distributions, we vary the streaming data dimensionality, i.e., the number of distinct keys, from 1K to 100K, thereby systematically examining the robustness and scalability of each model across a wide range of stream sizes. Then, we train unconditional GMs on these synthetic streams using standard training protocols. All models adopt lightweight architectures composed of simple linear layers. VAEs, GANs, and FGMs are implemented with conventional adversarial or log-likelihood objectives, while the Wasserstein distance regularization is additionally included to improve adversarial training stability (WGAN) [45]. For DMs, we directly train the underlying network to predict the clean signal from noisy inputs [46, 47], in which 1,000 diffusion and sampling steps is used.

Figure 6 compares the sampling results of these five methods on the two datasets, in which the closer the curve is to the GT curve, the better the distribution fitting capability. The figure shows that all GMs perform very well under the 1K and 10K settings. Among them, FGM and DM exhibit the most accurate fits, whereas VAE and GAN perform slightly worse. Compared to GAN, WGAN yields more confident generations, with its curve consistently exhibiting a sharper peak around the GT mode. However, as the dimensionality of the data stream increases to 50K and 100K, the performance landscape changes markedly. VAE fails to approach the target distribution, and GAN-based methods suffer from severe instability (see 1st row), from which even WGAN cannot escape this “fate”. In contrast, FGM and DM remain robust under high-dimensional settings. Even worse, in heavy-tailed distributions (see 2nd row), the aforementioned methods cannot even capture the overall “shape”. Notably, DM demonstrates superior capability in accurately shaping the underlying distribution. Hence, from the perspective of unconditional distribution learning, FGM and DM emerge as the most suitable candidates.

4.3.2 Conditional Generation for Stream Recovery

Next, we evaluate the transferability of the (unconditional) generative capabilities of different models to downstream data stream (conditional) recovery. To elicit the GM’s intrinsic distribution-shaping capability and minimize the influence of the underlying architecture, we directly extend the unconditional GM to a conditional version. Our approach is based on gradient-based search in the latent space of the GM for a “good” latent variable z that produces content

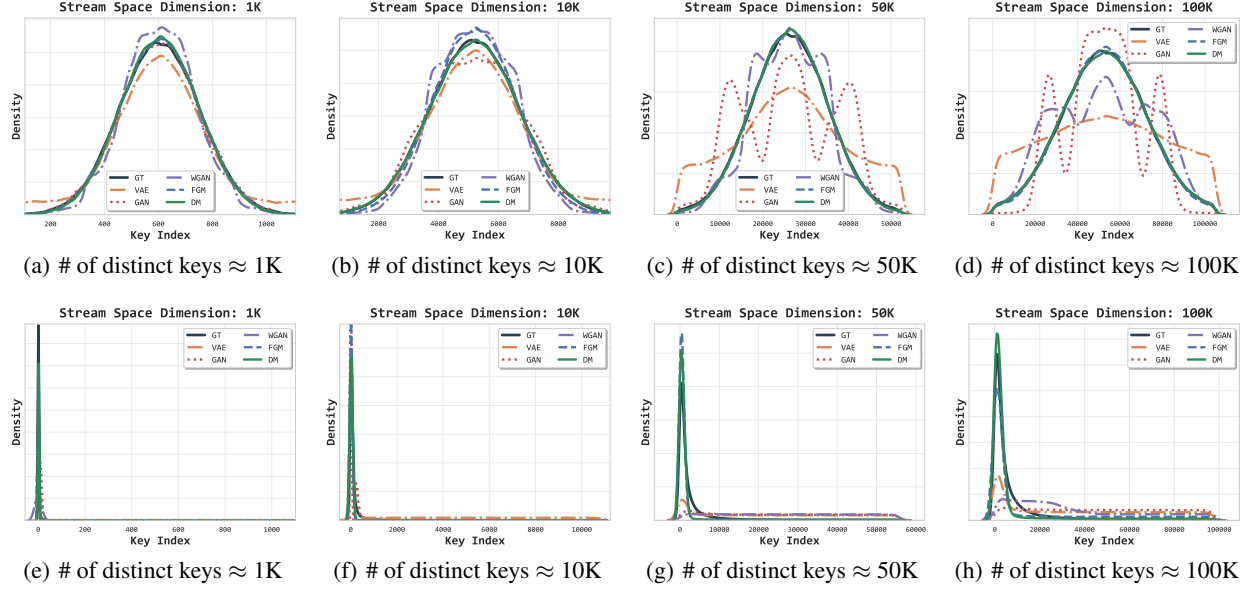


Figure 6: Visualization of Streaming data distribution fitting. The first row shows results on Gaussian distributions, while the second row shows results on Zipfian distributions. From left to right, the scale of the data stream increases.

Algorithm 1 Estimation Algorithm for Unconditional GMs

Ensure: Volume estimation using an unconditional GM

Require: GM f_G , strength ζ , matrix Φ , # of loops K

- 1: $\{z_0, \dots, z_i, \dots\} \leftarrow$ sample priors from $\mathcal{N}(0, I)$
 - 2: Select a good z such that $\arg \min_{z_i} \|b - \Phi f_G(z_i)\|_2^2$
 - 3: $b \leftarrow$ sample counters by linear sketching
 - 4: **for** $j = K$ to 0 **do**
 - 5: $z \leftarrow z - \zeta \nabla_z (\|b - \Phi f_G(z)\|_2^2 + \|z\|_1)$
 - 6: **end for**
 - 7: $x \leftarrow f_G(z)$
 - 8: **return** x
-

Algorithm 2 Estimation Algorithm for Unconditional DMs

Ensure: Volume estimation using an unconditional DM

Require: DM f_G , strength ζ , matrix Φ , # of steps T

- 1: $x_T \leftarrow$ sample prior from $\mathcal{N}(0, I)$
 - 2: $b \leftarrow$ sample counters by linear sketching
 - 3: **for** $t = T$ to 0 **do**
 - 4: $x'_{t-1} \leftarrow \text{REVERSE_SAMPLE_ONE_STEP}(x_t, t, f_G)$
 - 5: $\hat{x}_0 \leftarrow \text{PREDICT_X0_WITH_XT}(x'_{t-1}, t)$
 - 6: $x_{t-1} \leftarrow x'_{t-1} - \zeta \nabla_{x_t} \|b - \Phi \hat{x}_0\|_2^2$
 - 7: **end for**
 - 8: **return** x_0
-

consistent with the observed data. By following prior works [37, 48], for all GMs except DMs, we first identify a "starting point," and then perform backward updates on it using the gradient of $\|b - \Phi f_G(z)\|_2^2 + \|z\|_1$ with regard to z , where the former term enforces consistency with the observations, while the latter term promotes plausibility (or likelihood). Regarding DMs, we directly integrate this gradient adjustment into the reverse sampling process, as applying the aforementioned iterative refinement to them would be too time-consuming. See DPS [39] for more details. Algorithm 1 and Algorithm 2 present our two implementation strategies, respectively.

We still experiment on the two synthetic datasets described above, employ Count-Min as the underlying sketch, insert the GT vector into the sketch, construct the projection matrix Φ , and force the trained GM to recover the GT vector from the sketch counters b . As Figure 7 shows, we test their sketching performance along four dimensions to embrace our principles: *fidelity* measured by WMRE, *accuracy* measured by ARE, *inference* time measured by the logarithmic time (ms) taken for a single generation pass, and *training* time (s) measured by the time taken for training the GM. On one hand, consistent with their unconditional generation results, VAEs and GANs perform adequately on small-scale data streams, but their accuracy and fidelity drop sharply as data dimensionality increases, especially under Zipfian distributions, whereas FGM and DM remain robust. On the other hand, what is not evident from unconditional generation results is that: First, despite DM consistently achieving the highest fidelity, its accuracy does not follow suit. For instance, under the Zipfian distribution, DM's accuracy gradually falls behind that of FGM. Second, GANs suffer from extremely long training times, roughly 5 ~ 10 times longer than other methods, due to their notoriously unstable adversarial training; as dimensionality increases, convergence to an optimal generator requires painstaking and delicate parameter tuning. Meanwhile, DM also faces severe inference inefficiency, as it requires lengthy iterative sampling

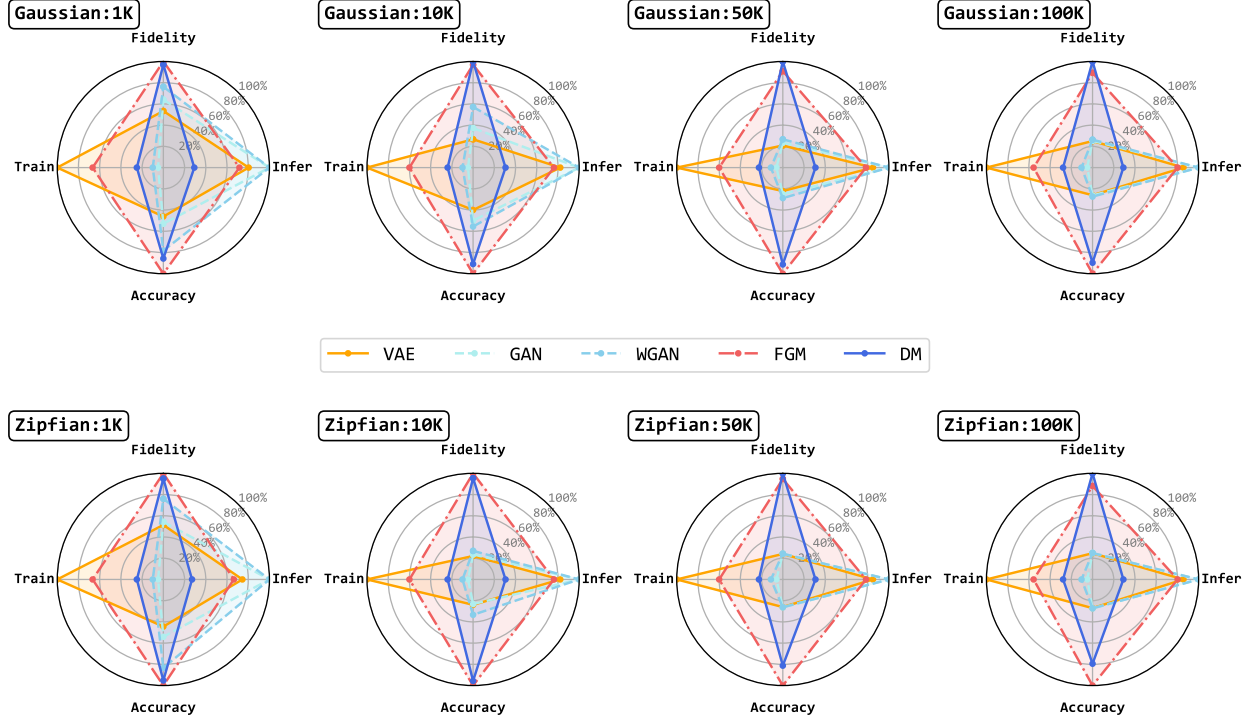


Figure 7: Performance comparison of different generative models under Gaussian and Zipfian distributions. For better visualization, metrics across different dimensions are normalized by the best result (higher the better).

Table 2: Additional comparison with flow matching on Zipfian-100K.

GM (# of steps)	ARE ↓	WMRE ↓	Gen. Time (s)
DDPM (500)	0.22 ± 0.04	0.25 ± 0.05	7.58
Flow Matching (15)	0.32 ± 0.11	0.30 ± 0.10	0.92
FGM (1)	0.18 ± 0.07	0.28 ± 0.06	0.001

along a Markov chain. Concretely, generating a single sample (or batch) of 100K-dimensional vectors with 1,000 diffusion steps consumes over 10 GB of GPU memory and takes tens of seconds, making it roughly $10^3 \times$ slower than alternative approaches. Given these two critical limitations, we exclude both GANs and DM from further consideration. Finally, due to the limited expressive power of VAEs, we find that FGMs offer a more favorable balance.

Compared to classical diffusion models, Flow Matching [49] enables efficient, simulation-free training for a broad class of transport problems using only samples from the endpoints, so it only requires a small number of sampling steps for generation. To evaluate the uncertainty it introduces to our GM selection, we conduct a simple comparative experiment on Zipfian-100K. Flow Matching adopts exactly the same model configuration as the diffusion model previously used in our work (i.e., a DDPM [23]), while requiring fewer than one-tenth of the sampling steps (only 15 steps). The corresponding metrics are reported in Table 2. It shows the inference latency remains non-negligible (nearly 1s) for our setting, where ms-level inference is required. Meanwhile, due to the reduced number of sampling refinement steps (i.e., T in Algorithm 2), its performance also degrades significantly compared with the 500-step diffusion model, with roughly 30% lower accuracy and 20% lower fidelity. We consider this an interesting direction for future work.

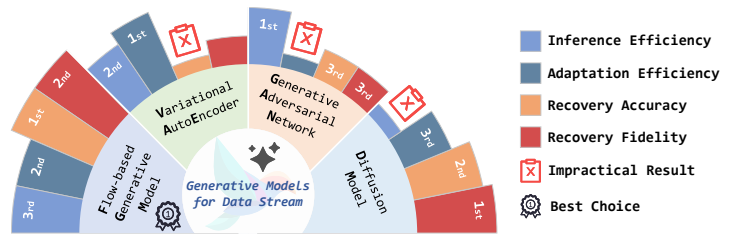


Figure 8: **Performance overview across different GMs.** FGM achieves the most favorable trade-off in sketching tasks.

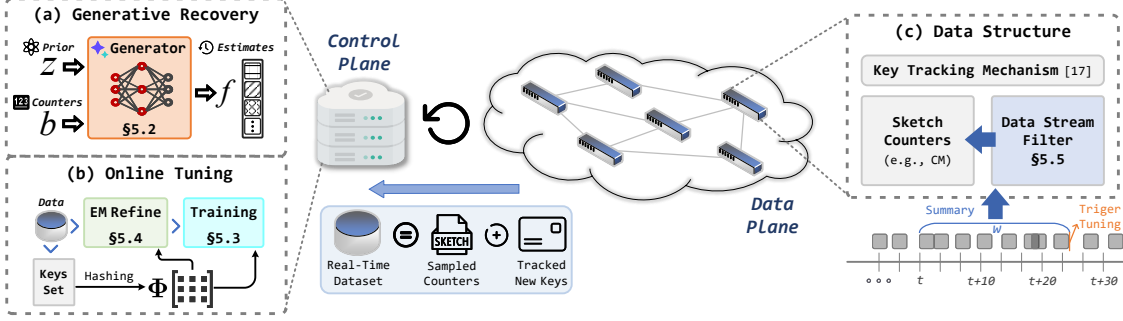


Figure 9: **System overview of FLORE.** (a) illustrates the control-plane recovery procedure, where flow-based generators are trained. (b) depicts the process of online tuning and scaling. The data-plane structure details are explained in (c).

Summary. Figure 8 presents a synopsis of the examination results (each bar represents the quality achieved, higher the better). To summarize, we have three main observations: ① GANs suffer from notorious sensitivity to adversarial training and a brittle optimization process even if we introduce the Wasserstein optimization [45], i.e., WGANs. ② VAEs rely on a loose surrogate objective, leading to poor performance as the size of the data stream increases (e.g., beyond 10K). ③ Although DMs are robust and enjoy stable training, their sampling complexity is prohibitively high, typically requiring several seconds, which makes them unsuitable for real-world applications. In fact, it is unnecessary to perform multi-round reverse sampling computations by encoding them into the model itself. Based on these observations, we select FGM as our underlying model due to its best trade-off, as discussed later.

5 FLORE: A Flow-based Sketching Framework

In what follows, we present and elaborate the algorithmic implementation of our proposal, FLOW-based ORthogonal REcovery (or FLORE for short).

5.1 System Overview

As illustrated in Figure 9, we consider a distributed system design similar to Network Telemetry [18, 50], where encoded sketching information are captured on local devices³ such as switches, and then decoded at a remote collector such as data center. We refer to the former as data-plane summary while the latter control-plane recovery. They operate in parallel, and here the control plane doesn't suffer from stringent computing resource limitations. Periodically, the data plane transmits the collected data (i.e., tracked keys and sampled counters) to the control plane for tuning GM via sliding windows (SWs) [51].

FLORE vs existing solutions. From the figure, what clearly sets FLORE apart from learning-based competitors [29, 11] is that the learned model resides in the control plane (also see Figure 1), leaving the data-plane summary procedure and time-space efficiency largely unchanged. Although CS-based approaches [18, 17] do adopt the same decoupled system design and require no training, their recovery algorithm incurs substantial overhead (e.g., computing the pseudoinverse matrix of Φ [52]), while the resulting sparse estimations often lack realism. In comparison, FLORE provides generative one-step solution, uses a scalable model architecture, and handles online tuning without manually annotated data.

Scope of FLORE. Similar as CS-based sketch solutions [18, 13, 17, 19], FLORE relies on the control plane to query the measurement results. The data plane is only responsible for updating the data structure and tracking the operator Φ . Thus, any applications that require computation locally or nonlinear aggregation are not targeted use cases of FLORE.

5.2 Model Architecture

As with classical FGMs, FLORE leverages INNs to learn the target distribution. The INN is constructed as a sequence of bijective transformations T_i with tractable Jacobian determinants, whose composition defines the overall mapping:

$$f = z_K = T_K \circ T_{K-1} \circ \dots \circ T_1(z_0), \quad z_0 = [b, z].$$

³FLORE can be extended so that multiple devices or sketches share a few base models (§5.2), instead of maintaining one model per sketch. As this is beyond the scope of this paper, we leave a deeper exploration to future work.

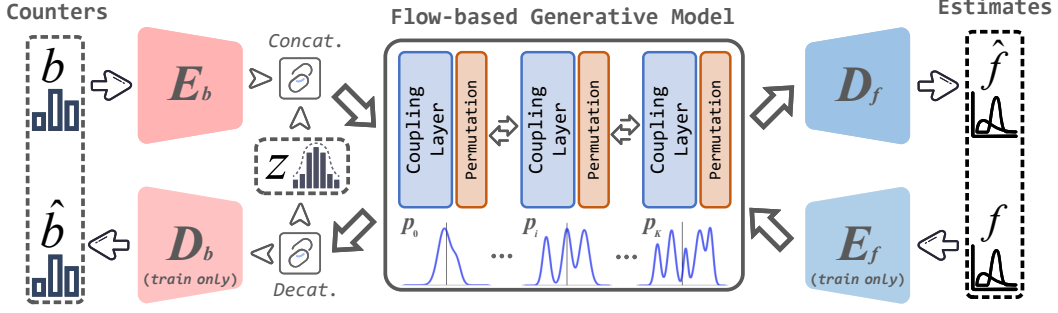


Figure 10: **The architecture of FLORE.** FLORE is modeled as an invertible mapping in the latent space.

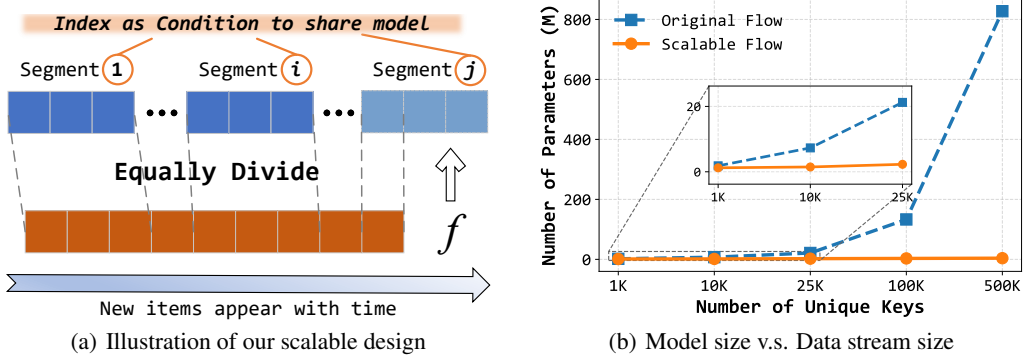


Figure 11: Scalable flow-based generative model for online learning in one-pass data streams.

Similar to DMs [53], such a chain preserves dimensionality and thus precludes intrinsic dimension reduction within INNs. Consequently, modeling large-scale data streams (with 10^4 s unique items) makes them prone to spending excessive amounts of network parameters. So in contrast to purely INN-based approaches, our framework introduces two autoencoders to project b and f into lower-dimensional representations (latent space) respectively, which scales more gracefully to higher-dimensional data. The overall architecture is depicted in Figure 10, where the GM learns the semantic and conceptual structure of the data. The coupling operations in FLORE are built on the foundations of prior works [54, 55, 56, 38]. The fundamental unit of this INN is a reversible block consisting of two complementary affine coupling layers. Concretely, each block partitions its input into two components, $[u_1, u_2]$, and applies alternating affine transformations with strictly upper or lower triangular Jacobians:

$$v_1 = u_1 \odot \exp(s_1(u_2)) + t_1(u_2), \quad v_2 = u_2 \odot \exp(s_2(v_1)) + t_2(v_1).$$

Here, $\odot$ denotes element-wise multiplication. The resulting outputs $[v_1, v_2]$ are concatenated and passed to the subsequent block. Crucially, the internal functions s_j and t_j can be represented by arbitrary neural networks and do not require invertibility themselves. In our implementation, these are realized by a sequence of fully connected layers with RELU activations. Given the outputs $[v_1, v_2]$, the forward pass is analytically invertible:

$$u_2 = (v_2 - t_2(v_1)) \oslash \exp(s_2(v_1)), \quad u_1 = (v_1 - t_1(u_2)) \oslash \exp(s_1(u_2)).$$

Here, $\oslash$ denotes element-wise division. To enhance model capacity and ensure global dependency, we interleave permutation layers [56] between reversible blocks. These layers shuffle the elements of the input in a fixed, pseudo-random manner, forcing the splits $[u_1, u_2]$ to vary across layers and facilitating interaction among all variables.

Scalability under data stream uncertainty. As discussed in §2.1, FLORE may support dynamic capacity expansion to accommodate streaming data recovery. Direct retraining on the continually expanded key space would incur prohibitive computational overhead, while high-dimensional data streams (often reaching millions of dimensions) would also lead to a dramatic increase in parameter count.

To address these challenges, we partition the GT vector f into equal-length segments and incrementally increase the number of segments over time to adapt those new elements (see Figure 11(a)). To maintain parameter efficiency, these segments are able to share the same network conditioned on their respective indices. Such design ensures that when the brand-new keys arrive, only the parameters associated with the most recent index require fine-tuning, thereby

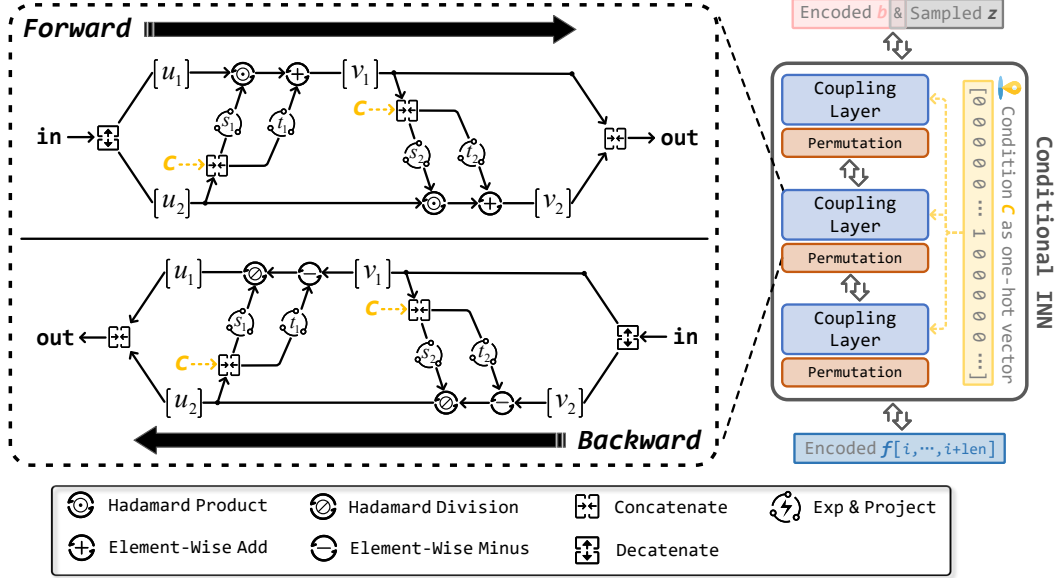


Figure 12: The underlying cINN architecture used in FLORE.

minimizing the impact on existing segments. As a result, the model’s parameter complexity is reduced from $\mathcal{O}(N)$ to $\mathcal{O}(\text{SEGMENT LENGTH})$. We empirically demonstrate this benefit in Figure 11(b) by plotting how the number of FLORE’s parameters scales with the size of the data stream (i.e., data dimensions). As can be seen, the scalable design substantially reduces the parameter count, achieving a reduction of over 200 times at 500K unique keys. In our context, this approach is implemented using a Conditional INN (cINN) [57]. Since the subnetworks s_j and t_j are not explicitly inverted, they can be conditioned on condition c via concatenation without compromising the overall invertibility of the block (i.e., replacing $s_1(u_2)$ with $s_1(u_2, c)$). Specifically, the index c is represented as a one-hot vector and processed by a feedforward conditioning network to extract intermediate representations, which is trained jointly with the cINN. Figure 12 illustrates the final architecture; note that two autoencoders are omitted from the diagram for clarity.

5.3 Training Objective (Theorem 3)

Below, we shift focus to the training scheme of FLORE that enforces both the measurement consistency and the fidelity condition. In our design, the overall objective consists of three types of training loss terms as follows.

L1 For aligning with the observation, there are two loss terms: *consistency* loss $\mathcal{L}_{\text{con}}$ and *reconstruction* loss $\mathcal{L}_{\text{rec}}$. They penalize the deviations between b and f respectively. Let $f_G := G([b, z])$, then we can write

$$\mathcal{L}_{\text{con}} = \mathbb{E} \left[(\Phi f_G - b)^2 \right], \quad \mathcal{L}_{\text{rec}} = \mathbb{E} \left[(f_G - f)^2 \right].$$

L2 For modeling the distribution, we also have two loss terms: *invertibility* loss $\mathcal{L}_{\text{inv}}$ and *orthogonal* loss $\mathcal{L}_{\text{ort}}$. The former forces FLORE to remain its invertibility by

$$\mathcal{L}_{\text{inv}} = \mathbb{E} \left[(f - G(G^{-1}(f)))^2 \right].$$

While the latter embraces the theoretical analysis presented in §3.2, which aims to make the distributions of z and b as orthogonal as possible, i.e., $q(b, z) = p_B(b)p_Z(z)$. Using an appropriate $\mathcal{D}$ that measures the similarity of two distributions, e.g., Kullback-Leibler Divergence (KLD) [58] or Maximum Mean Discrepancy (MMD) [59], we can define it as

$$\mathcal{L}_{\text{ort}} = \mathcal{D}(q(b, z) \parallel p_B(b)p_Z(z)).$$

◇ **Note:** During the training of FLORE, we don’t incorporate the traditional FGM’s negative log-likelihood loss term, i.e., $\log p(z_0) - \log \sum_{i=1}^K \left| \det \frac{dT_i}{dz_{i-1}} \right|$, into the objectives [60, 61]. Empirically, we observe that it tends to slow down model convergence (see Table 3 and CAIDA-2018 is the default dataset hereafter); in theory, Theorem 3 shows that the aforementioned losses alone suffice to yield an unbiased estimation of the target (conditional) distribution.

Table 3: Ablation of the Jacobian Loss Term

Method (256KB)	ARE ↓	WMRE ↓	Training Iters
FLORE w/ Jac.	1.20 ± 0.78	0.41 ± 0.18	200s $\sim$ 400s
FLORE w/o Jac.	1.03 ± 0.38	0.45 ± 0.10	50s $\sim$ 100s

Theorem 3. *If the designed loss terms, i.e., $\mathcal{L}_{\text{con}}$, $\mathcal{L}_{\text{inv}}$ and $\mathcal{L}_{\text{ort}}$ vanish, the invertible mapping $G : [b, z] \leftrightarrow f$ applied to the $p(b, z)$ recovers the true posterior $p(f | b)$.*

Proof. Suppose we take a posterior conditioned on a fixed b , i.e., $p(f | b)$, and transform it using the forward pass of our perfectly converged INN. First, if the orthogonal loss $\mathcal{L}_{\text{ort}}$ converges to zero, we have $q(b, z) = p_B(b)p_Z(z)$, according to Definition 5 (for KL divergence) and Lemma 4 (for MMD). Then because $\mathcal{L}_{\text{con}} = \mathbb{E} \left[(\Phi f - b)^2 \right] \rightarrow 0$ and $\mathcal{L}_{\text{inv}} = \mathbb{E} \left[(f - G(G^{-1}(f)))^2 \right] \rightarrow 0$, we know that the INN’s output distribution of b' (marginalized over z) must be $p'_B(b') = \delta(b' - b)$ where $\delta(\cdot)$ is the Dirac delta function. Also, because of the independence between z and b' in the output, the output distribution of z is still pre-defined $p_Z(z)$. So the joint distribution of outputs is $q(b', z) = \delta(b' - b)p_Z(z)$.

When we run forward the INN, and repeatedly input b while sampling $z \sim p_Z(z)$, this is the same as sampling $\pi := [b', z]$ from the joint distribution $q(b', z)$ above. Next, we show that the INN G will transform these samples back to $p(f | b)$ or $p_F(f)$ for convenience.

Note that by the change of variables theorem, we can write the backward transformation as $q(\pi) = p_F(f = G(\pi)) \left| \det \frac{dG}{d\pi} \right|$ where $\det \frac{dG}{d\pi}$ is the Jacobian determinant of the bijective function. Similarly, for the forward direction, we have $p'_F(f) = q(\pi = G^{-1}(f)) \left| \det \frac{dG^{-1}}{df} \right|$. Putting it together, we have

$$p'_F(f) = p_F(f = G(G^{-1}(f))) \left| \det \left[\frac{dG}{d\pi} \frac{dG^{-1}}{df} \right] \right| = p_F(f) |\det [\mathbf{Id}]| = p_F(f) = p(f | b),$$

which concludes our proof. $\square$

L3 To instill *sparsity* into FLORE’s recovery (i.e., the idea of CS), we append a regularization term $\mathcal{L}_{\text{sp}} = \|f_G\|_1$.

Taken together, we train FLORE by minimizing the loss

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{con}} + \alpha_1 \mathcal{L}_{\text{rec}} + \alpha_2 \mathcal{L}_{\text{inv}} + \alpha_3 \mathcal{L}_{\text{ort}} + \alpha_4 \mathcal{L}_{\text{sp}} \quad (6)$$

where $\alpha_{(\cdot)}$ are trade-off parameters controlling the relative contributions of individual loss terms. After training, the learned generator G can be directly deployed to recover the streaming ground truth f from sampled inputs $[b, z]$, for both training data and previously unseen observations.

Algorithm 3 (Pytorch) EM refinement pseudocode for FLORE

```

1  # f: Count-Min initial solution
2  # b, Phi: sketch observations and matrix
3  # T: number of EM steps
4
5  for t in range(T):
6      # E-step: compute normalization factors
7      C1 = Phi / (Phi @ f)
8      C2 = f / Sum(Phi, axis=0)
9
10     # M-step: multiplicative update
11     f = C2 * WeightedSum(b, C1)
```

5.4 GT-free Learning (Theorem 4)

It’s often extremely hard to construct datasets that capture all aspects of very large databases in high-speed environments or testbeds. Hence, a particular challenge against this backdrop is to train FLORE without ground truth (only measurement

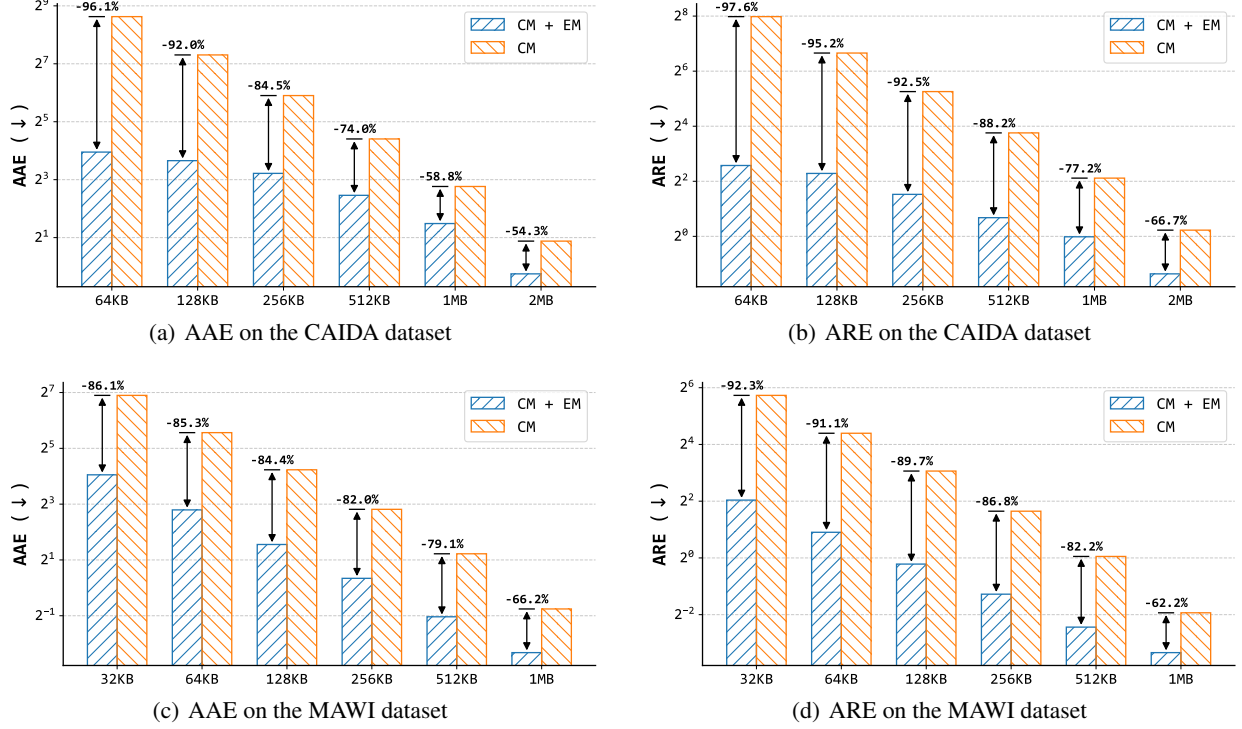


Figure 13: Ablation comparing Count-Min Sketch and Count-Min Sketch + EM refinement (with ten steps).

b is available). Fortunately, taking into account the recent history of those counters of a linear sketch data structure, we are able to automatically infer hidden GT patterns. The underlying original sketch approximation rules, e.g., Count-Min and Count, provide a natural inductive bias for FLORE’s learning [62]. Nonetheless, naively substituting the true f in the aforementioned objectives by querying the sketch would destabilize the training procedure, because these estimations are quite coarse in general as discussed in §3.1.

To address this challenge, we connect the posterior approximation problem with maximum likelihood estimation (MLE). Specifically, we can apply an EM algorithm to refine estimates obtained from the underlying sketch, and this results in the iterations described in Theorem 4 as follows.

Theorem 4. *Minimizing the Kullback–Leibler Divergence $\mathcal{D}_{KL}(p(f | b) \| p(f^{(\cdot)}))$ yields the following EM update step: $f_i^{(n)} = f_i^{(n-1)} \sum_j \frac{\Phi_{i,j}^\top}{(\Phi_{f^{(n-1)}})_j} b_j$, initialized with $f_i^{(0)} > 0$.*

Proof. Without loss of generality, we start by a rescaled version of the linear sketching problem:

$$g_j = \sum_i f_i h_{i,j}, \quad \text{s.t. } f_i \geq 0, \sum f_i = 1, g_j = \frac{b_j}{\sum_j b_j}, \text{ and } h_{i,j} = \frac{\Phi_{i,j}^\top}{\sum_i \Phi_{i,j}^\top} = \frac{\Phi_{i,j}^\top}{k_j},$$

which transforms all measures, i.e., f and b , into probability measures. Then, if the obtained solution is given, the solution can be transformed as follows: $\hat{f} = \frac{k_j}{\sum_j b_j} f = \frac{k}{\sum_j b_j} f$.

Our formulation can also be regarded as a statistical model. Suppose we observe N samples $Y_1, \dots, Y_N$, which are independently drawn from a distribution g . If the distribution $g(y)$ is given by $g(y) = \int h(x, y) f(x) dx$ where $h(x, y)$ is the corresponding kernel function of $h(\cdot)$, this is equivalent to the following two-step sampling process: first draw X from $f(x)$, and then draw Y from the conditional distribution $h(x, \cdot)$ given X . In other words, the observed data Y arise from a latent variable model:

$$X \rightarrow Y, \quad \text{where } X \sim f \text{ and } Y | X = i \sim h_{i,j}.$$

As a result, the joint and marginal probabilities are given by

$$P(X = i, Y = j) = f_i h_{i,j}, \quad P(Y = j) = g_j = \sum_i f_i h_{i,j}.$$

This structure constitutes a standard latent variable model, which is formally very similar to a Gaussian Mixture Model (GMM), except that in this case, the transition probabilities $h_{i,j}$ are fixed and known. If we only observe the data $Y_1, \dots, Y_N$, the likelihood function is given by $L_N(f) = \prod_j (\sum_i f_i h_{i,j})^{n_j}$, where n_j denotes the number of times Y takes the value j , and $\sum_j n_j = N$. Therefore, the log-likelihood is $\log L_N(f) = \sum_j n_j \log (\sum_i f_i h_{i,j})$. If we express the empirical distribution as $g_j = \hat{g}_j = \frac{n_j}{N}$, then we obtain

$$\frac{1}{N} \log L_N(f) = \sum_j g_j \log \left(\sum_i f_i h_{i,j} \right) \quad (7)$$

Next, we show its connection with KL divergence. First, we can interpret the sketching problem $g_j = \sum_i f_i h_{i,j}$ as solving the following optimization problem: $\min_{f \geq 0, \sum f_i = 1} \mathcal{D}_{\text{KL}}(g \parallel H^\top f)$ where $H^\top f$ represents the distribution induced by f through the transition matrix H , i.e., $(H^\top f)_j = \sum_i f_i h_{i,j}$. In other words, we seek a distribution f such that the resulting mixture distribution $H^\top f$ is as close as possible to the observed empirical distribution g , measured in terms of KL divergence. By Definition 5, this minimization problem is equivalent to: $\max_f \sum_j g_j \log (\sum_i f_i h_{i,j})$, which is precisely Eqn. (7).

Now we return to the latent variable interpretation introduced earlier. Let us clarify the assumptions and notation: ① Complete data: (X, Y) , with joint distribution $P(X = i, Y = j) = f_i h_{i,j}$; ② Observed data: only Y is observed, $P(Y = j) = \sum_i f_i h_{i,j}$. This is precisely the definition of an incomplete data problem. Consequently, the log-likelihood based on the complete data is

$$\log L_C(f) = \sum_{i,j} n_{i,j} \log(f_i h_{i,j}) = \sum_{i,j} n_{i,j} \log f_i + \text{constant}$$

where $n_{i,j}$ is the number of times $(X = i, Y = j)$ occurs. However, in practice, we can only compute the log-likelihood based on the observed data $\log L_N(f) = \sum_j n_j \log (\sum_i f_i h_{i,j})$. Therefore, this problem is well-suited for iterative solution using the EM algorithm, which alternates between estimating the missing data (Expectation step, E-step) and maximizing the likelihood (Maximization step, M-step) under the current estimate of the parameters. Formally, in each iteration of the EM algorithm, we can proceed as follows:

• **E-step:** Based on the current estimate $f^{(n-1)}$, we compute the hidden responsibility probabilities, i.e., the posterior probability that a given observation $Y = j$ originated from latent component $X = i$. This is given by Bayes' rule:

$$r_{i,j}^{(n-1)} := P^{(n-1)}(X = i \mid Y = j) = \frac{P^{(n-1)}(X = i, Y = j)}{P^{(n-1)}(Y = j)} = \frac{f_i^{(n-1)} h_{i,j}}{\sum_k f_k^{(n-1)} h_{k,j}} \quad (8)$$

Then, we have the estimate $\mathbb{E}[n_{i,j}] = n_j P^{(n-1)}(X = i \mid Y = j) = n_j r_{i,j}^{(n-1)}$.

• **M-step:** We update the parameter f_i by maximizing the expected complete-data log-likelihood with respect to f . We first construct the Q-function $Q(f \mid f^{(n-1)}) := \mathbb{E}[\log L_C(f)] = \sum_{i,j} n_j r_{i,j}^{(n-1)} \log f_i + \text{constant}$. So, the M-step is equivalent to

$$\max_f \sum_{i,j} n_j r_{i,j}^{(n-1)} \log f_i \quad \text{subject to} \quad f_i \geq 0 \text{ and } \sum_i f_i = 1$$

We can utilize Lagrange Multipliers to solve this problem. Define $L = \sum_{i,j} n_j r_{i,j}^{(n-1)} \log f_i + \lambda (\sum_i f_i - 1)$, and take the partial derivative with respect to f_i : $\frac{\partial L}{\partial f_i} = \frac{\sum_j n_j r_{i,j}^{(n-1)}}{f_i} + \lambda$. Set it to zero, we have $f_i = -\frac{\sum_j n_j r_{i,j}^{(n-1)}}{\lambda}$. Applying the constraint $\sum_i f_i = 1$, it yields $-\lambda = \sum_i \sum_j n_j r_{i,j}^{(n-1)}$. Since $\sum_i r_{i,j}^{(n-1)} = 1$, we have $-\lambda = \sum_j n_j = N$. Thus, the updated estimate is obtained as

$$f_i^{(n)} = \frac{1}{N} \sum_j n_j r_{i,j}^{(n-1)} = \sum_j g_j P^{(n-1)}(X = i \mid Y = j) \quad (9)$$

Substituting the expression for the conditional probability, i.e., Eqn. (8), into the update rule, i.e., Eqn. (9), we have

$$f_i^{(n)} = f_i^{(n-1)} \sum_j \left(\frac{h_{i,j}}{\sum_k f_k^{(n-1)} h_{k,j}} \right) g_j \iff \hat{f}_i^{(n)} = \hat{f}_i^{(n-1)} \sum_j \frac{\Phi_{i,j}^\top}{(\Phi \hat{f}^{(n-1)})_j} b_j.$$

This recurrence relation provides an iterative scheme for updating the distribution f , and it corresponds exactly to the formula presented in Theorem 4, which concludes the proof. $\square$

Table 4: Time-accuracy Trade-off of the EM Algorithm

# of EM Steps	1	3	5	10	20	50
Total Time (s) ↓	0.54	1.72	3.22	6.85	13.91	31.05
Normal. ARE ↓	9.71	2.04	1.55	1.41	1.40	1.39

If the given linear sketching problem does not have a non-negative solution, then the EM algorithm can be proven to yield the closest approximation of f in terms of KLD [63]. Empirically, Figure 13 compares the simple CM and CM equipped with 10-step EM refinement (as indicated by “CM + EM”) when testing in the CAIDA dataset (1st row) and MAWI dataset (2nd row). It can be seen that the enhancement brought by EM is truly impressive. To be more specific, on the CAIDA dataset, it reduces CM’s AAE by approximately 76% ARE by about 86%. Similarly, on the MAWI dataset, it lowers CM’s AAE by roughly 80% and ARE by around 84%. These results are consistent with Theorem 4. From an interpretative perspective, we can view this EM algorithm as “denoising” the Count-Min solutions using global information, i.e., the linear system $b = \Phi f$. To realize it, we have the following formula:

$$f_i^{\text{new}} \leftarrow \frac{f_i^{\text{old}}}{\sum_{j=1}^m \Phi_{i,j}^\top} \sum_{j=1}^m \frac{\Phi_{i,j}^\top \times b_j}{\sum_{k=1}^N \Phi_{i,k} \times f_k} \quad (10)$$

While the update is defined on per-key basis, it admits GPU parallelism. Algorithm 3 illustrates the EM algorithm’s pseudocode, in which all batch-wise operations are implemented via tensor primitives. Finally, Algorithm 4 depicts the complete training procedure of FLORE in a Pytorch-like style.

Why is EM alone not a good solver? One may naturally ask whether the EM algorithm can be directly applied to linear sketch recovery. Although parallel computation can accelerate its iterative process, it comes at the cost of memory-expensive per-key-wise optimization (e.g., 10s GB GPU memory usage for 10^4 s keys). Furthermore, Table 4 reports the impact of different numbers of EM update steps from two perspectives: normalized ARE (divided by that of FLORE) and execution time. We can see that it consistently underperforms our method which has *ms*-level latency (§6), and the benefit drops sharply after 5-10 steps. A core reason is that each EM-based recovery is solved independently and thereby fails to exploit collective gains (i.e., the underlying distribution), rendering EM not even an “overkill”. Thus, in our implementation, where tuning is not subject to strict query response time constraints, the EM is performed 3–5 steps as a *one-time* preprocessing operation.

Algorithm 4 (Pytorch) Training pseudocode for FLORE

```

1  # Phi: linear sketching matrix (Count-Min)
2  # G.forw, G.back: forward and reverse process of GM G
3  # a: list of trade-off parameters (alphas)
4
5  for b in loader:          # load a minibatch from the SW
6      # approximate GT via EM (can be pre-computed)
7      f = EM_OPT_CM(b, Phi)    # CM as the start point
8      z = Gaussian(0, I)       # sample the prior z
9      f0 = G.forw([b, z])      # reconstruct f from b
10     f1 = G.forw([Phi@f, z])   # reconstruct f from Phi@f
11     f2 = G.forw(G.back(f))    # reconstruct f from itself
12     bz = G.back(f0)          # get the joint distribution
13
14     # training loss, Eqn. (6)
15     loss = L2Loss(Phi@f0, b)   # consistency loss term, i.e.,  $\mathcal{L}_{\text{con}}$ 
16         + a[0]*L2Loss(f1, f)  # reconstruction loss term, i.e.,  $\mathcal{L}_{\text{rec}}$ 
17         + a[1]*L2Loss(f2, f)  # invertibility loss term, i.e.,  $\mathcal{L}_{\text{inv}}$ 
18         + a[2]*MMD_or_KLD([b, z], bz) # orthogonality loss term, i.e.,  $\mathcal{L}_{\text{ort}}$ 
19         + a[3]*L1Loss(f0)     # sparsity loss term, i.e.,  $\mathcal{L}_{\text{sp}}$ 
20
21     loss.backward()           # compute the gradient
22     update(G.params)          # update G network
    
```

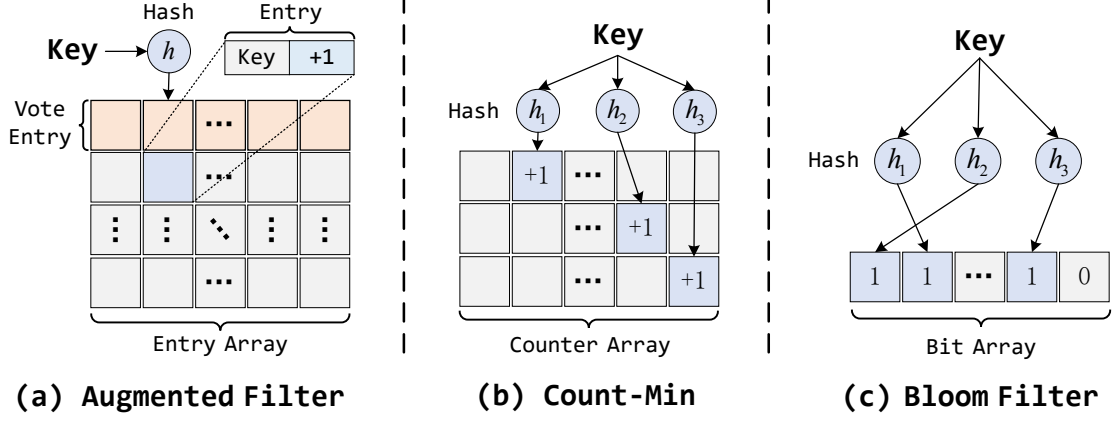


Figure 14: Data structures used in FLORE: a Bloom Filter for key tracking, a Count-Min Sketch for store infrequent items, and an augmented filter for saving frequent items as key-value pairs.

5.5 Data Structure

In this part, we provide data-plane details of FLORE. As illustrated in Figure 14, the local data structure of FLORE consists of three components: (1) an augmented filter for storing frequent elements, (2) a Count-Min Sketch for recording the remaining elements, and (3) a Bloom Filter for tracking newly observed keys. We first describe the Bloom Filter-based key tracking mechanism and the stream filtering algorithm for automatically separating hot items from the data stream into the augmented filter.

5.5.1 Key Tracking Mechanism

As noted before, constructing Φ^4 requires tracking all distinct inserted keys, either during training or/and at query time (retroactively). While we do not concern ourselves with this problem and assume that the key set $\mathcal{I}$ is known, we briefly introduce a standard key tracking mechanism commonly adopted in prior studies [64, 65, 18, 17, 19] to collect $\mathcal{I}$ from the data stream.

The mechanism consists of two parts: key identification and key recording. Specifically, key identification part maintains a lightweight data structure, i.e., a Bloom Filter, to identify keys in the summary phase (or data plane). As depicted in Figure 14(c), the Bloom filter is implemented as a one-dimensional array with m_{bf} entries and is associated with k_{bf} independent hash functions. Each entry stores a single bit indicating whether the mapped input corresponds to a key that has not been observed before. For each incoming item, we examine the bit statuses of the k_{bf} mapped entries prior to updating them. Once a new key is identified in the summary phase, it is forwarded to the recovery phase (or control plane). In the recovery phase, the key recording component stores these keys for subsequent analysis. This offloading strategy reduces the resource consumption of the key-tracking mechanism, as each unseen key is identified and transmitted at most once.

5.5.2 Stream Filtering Mechanism

In the light of previous studies [12, 19], we also separate frequent elements as explicit key-value pairs while compressing infrequent elements in a sketch (see §3.1). On one hand, it is well acknowledged that such separation can be realized with limited overheads, and improve summarization efficiency; on the other hand, its operations do not affect our analytical results thus far. To achieve this, in the data plane of FLORE, we employ a *oracle-free* stream filtering technique based on a *separate-and-guard-hot* strategy used by Elastic Sketch [27]. It tracks hot keys and records their information with dedicated entries. We first attempt to insert keys to the dedicated part and then evict infrequent keys to the shared part, i.e., a Count-Min. As illustrated in Figure 14(a), the augmented filter is composed of some arrays, and each array is comprised of a number of entries. In spirit to Ostracism (Greek: ostrakismos, where any citizen could be voted to be evicted from Athens for ten years), we aim to use two types of entries to store hottest items with their frequencies: the negative vote lives in the first entry, while the dedicated counters live in the other entries that consists of two fields—fingerprint (key) and counter (frequency). For each incoming tuple (k, v) , we compute a hash function to map it to a specific array. Within the selected array, we identify the entry with the minimum counter value and treat it as the positive vote. The update procedure then follows one of three cases:

⁴We do not explicitly store the 0–1 sparse matrix Φ in memory. Instead, we generate its entries on demand during adaptation.

Algorithm 5 FLORE Data-plane Summary

Ensure: Encode (key, v) into FLORE data structures

- 1: $i \leftarrow \text{Hash}(k)$
- 2: $\text{ARRAY} \leftarrow \text{AUGMENTEDFILTER}[i]$
- 3: Lookup k in $\text{ARRAY}[1 \sim \text{end}]$
- 4:
- 5: **if** Case 1: item found at position j **then**
- 6: $\text{ARRAY}[j].\text{value} + v \rightarrow \text{ARRAY}[j].\text{value}$
- 7: **else if** Case 2: position j is empty **then**
- 8: $k \rightarrow \text{ARRAY}[j].\text{key}$ and $v \rightarrow \text{ARRAY}[j].\text{value}$
- 9: **else**
- 10: $\text{ARRAY}[0].\text{value} + v \rightarrow \text{ARRAY}[0].\text{value}$
- 11: $\text{VOTE}^- \leftarrow \text{ARRAY}[0].\text{value}$
- 12: $m \leftarrow \text{find minimum entry in } \text{ARRAY}[1 \sim \text{end}]$
- 13: $\text{VOTE}^+ \leftarrow \text{ARRAY}[m].\text{value}$
- 14: **if** Case 3: $\text{VOTE}^- / \text{VOTE}^+ > \text{THRESHOLD}$ **then**
- 15: load tuple (k_{ex}, v_{ex}) from position m
- 16: clear entry at position m
- 17: add tuple (k, v) to position m (like Case 2)
- 18: # identify and record k_{ex} with BLOOMFILTER
- 19: insert tuple (k_{ex}, v_{ex}) into COUNT-MIN
- 20: **else**
- 21: # identify and record k with BLOOMFILTER
- 22: insert tuple (k, v) into COUNT-MIN
- 23: **end if**
- 24: **end if**

Algorithm 6 FLORE Control-plane Recovery

Ensure: Decode all counters using the GM of FLORE

- 1: $z \leftarrow \text{sample priors from GAUSSIAN distribution}$
- 2: $b \leftarrow \text{flatten counters in COUNT-MIN}$
- 3: $\text{SKETCHVEC} \leftarrow \text{GENERATIVEMODEL}(b, z)$
- 4: $\text{FILTERVEC} \leftarrow \text{Query}(\text{AUGMENTEDFILTER}, \text{keys})$
- 5: $\text{DECODEVEC} \leftarrow \text{SKETCHVEC} + \text{FILTERVEC}$
- 6: return DECODEVEC and keys



Figure 15: Ostracism: the exclusion from the collective; figure is polished and excerpted from page 18 of a [publicly available slide](#).

- **Case 1.** A dedicated entry with a matching fingerprint is found; its counter is incremented by v .
- **Case 2.** An empty dedicated entry is found and initialized with (k, v) .
- **Case 3.** Otherwise, an opposing vote is cast to determine whether eviction should occur. Concretely, we increment the negative vote by v and compute the decision variable $\lambda = \frac{\text{negative vote}}{\text{positive vote}}$. If λ exceeds a predefined threshold (e.g., 8), the tuple (k, v) replaces the item in the minimum entry, and the evicted item is forwarded to the Count-Min. Otherwise, (k, v) is directly sent to the Count-Min.

The pseudo-code of the data-plane summary procedure is provided in Algorithm 5. To query the frequency of a key, we perform a single hash operation to locate the corresponding array. If a stored fingerprint matches the queried key, the associated counter value is returned; otherwise, the query returns zero.

5.6 Other Details

Below, we summarize several auxiliary yet important details that complement the main methodology, covering dataset construction, normalization, and fine-tuning.

Dataset Construction First of all, FLORE eliminates the need for real samples (f) during training, facilitating online dataset construction, which requires only sketch summaries (b). Then, counters are periodically sampled at fixed intervals (defined by either the number of updates or elapsed time), which we term sampling epochs. To make the model capture the most recent data stream characteristics, we maintain a sliding window of these sampled states by following [43]. This window operates on a first-in-first-out (FIFO) style, discarding stale states once its capacity is reached. In our experimental setup, we configure the sampling interval to 5,000 updates and the sliding window length to 200~500.

Data Normalization As discussed in the above dataset construction process, we do not have fixed global statistics such as the overall minimum/maximum values or mean/std, due to the evolving nature of the data stream. Therefore, instead of classical min-max normalization or mean normalization, we employ instance normalization [66, 43] to mitigate the influence of the sliding window. It is noteworthy that, in FLORE’s CM with k rows and m/k counters per

row, the arrival of a streaming item triggers an update to exactly one counter per row, indexed by hash functions. Then, by leveraging the non-negativity of CM, we can define a scaling factor, *scale*, as the minimum of the row-wise maximum values: $\text{scale} := \min_{1 \leq j \leq k} \left(\max_{1 \leq i \leq m/k} b \left[\frac{m(j-1)}{k} + i \right] \right)$. This important factor can be computed in $\mathcal{O}(m)$ linear time (often very fast), and every element in f is smaller than this constant. The normalized counters are then input into the neural network as $b' = b/\text{scale}$. At inference time, the model f_G predicts the GT vector in the normalized space, which is subsequently mapped back to the original space via: $\hat{f} = \text{scale} \cdot f_G(b')$.

Online Tuning FLORE Since FLORE’s generative model is trained dynamically on sampled data, its operational model can be periodically updated by the cINN fine-tuned in the background on more recent and up to date data, to gracefully adapt to *planned* changes in the database system (resetting sketch parameters, planned addition/removal of keys) and to natural drifts in data stream distributions. Our evaluation results indicate that FLORE produces high-quality recoveries even epochs after being deployed, and even if the distribution changes during this time (e.g., due to time or/and attacks). This provides ample time for training substitute invertible functions, and one can tentatively handle previously unencountered queries using a simple algorithm such as Count-Min, since FLORE does not disrupt the original data-plane sketching operation.

5.7 End-to-End View

Putting everything together, we now present an end-to-end view of FLORE. As described in §5.1 and Figure 9, we have a data plane component to process stream with sketching and a control plane component to estimate the statistics from the sketch. We further split the control plane into a recovery stage and a tuning stage. Below, we describe the three stages.

Data-plane stage (Algorithm 5). Once tuple batches arrive, FLORE first attempts to insert the item into the Augmented Filter. If it hits, the remaining operations are skipped. Otherwise, FLORE inserts it into the CM and use the Bloom Filter to determine whether the key is unseen, thereby deciding whether it should be forwarded to the control plane.

Control-plane recovery stage (Algorithm 6). When a query is issued or recovery is needed for downstream analysis (e.g., anomaly detection), FLORE requests the data plane to transmit its data structures to the control plane. The control plane then decodes the heavy part and light part independently: the former is decoded losslessly (with unrecorded ones treated as zero), while the latter, together with a sampled prior z , is fed into the invertible solver for reconstruction. The final estimate is obtained by summing the two results.

Control-plane tuning stage (Algorithm 4). FLORE only delivers the sampled CM counters to this stage periodically which minimizes manual overheads. Using either fixed number of updates or elapsed time as a data-plane period, the control plane maintains a SW dataset to fine-tune the GM. FLORE pre-process these new counters using the EM, and the training offers quick convergence time as in our experiments in §6.

6 Evaluation

In this section, we seek to understand: (1) With only sketch counters, can FLORE satisfy the requirements of downstream tasks? (2) Can FLORE achieve real-time stream processing? (3) How does those changing stream conditions affect FLORE’s performance? (4) How does each design in FLORE contribute to its performance? Note that our approach has two variants: FLORE and FLORE (perfect). The former applies when $\mathcal{I}$ is unknown and therefore requires allocating an additional portion of memory to deploy Bloom Filters. This leads to the final question: (5) do they exhibit a similar performance in the conditional generation?

Summary of Main Results:

- For both real-world and synthetic data sketching, FLORE, using only counters, achieves up to 98.8% and 93.1% reduction in terms of accuracy and distribution/entropy estimation, respectively. And for heavy-hitters detection, FLORE can almost always achieve an F1 score close to 1.0 with only 64KB-128KB memory usage.
- FLORE relaxes the requirements for data collection and computational cost for ML models. On a local device, FLORE can reach a throughput of 70+ summary Mops, while on a centralized server, it can recover all the information within 10s milliseconds, proving its real-time synthesis capacity.
- We observe that, regardless of temporal variation, spatial variation, or natural drift, the performance degradation of FLORE can almost always be kept within 10%, demonstrating strong robustness to environmental changes.
- We confirm the effectiveness of the data-plane data structures and control-plane generative models in FLORE.
- Across all experiments, FLORE (perfect) and FLORE only exhibit some differences under tight memory budgets.

Table 5: Statistics of Real-world and Synthetic Data Streams Used in the Evaluation

Real-World	# of Keys	# of Items	Skewness	Synthetic	# of Keys	# of Items	Skewness
<i>CAIDA</i>	157 269	2 000 000	291.56	<i>Zipf-icml</i>	$\sim 30\,000$	1 000 000	24.27
<i>Webdocs</i>	125 623	2 000 000	25.42	<i>Zipf</i>	$\sim 28\,000$	1 000 000	125.93
<i>MAWI</i>	41 471	2 000 000	92.67	<i>Pareto</i>	$\sim 28\,000$	1 000 000	10.24
<i>Kosarak</i>	30 495	2 000 000	89.56	<i>Exponential</i>	$\sim 26\,000$	1 000 000	3.81
<i>Retail</i>	16 470	908 576	71.27	<i>Log-Normal</i>	$\sim 30\,000$	1 000 000	5.64

6.1 Experimental Setup

We conducted all experiments on a server equipped with an 8-core 3.80GHz CPU, 32GB DDR5 and a 16GB NVIDIA GPU. The server runs on Ubuntu 20.04, and all all experiments are repeated 5-10 times. By default, we implemented all programs in Python 3.10 and use PyTorch 2.9 for tensor operations and ML models. Below, we first detail the datasets used in our experiments, followed by the introduction of baseline methods and hyperparameters. Finally, we formulate the evaluation metrics.

6.1.1 Datasets

We evaluate the sketching quality of our approach on *ten* datasets, in which the data stream key-space size ranges from 10^3 s to 10^4 s, comprising *five* real-world data streams and *five* synthetic data streams. The real-world datasets include: CAIDA-2018 backbone IP packet traces (*CAIDA* [67]), Web document access traces (*Webdocs* [68]), traffic traces from the WIDE project (*MAWI* [69]), click-stream data (*Kosarak* [70]), and retail market basket data (*Retail* [71]). For the CAIDA and MAWI datasets, we construct 13-byte flow keys consisting of five fields: source and destination IP addresses, source and destination ports, and transport protocol. For the other datasets, item IDs are converted into 4-byte-long keys. In addition, we synthesize data streams following representative heavy-tailed distributions (e.g., Zipf, Pareto, exponential, and log-normal) and report their results in our technical report [72]. TABLE 5 shows the important details of these datasets.

In addition, we generate five synthetic datasets following representative heavy-tailed distributions: *Zipf-icml*, *Zipf*, *Pareto*, *Exponential*, and *Log-Normal*. Formally, let N denote the number of distinct items, and let f_i be the frequency associated with item i , we can write

- **Zipfian Distribution.** Frequencies follow Zipf’s law $f_i \propto \frac{1}{i^\alpha}$, $i = 1, 2, \dots, N$, where $\alpha > 0$ controls the degree of skew. Note that Zipf-icml is generated by following the implementation of [66], i.e., $f_i \propto \lfloor \frac{N}{i^\alpha} \rfloor$, while Zipf strictly follows the aforementioned formulation with a zipfian skewness of 1.4.
- **Pareto Distribution.** Frequencies are sampled from a Pareto distribution with shape parameter $\alpha > 0$, whose CDF is given by $P(F > x) = \left(\frac{x_m}{x}\right)^\alpha$, $x \geq x_m$, where x_m is the minimum value. We then draw samples $f_i \sim \text{Pareto}(\alpha)$, scale them by a constant factor, and discretize the results to obtain integer frequencies.
- **Log-Normal Distribution.** Frequencies follow a log-normal distribution, i.e., $\log f_i \sim \mathcal{N}(\mu, \sigma^2)$, where μ and σ denote the mean and standard deviation of the underlying normal distribution. The sampled frequencies are also scaled and rounded to integers.
- **Exponential Distribution.** Frequencies are generated from an exponential distribution with rate parameter $\lambda > 0$, whose density function is $p(x) = \lambda e^{-\lambda x}$, $x \geq 0$. As with the other distributions, samples are scaled and discretized to form integer-valued frequencies.

For most synthetic workloads, the resulting frequency vectors are randomly permuted to eliminate any ordering bias, and each distinct item is encoded as a fixed-length 4-byte key that appears in the stream according to its assigned frequency. Table 5 shows the important details of the datasets (i.e., the number of keys, the number of items, and the skewness of streaming data distribution).

6.1.2 Baselines

We compare our proposal to *ten* SOTA streaming solutions, covering four mainstream sketch techniques as follows. ① Two classical sketches: *CM* [25] and *CS* [26]. ② Three classical optimizations: *AG* [12], *ES* [27] and Conservative

Update Sketch (*CU* [73]). ③ Two CS-based sketches: *PR* [17] and *NZE* [19]. ④ Three learning-augmented sketches: Learned CM (*LCM*) [29], Learned CS (*LCS*) [29], and Learned Sketch without predictions (*LS*) [20]. We list these algorithms as follows:

1. **Count-Min Sketch (CM)** [25]: CM maintains k pairwise independent hash functions $h_\ell : U \rightarrow [E]$ and a counter array $C \in \mathbb{R}^{k \times E}$, where $E = m/k$. Upon processing the stream, each counter is updated as $C[\ell, h_\ell(j)] = \sum f_j$. For any item $i \in U$, its frequency estimate is $\tilde{f}_i = \min_{\ell \leq k} \{C[\ell, h_\ell(i)]\}$. In our experiments, we fix $k = 4$; all subsequent sketches follow this configuration unless otherwise specified.
2. **Count Sketch (CS)** [26]: Similar to CM, CS employs k hash functions $h_\ell : U \rightarrow [E]$ and a counter array $C \in \mathbb{R}^{k \times E}$. Additionally, it utilizes k independent sign hash functions $g_\ell : U \rightarrow \{-1, 1\}$. The counters are updated as $C[\ell, h_\ell(j)] = \sum f_j \cdot g_\ell(j)$, and the estimate for item i is computed as $\tilde{f}_i = \text{median}_{\ell \leq k} \{g_\ell(i) \cdot C[\ell, h_\ell(i)]\}$.
3. **Conservative Update Sketch (CU)** [73]: As a variant of CM, CU employs a conservative update policy to maintain an upper bound on the ground truth ($\tilde{f}_i \geq f_i$). For each incoming tuple (i, v) , a counter $C[\ell, h_\ell(i)]$ is updated only if it achieves the minimum value among all k rows. Formally, $C[\ell, h_\ell(i)] \leftarrow C[\ell, h_\ell(i)] + v$ for all $\ell \in \arg \min_{j \leq k} \{C[j, h_j(i)]\}$.
4. **Augmented Sketch (AG)** [12]: AG utilizes the skew of the underlying stream data to improve the frequency estimation accuracy for the most frequent items by filtering them out earlier. It exchanges data items between the filter and the sketch (i.e., a CM) such that hot items are kept in the filter. We configure the filter with ~ 100 key-value pairs; this relatively small size improves accuracy while preventing lookup overhead from degrading data stream processing throughput.
5. **Elastic Sketch (ES)** [27]: Using the same data structure with us (except for the Bloom Filter), ES tracks hot keys and records their information with dedicated entries. It first attempts to insert keys to the dedicated part and then evict infrequent keys to the shared part, i.e., a Count-Min. In the experiments, we allocate 50% of the memory budget to the shared part and the remaining 50% to the dedicated part.
6. **PR-sketch (PR)** [17]: PR also improves estimation accuracy by constructing linear equations that relate counter values to per-key aggregations. Solving the resulting least-squares problem requires an iterative solver. While the original implementation adopts a conjugate gradient method [74], we instead use LSMR [75] from `scipy.sparse.linalg`. We set the number of Bloom Filter hash functions to 7. Memory is partitioned equally between the sketch and the Bloom Filter, subject to an upper bound on the latter: to ensure a false positive rate below 1%, we limit its size to $9.6N$ bits (e.g., 120 KB for $N = 100K$) as suggested by [76].
7. **NZE-sketch (NZE)** [19]: NZE leverages compressive sensing by constructing a fractional sensing matrix and employing a hash table for elephant element filtering. Specifically, it maintains three data structures: key-value pairs for heavy hitters, a fractional-valued sketch for small elements, and a Bloom filter. We implement the SeqSketch variant due to its superior memory efficiency, using OMP [77] for signal recovery. The configuration mirrors PR, but incorporates a hash table; we increase the hash table allocation to 75% in high-memory scenarios to reduce computational overhead during reconstruction.
8. **Learned CM Sketch (LCM)** [29]: The idea of LCM is similar to AG, which is to store hot and cold elements separately. However, while AG relies on a data-exchange algorithm for classification, LCM employs a learned model. We implement an RNN-based classifier⁵ with a 10–20KB memory footprint for label prediction. The remaining memory budget is split equally (1:1) between the heavy-hitter (unique buckets) and light-hitter (CM) components.
9. **Learned Count Sketch (LCS)** [29]: LCS follows the exact configuration of LCM but substitutes the underlying Count Min Sketch with a Count Sketch.
10. **Learned Sketch without Predictions (LS)** [20]: LS extends LCS by introducing two key query-time optimizations. (i) Efficiency: To mitigate inference overhead, LS performs model predictions probabilistically rather than for every item; we set this sampling probability to 15% in our experiments. (ii) Accuracy: To improve precision for small elements, LS truncates low-value estimates to zero. The threshold is defined as $\frac{C \cdot kN}{m}$, where C is a constant (set to 10 in our experiments). Then, any query result obtained by its light part below this threshold is set to zero.

⁵The original implementation in the prior work [29] was based on TensorFlow for IP address identification. In our implementation, we re-implement this component using PyTorch. For data streams where such correlations are absent, we directly consider an idealized setting by removing the Top-K hot keys from the stream.

6.1.3 Hyperparameters

Data-plane structural parameters. The memory allocation of FLORE follows the NZE-sketch instance. Specifically, the total memory budget is initially partitioned: 50% is allocated to the Bloom Filter, while the remaining 50% is split equally between the Count-Min Sketch and the augmented filter. Both the Bloom Filter and the CM sketch are subject to capacity constraints: a maximum false-positive rate threshold for the Bloom Filter and a hard memory cap of 256KB for the CM sketch. If either limit is reached, any residual memory budget is reallocated to the augmented filter. For CUDA (GPU) implementation of FLORE, we vary the batch size from 100K to 1M and recompile the program for each configuration to evaluate its impact on GPU insertion throughput.

Control-plane model hyperparameters. We evaluate segment lengths from the set $\{512, 1024, 2048\}$. The hidden dimension of FLORE’s f -autoencoder is selected from $\{64, 128, 256, 512\}$. For the hidden representation of b , we set its dimension to $2/3$ of that of f , reserving the remaining $1/3$ for concatenation with the sampled Gaussian noise vector z . The number of cINN layers is chosen from $\{2, 3, 4, 5\}$. The final layer employs a TANH activation function, consistent with the encoder’s projection of inputs into the $[-1, 1]$ range.

Training parameters. The loss trade-off coefficients α_1 through α_4 are set to (or selected from) 0.5, 0.001, 0.01, and $\{0.01, 0.05, 0.1, 0.25\}$, respectively. Notably, α_2 is assigned a near-zero value, as we observed that the INN demonstrates near-perfect invertibility even without strong regularization. The initial training phase consists of 20~30 epochs, while subsequent fine-tuning to incorporate new elements needs 3~5 epochs. By default, the MMD metric is used to measure the loss term $\mathcal{L}_{\text{ort}}$, and the Adam optimizer [78] is employed throughout all experiments.

6.1.4 Metrics

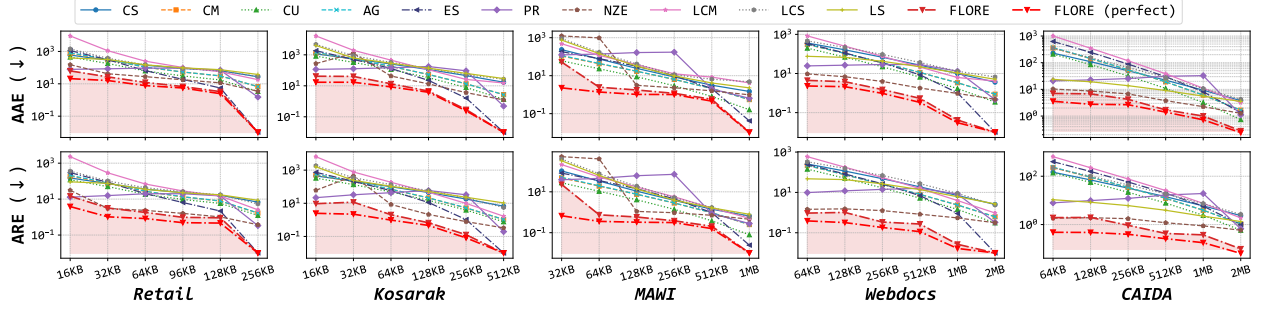
We evaluate the performance of sketching algorithms using the following metrics. Again, all experiments are repeated multiple times and the average results are reported.

- **Average Absolute Error (AAE):** $\frac{1}{N} \sum_i^N |f_i - \hat{f}_i|$, which measures the average magnitude of estimation errors.
- **Average Relative Error (ARE):** $\frac{1}{N} \sum_i^N \frac{|f_i - \hat{f}_i|}{f_i}$, used to evaluate the error normalized by the true element frequency. The ARE metric is highly sensitive to mouse elements, because their small true sizes reside in the denominator, even marginal overestimations yield extreme relative errors that inflate the reported mean.
- **Weighted Mean Relative Error (WMRE):** $\frac{\sum_{j=1}^z |F_j - \hat{F}_j|}{\sum_{j=1}^z \frac{F_j + \hat{F}_j}{2}}$, where z is the maximum element frequency, and F_j and $\hat{F}_j$ are the true and estimated counts of elements with frequency j . This metric assesses how well the sketch preserves the overall frequency distribution.
- **Entropy Absolute Error (AE):** $|H - \hat{H}|$, defined as the absolute difference between the true entropy H and the estimated entropy $\hat{H}$, where $H = -\sum_{i=1}^N \frac{f_i}{S} \log \frac{f_i}{S}$ and $S = \sum f_i$.
- **F1 Score:** $\frac{2 \cdot PR \cdot RR}{PR + RR}$, the harmonic mean of Precision (PR) and Recall (RR). Here, Recall is the ratio of correctly identified heavy hitters to the total number of true heavy hitters, and Precision is the ratio of correctly identified heavy hitters to the total number of reported instances.
- **Throughput:** Million operations (insertions/queries) per second (Mops).

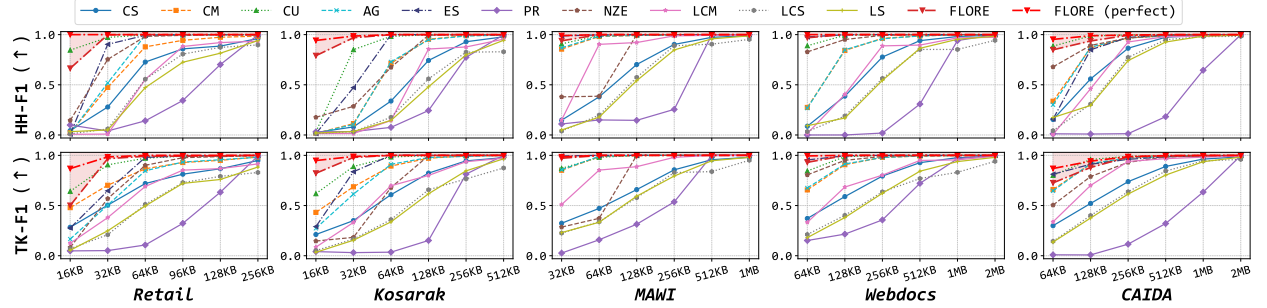
Note: when the key space is unknown, traditional approaches typically rely on optimization-based methods to approximate the cardinality or distribution of data streams from counter values, such as the basic MRAC algorithm [79]. In this paper, we assume that the key space is known or can be efficiently tracked. Accordingly, for a fair comparison, all traditional algorithms are allowed to directly leverage the known keys when detecting heavy hitters or estimating distribution (i.e., computing WMRE) and entropy.

6.2 Experiments on Real Data

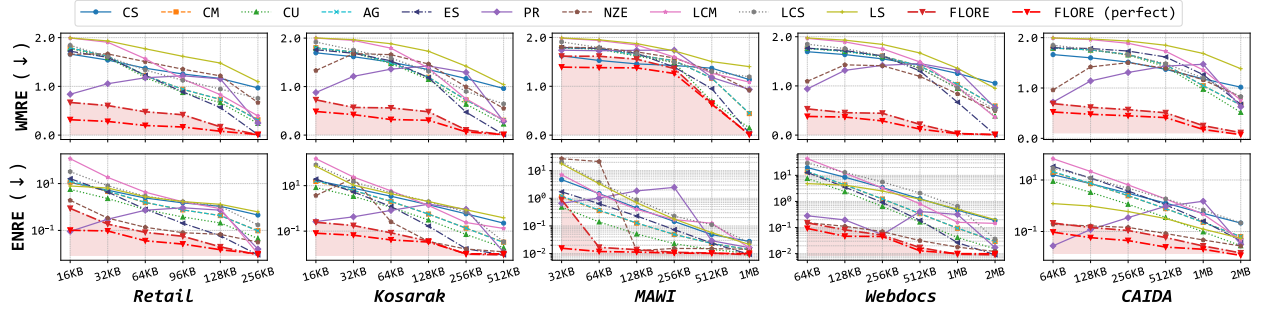
Per-element frequency recovery. We first evaluate the frequency estimation for both frequent and infrequent elements, with results shown in Fig. 16(a) (logarithmic scale). Generally, accuracy improves as memory increases. The baseline CM and its optimizations (AG, CU) provide stable improvements, whereas CS typically lags behind CM except under some small memory usage. When there is enough memory to identify and store all frequent elements, ES can perform very well but below certain memory threshold, its performance is average. Learning-augmented sketches (LCM, LCS) reveal a clear trade-off: they struggle in resource-constrained settings due to the high memory overhead of auxiliary



(a) Comparison of frequency estimation absolute (1st row) and relative (2nd row) errors.



(b) Comparison of heavy hitters (1st row) and Top-k (2nd row) detection accuracy.



(c) Comparison of distributional (1st row) and entropy (2nd row) distances to the GT.

Figure 16: Sketching performance on five real-world datasets.

structures (buckets/classifiers). While LS mitigates this via the truncation trick, its benefits diminish as memory expands. In contrast, CS-based methods (PR, NZE) achieve remarkable precision under high compression, which beat CM by two orders of magnitude, though their performance is non-monotonic due to the under-determined recovery process. We also observe that FLORE outperforms almost all methods by several orders of magnitude. At a 256KB budget, it reduces AAE and ARE significantly, consistently surpassing CM by over $400\times$, and usually dominates PR and NZE even without GT data for its learning. Meanwhile, FLORE (perfect) marginally outperforms the standard FLORE, as expected.

Heavy element identification. Next, we report the results for heavy-hitter detection and finding global top-k hot elements, where the predefined frequency threshold is 1,000 and the top $\log N$ frequent elements are selected, respectively. Fig. 16(b) compares the F1 Score of all solutions in heavy hitter (HH-F1) and top-k (TK-F1) detection under different memory sizes. As in the figure, HH-F1 and TK-F1 exhibit a totally same characteristics. Since only frequent elements are counted here, we observe the following phenomenon. First, CM and CU perform exceptionally well due to their inherent non-negativity. While AG employs an augmented filter to accurately track a small subset of hot items (e.g., ≤ 100 positions), its overall improvement remains limited. ES is free from this problem; however, it often failed under a low memory budget. Second, CS and its derivatives (LCS, LS) lack this non-negative property, resulting in inferior accuracy. Third, the performance of PR and NZE is strikingly reversed, with PR consistently yielding the lowest F1 Score. This is because these CS-based methods prioritize global estimation fidelity over the precise recovery

of individual heavy hitters. In contrast, FLORE and FLORE (perfect) consistently outperform all these sketch techniques, maintaining a much higher F1 Score across the board. Specifically, on nearly all datasets, FLORE achieves near-perfect detection accuracy (90%~100%) with 64 KB of memory. This advantage becomes even more pronounced under fewer counters, where FLORE delivers up to a $4.5\times$ improvement in average.

Estimation of distribution & entropy. We further evaluate the capacity of each sketch to model the underlying stream distribution, representing the fidelity of the recovery. As the 1st row of Fig. 16(c) shows, an impressive performance disparity can be observed: despite the absence of GT data during training, FLORE achieves a substantially lower discrepancy to the real distribution than all baselines. Taking the CAIDA dataset as an example, FLORE with a mere 64KB memory budget attains a level of fidelity that competing sketches cannot achieve even with 2MB, corresponding to a $32\times$ reduction in memory footprint for comparable accuracy. This advantage arises from FLORE’s generative sketching paradigm, which explicitly captures and exploits the statistical structure of the data stream. Furthermore, our results reveal that per-element estimation accuracy is not necessarily indicative of global distribution fidelity. For instance, while CS-based methods may perform well in point estimation, they struggle to faithfully recover the overall distribution. A similar trend is evident in the 2nd row of Fig. 16(c), which illustrates the advantages of our approach in entropy estimation. Interestingly, PR’s estimates are highly accurate under extreme memory scarcity but degrade sharply before stabilizing once the system transitions from under-determined to well-determined. In contrast, FLORE maintains much stable performance, with the ENRE consistently remaining near or below 0.1 on average. This represents an improvement of up to two orders of magnitude compared to baselines. These findings demonstrate that FLORE’s recovery is both precise and statistically consistent with GT, marking a clear departure from prior works.

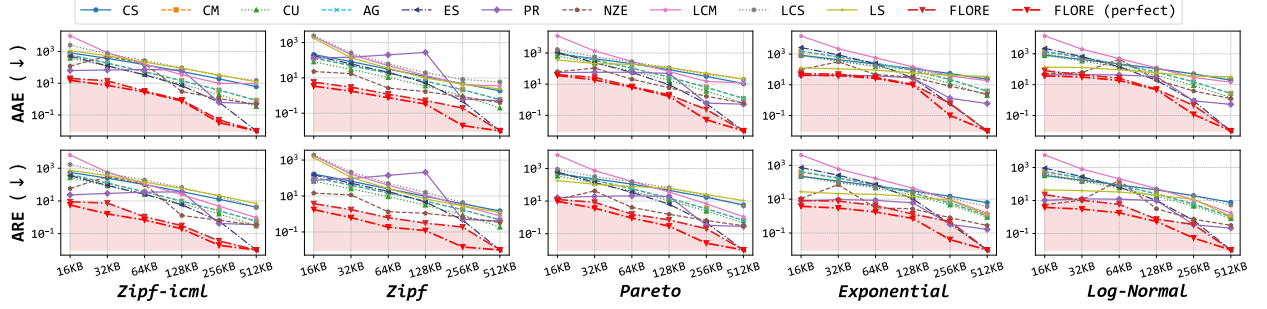
6.3 Experiments on Synthetic Data

To evaluate the adaptability of our method to data streams with different distributions, Figure 17 details its performance across five heavy-tailed distributions, including Zipf, Pareto, Exponential, and Log-Normal. The overall trend closely mirrors that observed on real-world datasets in the main text (with FLORE consistently achieving optimal or near-optimal results), yet the degree of distribution skew impacts performance.

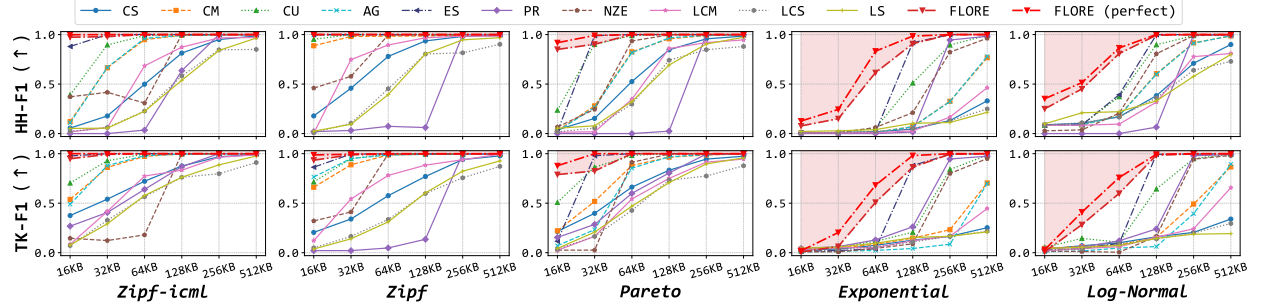
First, regarding per-element accuracy (Figure 17(a)): Under tight memory budgets, all non-CS-based approaches perform poorly. For instance, CM and CS typically incur errors on the order of 10^3 (both relative and absolute) at 16KB. Learning-augmented sketches fare even worse, as they require extra space to store a small subset of hot items and their classifiers. Interestingly, LS performs well under certain skew levels by directly truncating counters (e.g., highly skewed CAIDA, and moderately skewed Exponential and Log-Normal here), though this advantage quickly diminishes as memory increases. Among CS-based schemes, PR exhibits severe estimation instability, only converging to reasonable error levels when ample memory is available (as the system becomes determined and admits a unique solution). NZE steadily improves accuracy with more memory, making it the most accurate sketch algorithm (trading computational complexity for performance), but it is still outperformed by FLORE. Specifically, FLORE reduces error by up to $700\times$ compared to other methods and achieves near-zero error at memory sizes of 256KB and above.

Second, regarding heavy-hitter accuracy (Figure 17(b)): The behavior under the Zipf distributions closely mirrors that of the real-world distributions presented in the main text, where FLORE always achieves near 100% accuracy, requiring only 16KB. However, “surprises” emerge under the Exponential and Log-Normal distributions. Almost all methods fail to detect heavy hitters effectively under a low skewness, as the keys become less distinguishable. Notably, CM performs worse than usual, even falling behind PR (which typically ranks last), with its peak F1 score barely reaching around 80%. Other approaches, including CS, optimizations, and learning-augmented sketches, suffer similarly. LS is particularly noteworthy: its counter-truncation strategy yields unusually low accuracy on these latter two distributions. In contrast, FLORE not only consistently sits at the top of the curves but also demonstrates remarkable recovery: although its performance at 16KB is subpar, it rebounds almost linearly thereafter, pulling ahead of all competitors by a wide margin. Specifically, FLORE achieves roughly $2\times$ to $4\times$ higher F1 scores.

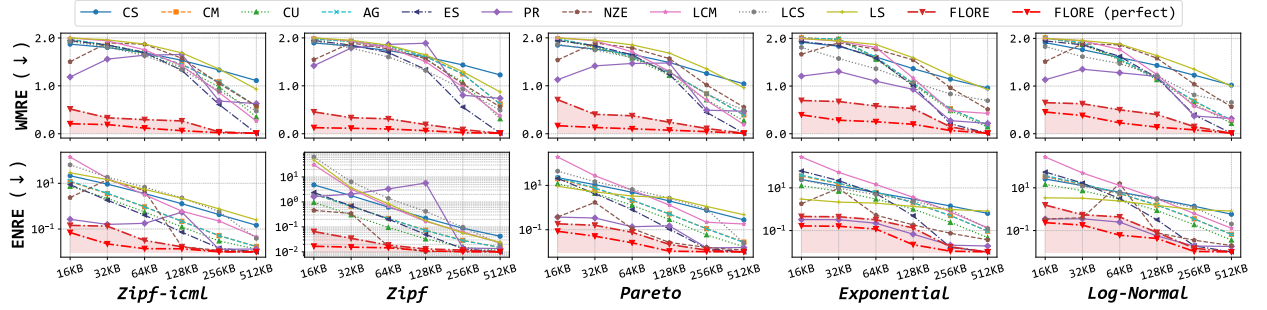
Third, regarding distribution alignment (Figure 17(c)): Thanks to its generative nature, though it is fully GT-free, FLORE continues to dominate decisively on these metrics. Specifically, FLORE’s WMRE rarely exceeds 0.5, while most baseline methods require as much as 512KB of memory to achieve comparable accuracy. Even more strikingly, under tight memory budgets, FLORE’s error is less than $1/3$ that of baselines. Moreover, even high-accuracy CS-based methods like PR and NZE suffer from inherent limitations in distribution modeling. Notably, NZE’s enforced sparsity during reconstruction inevitably compromises fidelity to the true underlying distribution. Regarding entropy estimation, although low-skew distributions narrow the performance gap between FLORE and other methods, FLORE still significantly outperforms all non-CS-based approaches. For example, on the Zipf dataset, FLORE’s absolute entropy estimation error is, on average, $105.14\times$ and $712.98\times$ smaller than that of CM and CS, respectively. Another observation is that PR’s performance improves noticeably in the bottom two rows, sometimes even surpassing FLORE, but its gains remain very



(a) Comparison of frequency estimation absolute (1st row) and relative (2nd row) errors.



(b) Comparison of heavy hitters (1st row) and Top-k (2nd row) detection accuracy.



(c) Comparison of distributional (1st row) and entropy (2nd row) distances to the GT.

Figure 17: Sketching performance on five synthetic datasets.

limited when ample memory is available. We attribute this stagnation to the solver’s inability to find a sparser solution, causing it to plateau.

In summary, the results across diverse synthetic distributions show that FLORE generalizes well to different patterns and consistently achieves superior accuracy under tight memory budgets.

6.4 Results on FLORE’s Efficiency

Owing to the data-control-plane separation design, FLORE is expected to achieve both high runtime efficiency and fast training convergence. We validate this property by evaluating its stream processing throughput, recovery speed, and convergence behavior.

6.4.1 Stream Processing Speed

Figure 18 presents the scaled stream processing speed relative to the CM baseline. We also implement and evaluate FLORE on various platforms, such as GPU using CUDA Toolkit [80]. Note that, due to the data-control-plane separation design, sketch summarization is performed on the CPU, while recovery is executed in parallel on the GPU through tensor operations. The results indicate that learning-augmented sketches suffer from low efficiency due to the required classifier forward pass. Although LS limits the usage of Oracle (to 15% of queries here), its performance is still suboptimal.

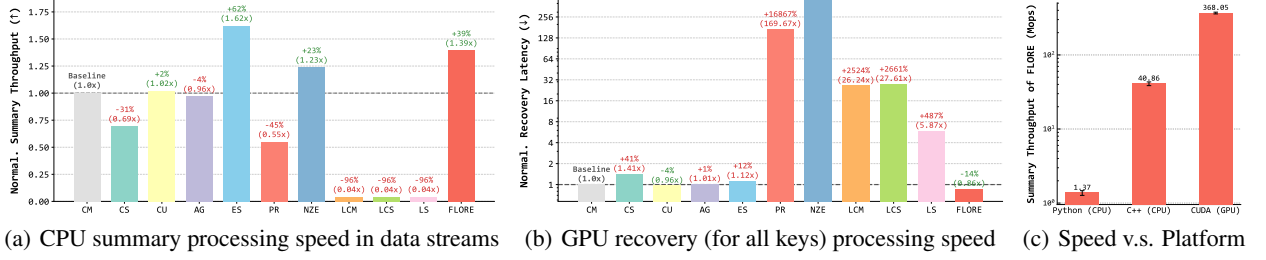


Figure 18: Comparison of stream processing speed on CAIDA with 1MB memory (Green means better than CM).

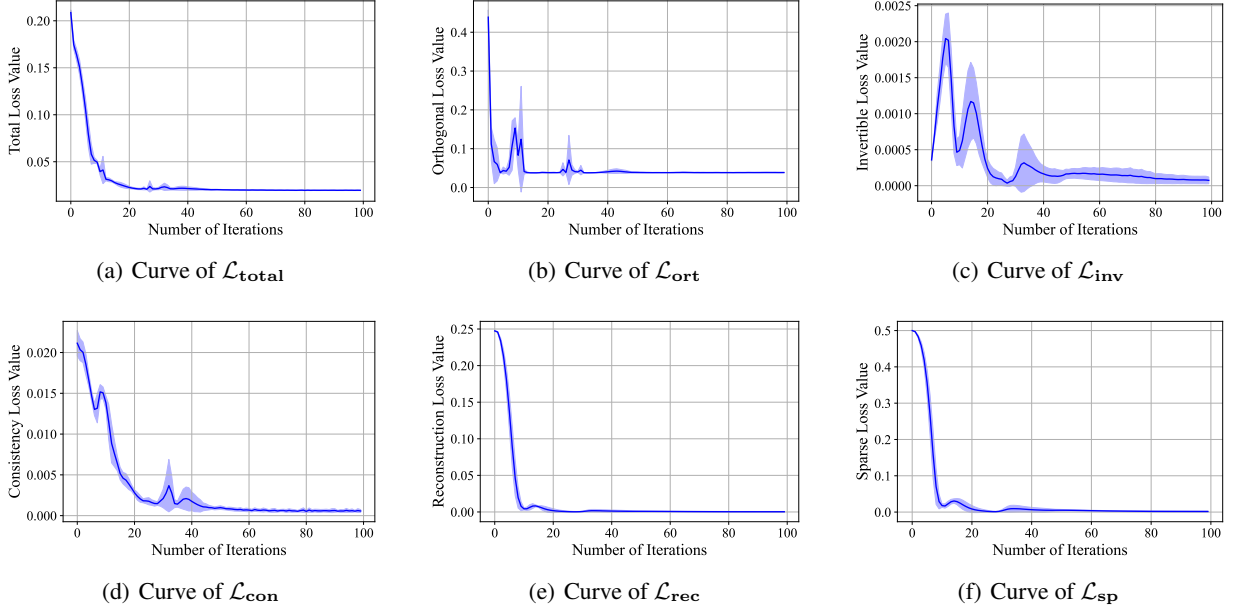


Figure 19: Training loss curves of FLORE on the CAIDA dataset.

CS decoding algorithms need over 10^2 times longer recovery time to get a reliable result, which is unacceptably high. Conversely, FLORE incurs no overhead on the data-plane stream processing as it is used solely for recovery, and filtered stream with $\mathcal{O}(1)$ complexity, which outperforms the baseline in both summary and recovery phases, with gains becoming even more pronounced across different platforms (see Figure 18(c) in the right). Specifically, the speed of FLORE on Cuda and C++ is 29.8, 268.6 times higher than in Python. We can also see that the summary throughput of FLORE is less than ES because the Bloom Filter will add extra operations when item replacement occurs.

6.4.2 Convergence Time

We indeed discuss at the start of §4 the benefits that a “perfect generator” could bring to the linear sketching problem. However, for Theorem 3 in our framework design, such an idealized setting (losses = 0) is not required; in practice, approximate satisfaction (≈ 0) is sufficient to approximate the true posterior $p(f | b)$. Compared to prior works (e.g., see [29, 20] that needs a perfect oracle), our assumption is much more weaker. Empirically, we conduct experiments to train FLORE on the CAIDA dataset using a 128KB memory. The curve of all losses can be found in Figure 19, and we observe that all losses are reduced to nearly-zero values during training. In particular, $\mathcal{L}_{ort}$ (i.e., the MMD between $[z, b]$ and $G(z, b)$) is driven to consistently small and stable levels (i.e., a good z - b independence), while $\mathcal{L}_{inv}$ is always $< 10^{-2}$, implying that the underlying model is strictly invertible. More importantly, each iteration typically completes within tens of milliseconds, meaning the model converges within tens of seconds and reaches a reasonable loss range in under ten seconds. Therefore, our method fully supports online iterative updates and is well suited for real-world database or network monitoring system deployment.

6.5 Robustness to Stream Changes

In what follows, we evaluate the robustness of FLORE under temporal, natural, and spatial variations in data streams. In these experiments, we randomly sample 10% to 20% of the key set to simulate changes in stream behavior in each sampling epoch, following the prior work [43]. All experiments are conducted on the CAIDA dataset.

Memory		Fluctuation Factor			
		0.2	0.5	1.0	2.0
128KB	average	0.0%	-1.0%	-1.8%	2.7%
	90th Pct.	0.4%	-0.2%	-2.1%	5.9%
256KB	average	-0.0%	-1.1%	-1.8%	1.9%
	90th Pct.	0.2%	-0.3%	-1.4%	2.8%
512KB	average	0.2%	0.0%	-1.0%	1.0%
	90th Pct.	0.3%	-0.0%	1.1%	1.3%
1MB	average	0.0%	-1.9%	0.2%	-0.3%
	90th Pct.	1.1%	-0.8%	0.2%	-0.5%

Table 6: ARE Performance decline with increased CAIDA network traffic fluctuation. Negative values indicate no degradation.

Memory		Training stream time windows		
		0%-25%	25%-50%	50%-75%
128KB	average	5.4%	3.6%	4.4%
	90th Pct.	9.6%	7.8%	8.7%
256KB	average	2.6%	3.2%	2.1%
	90th Pct.	4.0%	5.7%	4.6%
512KB	average	0.2%	1.8%	0.5%
	90th Pct.	0.9%	0.2%	1.0%
1MB	average	0.5%	1.6%	-1.5%
	90th Pct.	0.0%	1.0%	1.6%

Table 7: ARE Performance decline with natural drift in CAIDA network traffic. Negative values indicate no degradation.

Memory		Proportion Factor			
		0.9	0.5	0.25	0.1
128KB	average	0.3%	0.7%	1.8%	2.3%
	90th Pct.	1.7%	1.0%	3.1%	6.5%
256KB	average	0.2%	0.6%	1.5%	2.6%
	90th Pct.	0.4%	0.7%	-0.4%	2.4%
512KB	average	0.0%	-0.6%	1.7%	1.3%
	90th Pct.	0.2%	0.4%	1.4%	0.6%
1MB	average	0.5%	0.7%	-1.0%	-1.6%
	90th Pct.	0.0%	-0.6%	-0.4%	1.4%

Table 8: ARE Performance decline with spatial shift in CAIDA network traffic. Negative values indicate no degradation.

Temporal changes. To increase temporal variability, we proceed as follows. For each selected key, we generate a temporal change in one epoch by sampling from a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ and scaling it by a factor, where $\mu = 0$ and σ is the standard deviation across different sampling epochs of the key. The scaling factor $\in \{0.2, 0.5, 1.0, 2.0\}$ controls the amplitude of the fluctuation. We evaluate the impact of sudden changes by measuring the ARE degradation of FLORE relative to its performance under stable conditions. The results are summarized in Table 6. When the scaling factor is relatively small, FLORE exhibits no noticeable performance degradation. Even when the scaling factor = 2, the performance degradation is still within 10%.

Natural changes. To test the impact of natural shifts, we conduct training separately with 0%-25%, 25%-50%, and 50%-75% of the sampled counters, followed by testing on the last 25%. Note that evaluation is performed only on the selected keys that appear in all four parts. This protocol enables us to quantify performance degradation relative to the setting in which 75% of the data is used for training. The results w.r.t. ARE are summarized in Table 7. We observe that FLORE maintains stable performance on the selected keys even long after the completion of training (exceeding two times the total duration of the training data).

Spatial changes. To simulate changes in the spatial distribution of frequencies, we redistribute frequencies across different keys in one epoch. Specifically, we reassign the top 10% of hot keys to the selected keys such that they account for 90%, 50%, 25%, and 10% of their original total volume. We refer to the resulting scaling parameter as the proportion factor $\in \{0.9, 0.5, 0.25, 0.1\}$, which controls the magnitude of the spatial change. As presented in Table 8, the performance variations are not significant across all spatial distributions. Compared to the results in Table 6, although there is indeed a slightly greater decline in FLORE’s performance, the impact is still limited, and FLORE consistently maintains good accuracy across different proportion factors.

Overall, FLORE is robust to changing conditions in the real world. We attribute this partly to the GM’s ability to capture the underlying distribution inherent to the sketching problem, while compressive sensing enables the model to effectively detect anomalous changes [81].

6.6 Interpreting FLORE’s Effectiveness

Finally, we perform several ablation studies to assess the impact of FLORE’s key features on its overall performance.

6.6.1 Effect of Generative Model

To further validate our generative method, we replaced the INN-based generator with four SOTA CS algorithms: LSQR [82], OMP [77], LASSO [83], and LSMR [75]. Since Flore is built on top of Count-Min Sketch, a Count-Min query algorithm (i.e., CM) is included here as a traditional benchmark. The main results are shown in Figure 20. We conduct experiments on five relatively small datasets, since these CS algorithms are often very resource-consuming (see results discussed later). Overall, our method consistently outperforms the CS baselines in terms of accuracy. More notably, the improvement in distribution fidelity is substantially larger, achieving on average approximately a 65% gain over the best baseline.

Also, we compare the computation time for FLORE with other LP-based CS algorithms in Figure 21. As the stream size increases, we observe that FLORE demonstrates scalable performance precisely as intended, requiring less than 15 ms

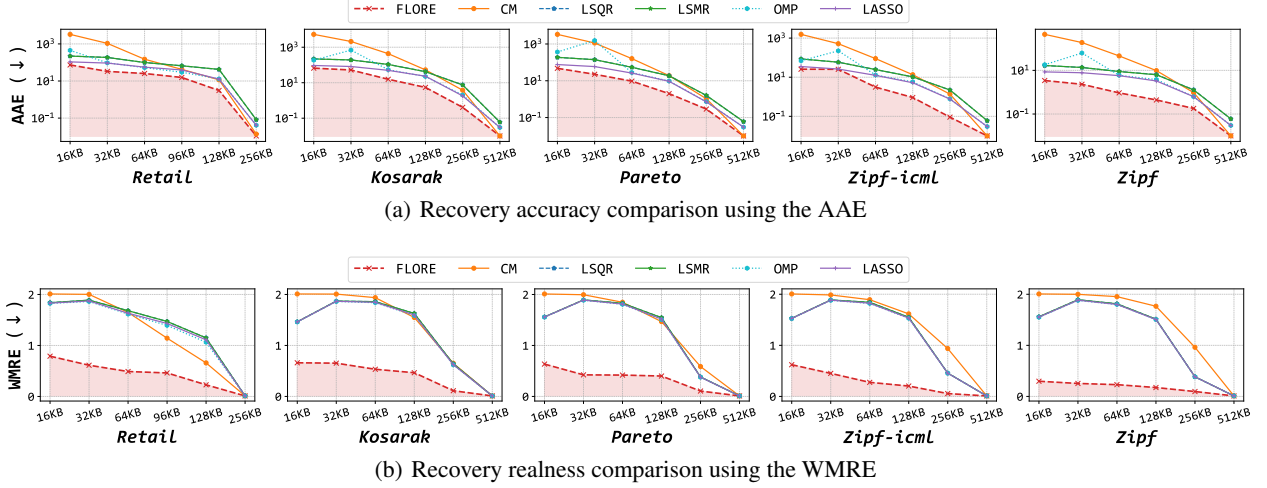


Figure 20: Comparing FLORE with the Count-Min and SOTA compressive sensing algorithms by replacing the FGM.

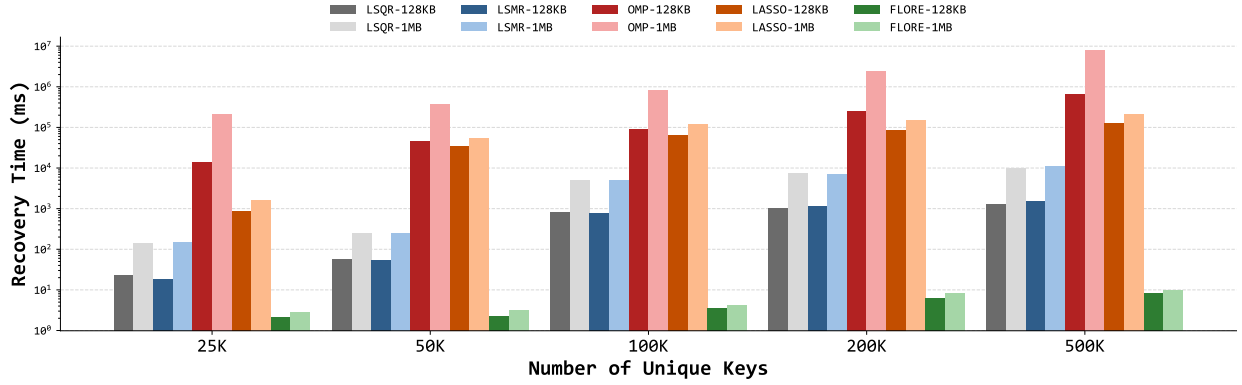


Figure 21: Comparison of computation times for recovery procedure.

of computation time while achieving better recovery. When the number of unique keys is 500K, for instance, FLORE achieves $10^2 \sim 10^6 \times$ speedups relative to the baselines, accentuating the importance of employing an FGM in the control plane.

6.6.2 Effect of Training Objectives

We next evaluate the impact of training losses used by FLORE. In accordance with our design choices, we consider the following five FLORE variants for comparison. (i) **Naive**: This variant removes all generative losses (i.e., $\mathcal{L}_{inv}$, $\mathcal{L}_{ort}$, and $\mathcal{L}_{rec}$) in the FGM and degenerates into a simple one-way mapping function. (ii) **FLORE w/o sp.**: This variant removes the sparsity constraint (i.e., $\mathcal{L}_{sp}$) introduced by compressed sensing theory. (iii) **FLORE w/o ort.**: This variant eliminates the orthogonality constraint (i.e., $\mathcal{L}_{ort}$) between z and b . (iv) **FLORE w/o rec.**: This variant discards the reconstruction guidance (i.e., $\mathcal{L}_{rec}$) provided by CM + EM and relies solely on counter-based learning. (v) **FLORE w/o con.**: This variant removes the explicit linear equation constraints (i.e., $\mathcal{L}_{con}$), leaving the model to learn without structural supervision. As shown in Figure 22 and Figure 23, we plot the cumulative distribution function (CDF) that shows absolute/relative errors achieved by FLORE and its variants on Kosarak and Zipf, respectively.

The simple mapping function **Naive** exhibits the most severe performance degradation, underscoring the critical role of the generative paradigm in recovery. The variant **FLORE w/o rec.** performs better than **Naive** but still suffers from a substantial loss in accuracy. This behavior arises because a generative model requires an effective “information channel” to anchor its learning to the underlying GT vector, a role that is fulfilled by our EM algorithm. While not as critically important as the two aforementioned loss terms, the other three variants demonstrate their impact through fine-grained adjustments. In particular, these variants tend to produce a small number of outliers with exceptionally large errors. For example, on the Kosarak dataset with a 16KB budget, **FLORE w/o ort.**, **FLORE w/o con.**, and **FLORE w/o**

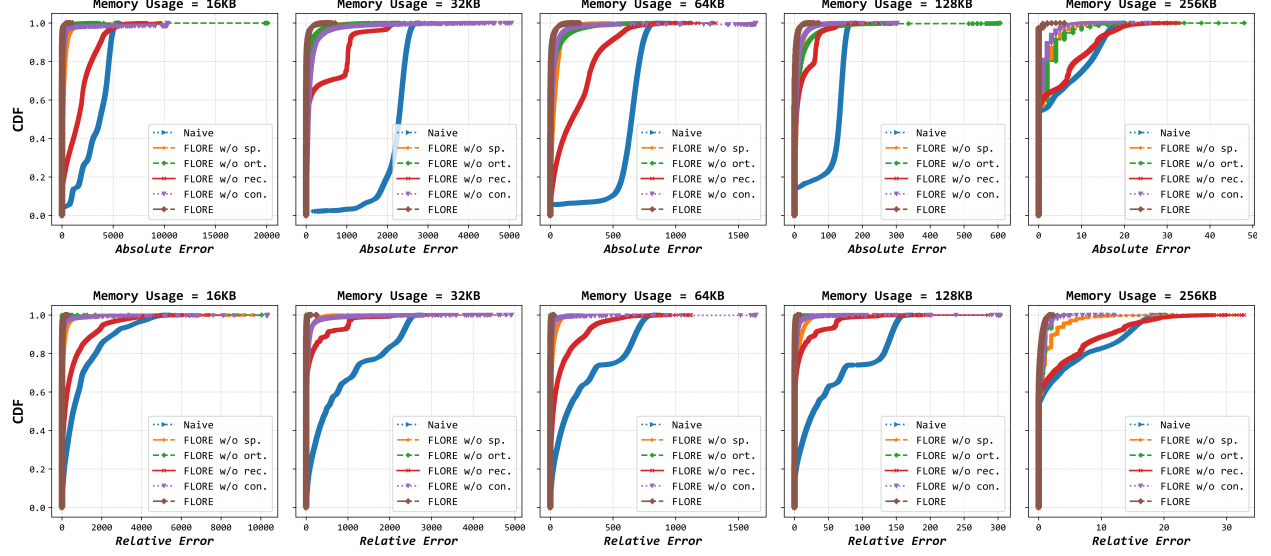


Figure 22: CDFs for the AAE and ARE of FLORE and its variants on the real-world Kosarak dataset.

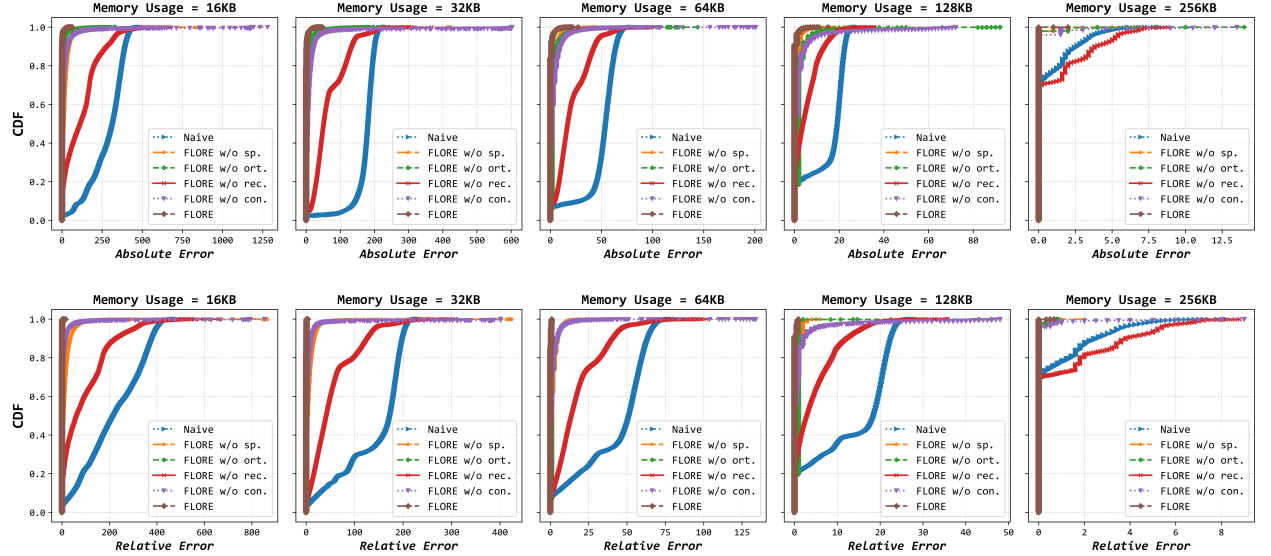
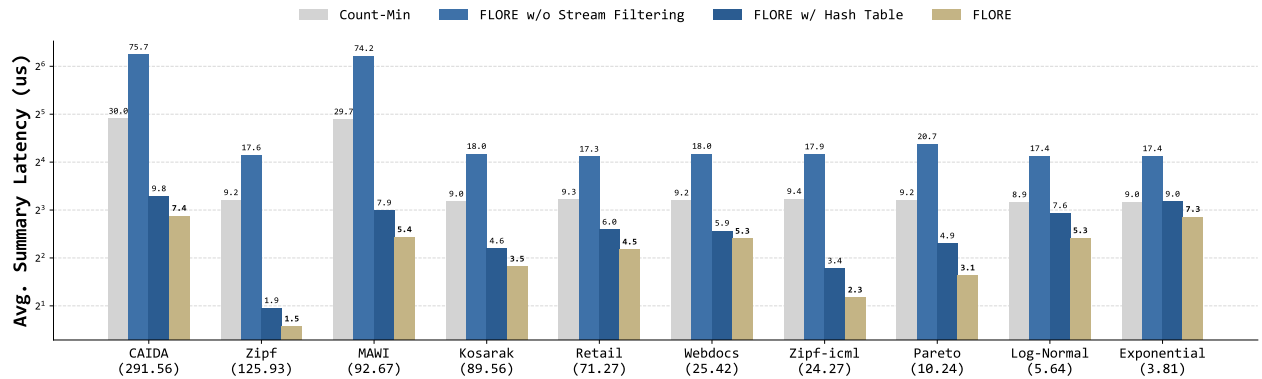


Figure 23: CDFs for the AAE and ARE of FLORE and its variants on the synthetic Zipf dataset.


 Figure 24: Ablation comparing FLORE and FLORE without the stream filtering mechanism or separation design, where (x) at the bottom indicates that the data stream has skewness x .

sp. each exhibit data points with absolute errors of 20,000, 10,000, and 8,000, respectively. Interestingly, although the consistency enforced by the linear equations is intuitively expected to play a central role, directly removing it does not cause the sketch measurement system to collapse. This is because the solution obtained by our EM algorithm already approximately satisfies this constraint, thereby implicitly guiding the model to learn the desired relationship. In contrast, FLORE exhibits consistently stable performance across all scenarios: more than 90% of its restoration errors lie within the lowest 10% of error magnitudes, and such extreme outliers are almost entirely eliminated. These observations corroborate the effectiveness of each training loss component in FLORE.

6.6.3 Effect of Stream Filtering

Because the key-value pairs stored in the augmented filter are guaranteed to be accurate, we further conduct an analysis on the processing speed improvement brought by FLORE’s stream filtering mechanism, as indicated by “FLORE w/o Stream Filtering”. To enable a finer-grained dissection of effective components, we also introduce two additional baselines: Count-Min and FLORE w/ Hash Table. The former is a standard Count-Min sketch that completely removes the idea of separation, representing a naive data stream summarization approach without key tracking mechanism. The latter replaces FLORE’s Ostracism-based filtering algorithm with a simpler hash-table strategy (following the design of NZE-sketch) to investigate the impact of the proposed algorithmic design. We report the average time (or latency) required to summarize a single data item across all ten datasets in Figure 24. From the figure, we have three important findings. First, FLORE w/o Stream Filtering is even slower than Count-Min because it requires additional time to consult the Bloom Filter to determine whether a key belongs to the existing set. Second, even simply adding a hash table (i.e., FLORE w/ Hash Table) dramatically reduces processing latency, by more than 5× on average, and the performance gain becomes even greater as the skewness of the data stream increases. Third, our Ostracism-based mechanism further accelerates processing speed. Although FLORE involves a lookup operation, this operation runs in constant time and yields an additional 15% improvement over the hash-table baseline.

7 Related Works

Classical sketching algorithms Sketches are classical streaming algorithms and compact data structures that support approximate queries over large-scale data. Because streaming data comes at them continuously, and in such volume, they try to record the essence of what they’ve seen while strategically forgetting the rest. For more than 30 years, computer scientists have worked to build a better sketching algorithm. Representative examples include Count-Min Sketch [25] and Count Sketch [26], which enable fast updates and queries with provable accuracy bounds. These sketches typically maintain an array of m counters organized into k rows, together with k pairwise-independent hash functions that map each item to one counter per row. For instance, in Count-Min Sketch, each update increments the k counters indexed by the hashed positions of the item. To estimate an item’s frequency, the sketch hashes the queried item and returns the minimum value among the corresponding counters across all rows. When increments are positive, a simple modification known as Conservative Update [73] leads to better accuracy in frequency estimation. It avoids unnecessary increments by first comparing stored counts for an item and then only incrementing the smallest ones [84]. Moreover, Augmented sketch [12] uses an extra elephant-flow recording structure to further improve accuracy. Due to the skewed nature of data streams, it can potentially provide even higher throughput. Based on this idea, Pyramid sketch [85], Elastic sketch [27], HeavyGuardian [86], HeavyKeeper [87], and Dynamic Hierarchical Sketch [88] all store the elephant and mice flow information separately in the same space. As these sketches use the hash value as the index, they can support $\mathcal{O}(1)$ insertion and query, but they have suboptimal estimation accuracy of per-element frequency, which in turn greatly reduces the accuracy of other statistics. Additional sketches have been proposed for specialized tasks, such as burst/DDoS victim detection [89, 90], graph summarization [7], reporting top- k items [91, 92], and super-spreader query [93]. However, they are typically optimized for a narrow set of queries and rely on carefully engineered heuristics. By comparison, our proposal enjoys a tighter theoretical error bound while remaining applicable to a broad class of sketching tasks.

Learning-based sketching algorithms In recent years, researchers have attempted to use machine learning (ML) methods to improve sketch performance. This type of sketching algorithm was first explored by [94] and [28]. The former tries to learn task-specific representations, while the latter trains a classifier to distinguish frequent data items; the overall idea and architecture are consistent with Augmented Sketch. Specifically, the learning-augmented sketch uses neural networks (NN), e.g., an RNN [29], to learn and infer whether an item is large and uses an additional lookup table to record large items. This general paradigm of leveraging ML to improve hashing-based sketches has inspired a series of follow-up works [95, 96, 66, 97, 62]. However, because a forward pass through an ML model is required prior to each hash operation, these methods often incur substantial computational overhead, resulting in significantly reduced stream processing throughput. This issue becomes more pronounced as model complexity increases; for example, a recent study [98] employs a large language model (LLM) as a learned “oracle”, which is typically impractical for high-speed

data streams. Although [15] and [20] partially mitigate this efficiency bottleneck by reducing or even eliminating the need for per-item model inference, a second fundamental challenge remains: the reliance on continuously collected ground-truth data to train the neural models. To address this limitation, Meta-sketch [11] is the first learning-based sketch that avoids ground-truth supervision by performing meta-learning over synthetically generated Zipfian datasets. This GT-free learning paradigm has subsequently been adopted and refined in Mayfly [99], Lego-sketch [100], and RatioSketch [101]. Nevertheless, beyond the open question of whether Zipfian assumptions consistently hold in real-world workloads, the neural architectures employed by these methods still exhibit much lower processing throughput compared to conventional, non-learning-based sketches. In contrast to the aforementioned sketches, our method is grounded in the theory of compressed sensing. It is both more practical and more efficient, as it requires no modification to the classical summary algorithm and relies only on sampled counters for training.

Decoding-based measurement The idea of decoding or recovery from coarse-grained, low-cost measurements obtained via linear systems can be traced back to Network Tomography [102]. By inferring the full end-to-end traffic matrix from more measurable link loads, this technique sparked a surge of research interest in network measurement [103, 104, 105, 106, 107, 81, 108, 109]. However, traffic matrices are defined at the wide area network (WAN) level and are far coarser than five-tuple packet measurements via data sketching, making them unsuitable for fine-grained applications and thus beyond the scope of this work. Turning to sketching algorithms, [110] were the first to apply the Least Linear Square method to reconstruct flow values from Count-Min Sketch. While Counter-Braids [111] highlighted the thematic connection between sketching and compressive sensing, it did not actually employ compressive sensing. In contrast, FlowRadar [18] and SketchVisor [13] explicitly demonstrated that sketches can be formulated as a special case of compressive sensing. Specifically, they typically use a Bloom filter to record unique keys, and at the end of each measurement epoch, leverage this information to recover the complete measurement results using compressed sensing algorithms. This encoding–decoding paradigm culminated in NZE-sketch [19] and PR-sketch [17], subsequently inspiring a line of research on sketching algorithms for high-precision measurement [112, 113, 52, 114]. However, the improved accuracy of these approaches comes at the cost of query efficiency: both NZE-sketch and PR-sketch solve regularized optimization problems involving potentially millions of variables, which introduces significant complexity even with state-of-the-art numerical solvers. Our work is inspired by UCL-sketch [43], which addresses this challenge using a learned solver. Similar to Meta-sketch, UCL-sketch relies on prior assumptions about the underlying data stream distribution. Different from them, this work leverages generative modeling to automatically infer and learn the distribution, thereby circumventing these potential limitations. Although some recent works [115, 116] have introduced deep generative models, such as GANs and autoregressive models, into network monitoring tasks, they neither perform systematic theoretical or experimental analysis nor avoid reliance on large pre-collected datasets for offline training.

8 Conclusion

This work revisits linear sketching techniques and highlights their limitations. We find that under resource (memory) constraints, existing techniques are fundamentally insufficient for near-perfect recovery. To bridge this gap, we introduce a novel generative sketching paradigm that reframes the problem as posterior generation. These analyses guide us to design FLORE, a GT-free ML framework, accordingly. The superior performance validates its effectiveness. We expect that more generative streaming algorithms can be designed in the future.

At the end of this work, we discuss the practical applicability of FLORE across a range of sketch-based tasks and outline its current limitations along with promising directions for future work.

Applicable Scenarios First, we have demonstrated the applicability of FLORE to four tasks, including *frequency estimation*, *heavy hitter detection*, *distribution estimation*, and *entropy estimation*. Since our analysis covers almost all linear aggregation scenarios, which are the most fundamental statistics, the proposed approach can also perform well on all the following use cases. (i) *Heavy changer detection* [117]. The task aims to identify the elements whose frequency changes between two adjacent epochs are larger than the predefined threshold, with applications to imply the existence of a DDoS attack [118]. (ii) *Blackhole detection* [119]. In this task, the blackhole behavior is modeled by a normal entity A and a faulty entity B. Specifically, when the data stream is transmitted from entity A to entity B, entity B drops the items associated with certain specific keys due to an interface failure. The goal of the task is to identify the missing (i.e., victim) keys. (iii) *Incorrect routing detection* [120]. Consider three entities A, B, and C, where entity A can send data items to both entity B and entity C. Incorrect routing refers to the scenario where, due to corruption in entity A’s routing table, a subset of keys that should have been sent to entity B are mistakenly forwarded to entity C. The goal of the task is to identify the keys affected by this incorrect routing. The above three binary classification tasks can be naturally derived from our heavy-hitter detection task. (iv) *Norm estimation* [121]. The problem of ℓ_p norm estimation is to provide a multiplicative approximation to $|f|_p := (\sum_i |f_i|^p)^{1/p}$, which is similar to distribution & entropy estimation. (v) *Graph sketching* [122]. The task falls into the scope of combinatorial

problems on graphs. We consider dynamic graph streams where edges can be both inserted and deleted. The input is a sequence $S = \langle a_1, a_2, \dots \rangle$, where $a_i = (e_i, \Delta_i)$ with e_i encoding an undirected edge as before, and $\Delta_i \in \{-1, 1\}$. The multiplicity of an edge e is defined as $f_e = \sum_{i: e_i=e} \Delta_i$. For simplicity, we restrict our attention to the case where f_e is in $\{0, 1\}$ for all edges. Then, let $f \in \{0, 1\}^{\binom{N}{2}}$ be the vector whose entries equal the current edge multiplicities f_e , and let $\mathcal{A}(f) \in \mathbb{R}^m$ denote the sketch of this vector. Upon arrival of an update (e, Δ) , we can simply update $\mathcal{A}(f)$ as: $\mathcal{A}(f) \leftarrow \mathcal{A}(f) + \Delta \cdot \mathcal{A}(\mathbf{i}^e)$ where $\mathbf{i}^e$ is the vector that is “1” only in the coordinate corresponding to edge e , and zero elsewhere. We can see that the formulation of this problem is entirely analogous to our linear sketching problem. Thus, it suffices to store the current sketch and leverage FLORE to compute the solution.

Limitations and Future Research We believe that our investigation of generative optimization for data sketching has but scratched the surface. In what follows, we outline current limitations of our approach, as well as intriguing directions for future research. (i) In the FLORE’s data structures, a Bloom Filter is employed to determine whether an element belongs to a known set. As a probabilistic data structure, however, the Bloom Filter may incur false positives, and achieving an extremely low false-positive rate requires memory usage that grows nearly linearly with N . To mitigate this overhead, future work may explore learning-enhanced Bloom Filter [28], biasing key mappings toward higher-indexed regions of the bit array can substantially reduce storage requirements while preserving query accuracy. (ii) Note that our method reconstructs the state information of all elements simultaneously, which is not well-suited for point queries. For instance, after the model outputs f , we must locate the position corresponding to a specific key and retrieve its value, an operation that incurs $\mathcal{O}(N)$ complexity. A good direction for future research is the development of accelerated mapping indices. By decoupling the generative reconstruction from the lookup mechanism, we aim to achieve sub-linear or constant-time retrieval for specific keys, thereby enhancing FLORE’s utility for real-time, per-element monitoring.

References

- [1] Balachander Krishnamurthy, Subhabrata Sen, Yin Zhang, and Yan Chen. Sketch-based change detection: Methods, evaluation, and applications. In *Proceedings of the 3rd ACM SIGCOMM conference on Internet measurement*, pages 234–247, 2003.
- [2] Graham Cormode. Data sketching. *Communications of the ACM*, 60(9):48–55, 2017.
- [3] Kasper Green Larsen, Jelani Nelson, Huy L Nguyen, and Mikkel Thorup. Heavy hitters via cluster-preserving clustering. *Communications of the ACM*, 62(8):95–100, 2019.
- [4] Burton H Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [5] Jayadev Misra and David Gries. Finding repeated elements. *Science of computer programming*, 2(2):143–152, 1982.
- [6] Amit Chakrabarti, Khanh Do Ba, and S Muthukrishnan. Estimating entropy and entropy norm on data streams. *Internet Mathematics*, 3(1):63–78, 2006.
- [7] Nan Tang, Qing Chen, and Prasenjit Mitra. Graph stream summarization: From big bang to big crunch. In *Proceedings of the 2016 International Conference on Management of Data*, pages 1481–1496, 2016.
- [8] Graham Cormode and S Muthukrishnan. Towards an algorithmic theory of compressed sensing. *Center for Discrete Math. and Comp. Sci.(DIMACS), Tech. Rep. TR*, 25:2005, 2005.
- [9] Graham Cormode. Sketch techniques for approximate query processing. *Foundations and Trends in Databases. NOW publishers*, 15, 2011.
- [10] Yi Li, Huy L. Nguyen, and David P. Woodruff. Turnstile streaming algorithms might as well be linear sketches. In *Proceedings of the Forty-Sixth Annual ACM Symposium on Theory of Computing*, page 174–183, 2014.
- [11] Yukun Cao, Yuan Feng, Hairu Wang, Xike Xie, and S Kevin Zhou. Learning to sketch: A neural approach to item frequency estimation in streaming data. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(11):7136–7153, 2024.
- [12] Pratanu Roy, Arijit Khan, and Gustavo Alonso. Augmented sketch: Faster and more accurate stream processing. In *Proceedings of the 2016 International Conference on Management of Data*, pages 1449–1463, 2016.
- [13] Qun Huang, Xin Jin, Patrick PC Lee, Runhui Li, Lu Tang, Yi-Chao Chen, and Gong Zhang. Sketchvisor: Robust network measurement for software packet processing. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 113–126, 2017.
- [14] S Foucart and H Rauhut. A mathematical introduction to compressive sensing. *The Annals of Statistics*, 50(6):3538–3562, 2022.
- [15] Francesco Da Dalt, Simon Scherrer, and Adrian Perrig. Bayesian sketches for volume estimation in data streams. *Proceedings of the VLDB Endowment*, 16(4):657–669, 2022.
- [16] Francesco Da Dalt and Adrian Perrig. Bfes: Towards optimal bayesian frequency estimation sketches in data-streams. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, pages 2337–2350. IEEE, 2025.
- [17] Siyuan Sheng, Qun Huang, Sa Wang, and Yungang Bao. Pr-sketch: monitoring per-key aggregation of streaming data with nearly full accuracy. *Proceedings of the VLDB Endowment*, 14(10):1783–1796, 2021.
- [18] Yuliang Li, Rui Miao, Changhoon Kim, and Minlan Yu. Flowradar: A better netflow for data centers. In *13th USENIX symposium on networked systems design and implementation (NSDI 16)*, pages 311–324, 2016.
- [19] Qun Huang, Siyuan Sheng, Xiang Chen, Yungang Bao, Rui Zhang, Yanwei Xu, and Gong Zhang. Toward nearly-zero-error sketching via compressive sensing. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 1027–1044, 2021.
- [20] Anders Aamand, Justin Chen, Huy Nguyen, Sandeep Silwal, and Ali Vakilian. Improved frequency estimation algorithms with and without predictions. *Advances in Neural Information Processing Systems*, 36:14387–14399, 2023.
- [21] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- [22] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.

- [23] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [24] Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International conference on machine learning*, pages 1530–1538. PMLR, 2015.
- [25] Graham Cormode and Shan Muthukrishnan. An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.
- [26] Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. In *International Colloquium on Automata, Languages, and Programming*, pages 693–703. Springer, 2002.
- [27] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. Elastic sketch: Adaptive and fast network-wide measurements. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 561–575, 2018.
- [28] Tim Kraska, Alex Beutel, Ed H Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *Proceedings of the 2018 international conference on management of data*, pages 489–504, 2018.
- [29] Chen-Yu Hsu, Piotr Indyk, Dina Katabi, and Ali Vakilian. Learning-based frequency estimation algorithms. In *International Conference on Learning Representations*, 2019.
- [30] Emmanuel J Candes. The restricted isometry property and its implications for compressed sensing. *Comptes rendus. Mathematique*, 346(9-10):589–592, 2008.
- [31] Amit Chakrabarti, Jeffrey Jiang, David P Woodruff, and Taisuke Yasuda. Streaming algorithms for ℓ_p flows and ℓ_p regression. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [32] Stanislaw J Szarek. Condition numbers of random matrices. *Journal of Complexity*, 7(2):131–149, 1991.
- [33] Emmanuel J Candes and Terence Tao. Near-optimal signal recovery from random projections: Universal encoding strategies? *IEEE transactions on information theory*, 52(12):5406–5425, 2006.
- [34] Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Jie Shi, Xiang Wang, Xiangnan He, and Tat-Seng Chua. Alphaedit: Null-space constrained knowledge editing for language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [35] Chao Dong, Chen Change Loy, Kaiming He, and Xiaoou Tang. Image super-resolution using deep convolutional networks. *IEEE transactions on pattern analysis and machine intelligence*, 38(2):295–307, 2015.
- [36] Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, et al. Photo-realistic single image super-resolution using a generative adversarial network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4681–4690, 2017.
- [37] Ashish Bora, Ajil Jalal, Eric Price, and Alexandros G Dimakis. Compressed sensing using generative models. In *International conference on machine learning*, pages 537–546. PMLR, 2017.
- [38] Lynton Ardizzone, Jakob Kruse, Carsten Rother, and Ullrich Köthe. Analyzing inverse problems with invertible neural networks. In *The Senventh International Conference on Learning Representations*, 2019.
- [39] Hyungjin Chung, Jeongsol Kim, Michael Thompson Mccann, Marc Louis Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *The Eleventh International Conference on Learning Representations*, 2023.
- [40] Yinhuai Wang, Jiwen Yu, and Jian Zhang. Zero-shot image restoration using denoising diffusion null-space model. In *The Eleventh International Conference on Learning Representations*, 2023.
- [41] Diana Cai, Michael Mitzenmacher, and Ryan P Adams. A bayesian nonparametric view on count-min sketch. *Advances in neural information processing systems*, 31, 2018.
- [42] David MW Powers. Applications and explanations of zipf’s law. In *New methods in language processing and computational natural language learning*, 1998.
- [43] Xinyu Yuan, Yan Qiao, Meng Li, Zhenchun Wei, Cuiying Feng, Zonghui Wang, and Wenzhi Chen. Learning-based sketches for frequency estimation in data streams without ground truth. *IEEE Transactions on Knowledge and Data Engineering*, 2026.
- [44] Allan Pinkus. Approximation theory of the mlp model in neural networks. *Acta numerica*, 8:143–195, 1999.
- [45] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein gan. *arXiv preprint arXiv:1701.07875*, 685, 2017.

- [46] Xinyu Yuan and Yan Qiao. Diffusion-ts: Interpretable diffusion for general time series generation. In *The Twelfth International Conference on Learning Representations*, 2024.
- [47] Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. *arXiv preprint arXiv:2511.13720*, 2025.
- [48] Xinyu Yuan, Yan Qiao, Pei Zhao, Rongyao Hu, and Benchu Zhang. Traffic matrix estimation based on denoising diffusion probabilistic model. In *2023 IEEE Symposium on Computers and Communications (ISCC)*, pages 316–322. IEEE, 2023.
- [49] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [50] Kaicheng Yang, Sheng Long, Qilong Shi, Yuanpeng Li, Zirui Liu, Yuhan Wu, Tong Yang, and Zhengyi Jia. Sketchint: Empowering int with towersketch for per-flow per-switch measurement. *IEEE Transactions on Parallel and Distributed Systems*, 34(11):2876–2894, 2023.
- [51] Liang Liang, Guang Yang, Ali Hadian, Luis Alberto Croquevielle, and Thomas Heinis. Swix: A memory-efficient sliding window learned index. *Proceedings of the ACM on Management of Data*, 2(1):1–26, 2024.
- [52] Linxi Li, Kun Xie, Shuyu Pei, Jigang Wen, Wei Liang, and Gaogang Xie. Cs-sketch: Compressive sensing enhanced sketch for full traffic measurement. *IEEE Transactions on Network Science and Engineering*, 11(3):2338–2352, 2023.
- [53] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.
- [54] Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation. *arXiv preprint arXiv:1410.8516*, 2014.
- [55] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real nvp. *arXiv preprint arXiv:1605.08803*, 2016.
- [56] Durk P Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. *Advances in neural information processing systems*, 31, 2018.
- [57] Lynton Ardizzone, Carsten Lüth, Jakob Kruse, Carsten Rother, and Ullrich Köthe. Guided image generation with conditional invertible neural networks. *arXiv preprint arXiv:1907.02392*, 2019.
- [58] Solomon Kullback. Kullback-leibler divergence. *Tech. Rep.*, 1951.
- [59] Arthur Gretton, Karsten M Borgwardt, Malte J Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *The journal of machine learning research*, 13(1):723–773, 2012.
- [60] Mingqing Xiao, Shuxin Zheng, Chang Liu, Yaolong Wang, Di He, Guolin Ke, Jiang Bian, Zhouchen Lin, and Tie-Yan Liu. Invertible image rescaling. In *European conference on computer vision*, pages 126–144. Springer, 2020.
- [61] Jingyun Liang, Andreas Lugmayr, Kai Zhang, Martin Danelljan, Luc Van Gool, and Radu Timofte. Hierarchical conditional flow: A unified framework for image super-resolution and image rescaling. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4076–4085, 2021.
- [62] Xiaojun Cheng, Xuyang Jing, Zheng Yan, Xian Li, Pu Wang, and Wei Wu. Alsketch: An adaptive learning-based sketch for accurate network measurement under dynamic traffic distribution. *Journal of Network and Computer Applications*, 216:103659, 2023.
- [63] Yehuda Vardi and D Lee. From image deblurring to optimal investments: Maximum likelihood solutions for positive linear inverse problems. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 55(3):569–598, 1993.
- [64] David Eppstein, Michael T Goodrich, Frank Uyeda, and George Varghese. What’s the difference? efficient set reconciliation without prior context. *ACM SIGCOMM Computer Communication Review*, 41(4):218–229, 2011.
- [65] Sisi Xiong, Yanjun Yao, Qing Cao, and Tian He. kbf: a bloom filter for key-value storage with an application on approximate state machines. In *IEEE INFOCOM 2014-IEEE Conference on Computer Communications*, pages 1150–1158. IEEE, 2014.
- [66] Elbert Du, Franklyn Wang, and Michael Mitzenmacher. Putting the “learning” into learning-augmented algorithms for frequency estimation. In *International Conference on Machine Learning*, pages 2860–2869. PMLR, 2021.
- [67] Caida. Caida anonymized internet traces 2018 dataset, 2018.

- [68] Claudio Lucchese, Salvatore Orlando, Raffaele Perego, and Fabrizio Silvestri. Webdocs: a real-life huge transactional dataset. In *FIMI*, volume 126, 2004.
- [69] Kenjiro Cho, Koushirou Mitsuya, and Akira Kato. Traffic data repository at the {WIDE} project. In *2000 USENIX Annual Technical Conference (USENIX ATC 00)*, 2000.
- [70] Bodon Ferenc. Anonymized click-stream data.
- [71] Tom Brijs, Gilbert Swinnen, Koen Vanhoof, and Geert Wets. Using association rules for product assortment decisions: A case study. In *Proceedings of the fifth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 254–260, 1999.
- [72] The technical report of our and other related algorithms.
- [73] Cristian Estan and George Varghese. New directions in traffic measurement and accounting: Focusing on the elephants, ignoring the mice. *ACM Transactions on Computer Systems (TOCS)*, 21(3):270–313, 2003.
- [74] Eigen Developers. Eigen::LeastSquaresConjugateGradient class reference, 2025.
- [75] David Chin-Lung Fong and Michael Saunders. Lsmr: An iterative algorithm for sparse least-squares problems. *SIAM Journal on Scientific Computing*, 33(5):2950–2971, 2011.
- [76] Graham Cormode, Minos Garofalakis, Peter J Haas, Chris Jermaine, et al. Synopses for massive data: Samples, histograms, wavelets, sketches. *Foundations and Trends® in Databases*, 4(1–3):1–294, 2011.
- [77] Yagyensh Chandra Pati, Ramin Rezaiifar, and Perinkulam Sambamurthy Krishnaprasad. Orthogonal matching pursuit: Recursive function approximation with applications to wavelet decomposition. In *Proceedings of 27th Asilomar conference on signals, systems and computers*, pages 40–44. IEEE, 1993.
- [78] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [79] Abhishek Kumar, Minh Sung, Jun Xu, and Jia Wang. Data streaming algorithms for efficient and accurate estimation of flow size distribution. *ACM SIGMETRICS Performance Evaluation Review*, 32(1):177–188, 2004.
- [80] NVIDIA, Péter Vingelmann, and Frank H.P. Fitzek. Cuda, release: 10.2.89, 2020.
- [81] Morteza Mardani and Georgios B Giannakis. Estimating traffic and anomaly maps via network tomography. *IEEE/ACM transactions on networking*, 24(3):1533–1547, 2015.
- [82] Christopher C Paige and Michael A Saunders. Lsq: An algorithm for sparse linear equations and sparse least squares. *ACM Transactions on Mathematical Software (TOMS)*, 8(1):43–71, 1982.
- [83] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996.
- [84] Younes Ben Mazziane and Othmane Marfoq. Count-min sketch with conservative updates: Worst-case analysis. *arXiv preprint arXiv:2405.12034*, 2024.
- [85] Tong Yang, Yang Zhou, Hao Jin, Shigang Chen, and Xiaoming Li. Pyramid sketch: A sketch framework for frequency estimation of data streams. *Proceedings of the VLDB Endowment*, 10(11):1442–1453, 2017.
- [86] Tong Yang, Junzhi Gong, Haowei Zhang, Lei Zou, Lei Shi, and Xiaoming Li. Heavyguardian: Separate and guard hot items in data streams. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2584–2593, 2018.
- [87] Tong Yang, Haowei Zhang, Jinyang Li, Junzhi Gong, Steve Uhlig, Shigang Chen, and Xiaoming Li. Heavykeeper: an accurate algorithm for finding top-*k* elephant flows. *IEEE/ACM Transactions on Networking*, 27(5):1845–1858, 2019.
- [88] Bohan Zhao, Xiang Li, Boyu Tian, Zhiyu Mei, and Wenfei Wu. Dhs: Adaptive memory layout organization of sketch slots for fast and accurate data stream processing. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 2285–2293, 2021.
- [89] Zheng Zhong, Shen Yan, Zikun Li, Decheng Tan, Tong Yang, and Bin Cui. Burstsketch: Finding bursts in data streams. In *Proceedings of the 2021 International Conference on Management of Data*, pages 2375–2383, 2021.
- [90] Minlan Yu, Lavanya Jose, and Rui Miao. Software {Defined}{Traffic} measurement with {OpenSketch}. In *10th USENIX symposium on networked systems design and implementation (NSDI 13)*, pages 29–42, 2013.
- [91] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. Efficient computation of frequent and top-*k* elements in data streams. In *International conference on database theory*, pages 398–412. Springer, 2005.
- [92] Jizhou Li, Zikun Li, Yifei Xu, Shiqi Jiang, Tong Yang, Bin Cui, Yafei Dai, and Gong Zhang. Wavingsketch: An unbiased and generic sketch for finding top-*k* items in data streams. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1574–1584, 2020.

- [93] Lu Tang, Qun Huang, and Patrick PC Lee. Spreadsketch: Toward invertible and network-wide detection of superspreaders. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, pages 1608–1617. IEEE, 2020.
- [94] Tong Yang, Lun Wang, Yulong Shen, Muhammad Shahzad, Qun Huang, Xiaohong Jiang, Kun Tan, and Xiaoming Li. Empowering sketches with machine learning for network measurements. In *Proceedings of the 2018 Workshop on Network Meets AI & ML*, pages 15–20, 2018.
- [95] Meifan Zhang, Hongzhi Wang, Jianzhong Li, and Hong Gao. Learned sketches for frequency estimation. *Information Sciences*, 507:365–385, 2020.
- [96] Tanqiu Jiang, Yi Li, Honghao Lin, Yisong Ruan, and David P Woodruff. Learning-augmented data stream algorithms. *ICLR*, 2020.
- [97] Yangsheng Yan, Fuliang Li, Wei Wang, and Xingwei Wang. Talentsketch: Lstm-based sketch for adaptive and high-precision network measurement. In *2022 IEEE 30th International Conference on Network Protocols (ICNP)*, pages 1–12. IEEE, 2022.
- [98] Yuanpeng Li, Zhen Xu, Zongwei Lv, Yannan Hu, Yong Cui, and Tong Yang. Llm-sketch: Enhancing network sketches with llm. *arXiv preprint arXiv:2502.07495*, 2025.
- [99] Yuan Feng, Yukun Cao, Wang Hairu, Xike Xie, and S Kevin Zhou. Mayfly: a neural data structure for graph stream summarization. In *The twelfth international conference on learning representations*, 2023.
- [100] Yuan Feng, Yukun Cao, Hairu Wang, Xike Xie, and S Kevin Zhou. Lego sketch: A scalable memory-augmented neural network for sketching data streams. In *Forty-second International Conference on Machine Learning*, 2025.
- [101] Mengbo Wang, Zhuochen Fan, Dayu Wang, Guorui Xie, Qing Li, Zeyu Luan, Yong Jiang, Tong Yang, and Mingwei Xu. Ratiosketch: Towards more accurate frequency estimation in data streams via a lightweight neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [102] Yehuda Vardi. Network tomography: Estimating source-destination traffic intensities from link data. *Journal of the American statistical association*, 91(433):365–377, 1996.
- [103] Yin Zhang, Matthew Roughan, Nick Duffield, and Albert Greenberg. Fast accurate computation of large-scale ip traffic matrices from link loads. *ACM SIGMETRICS Performance Evaluation Review*, 31(1):206–217, 2003.
- [104] Yin Zhang, Matthew Roughan, Carsten Lund, and David Donoho. An information-theoretic approach to traffic matrix estimation. In *Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications*, pages 301–312, 2003.
- [105] Yin Zhang, Matthew Roughan, Walter Willinger, and Lili Qiu. Spatio-temporal compressive sensing and internet traffic matrices. In *Proceedings of the ACM SIGCOMM 2009 conference on Data communication*, pages 267–278, 2009.
- [106] Rhys Alistair Bowden, Matthew Roughan, and Nigel Bean. Network link tomography and compressive sensing. In *Proceedings of the ACM SIGMETRICS joint international conference on Measurement and modeling of computer systems*, pages 159–160, 2011.
- [107] Yi-Chao Chen, Lili Qiu, Yin Zhang, Guangtao Xue, and Zhenxian Hu. Robust network compressive sensing. In *Proceedings of the 20th annual international conference on Mobile computing and networking*, pages 545–556, 2014.
- [108] Hiroki Ikeuchi and Hiroshi Saito. Network tomography based on adaptive measurements in probabilistic environments. *IEEE Transactions on Information Theory*, 70(11):8251–8265, 2024.
- [109] Xinyu Yuan, Yan Qiao, Zhenchun Wei, Zeyu Zhang, Minyue Li, Pei Zhao, Rongyao Hu, and Wenjing Li. Diffusion models meet network management: Improving traffic matrix analysis with diffusion-based approach. *IEEE Transactions on Network and Service Management*, 2025.
- [110] Gene Moo Lee, Huiya Liu, Young Yoon, and Yin Zhang. Improving sketch reconstruction accuracy using linear least squares method. In *Proceedings of the 5th ACM SIGCOMM conference on Internet Measurement*, pages 24–24, 2005.
- [111] Yi Lu, Andrea Montanari, Balaji Prabhakar, Sarang Dharmapurikar, and Abdul Kabbani. Counter braids: A novel counter architecture for per-flow measurement. *ACM SIGMETRICS Performance Evaluation Review*, 36(1):121–132, 2008.
- [112] Yongquan Fu, Dongsheng Li, Siqi Shen, Yiming Zhang, and Kai Chen. Clustering-preserving network flow sketching. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, pages 1309–1318. IEEE, 2020.

- [113] Yongqiang Liu and Xike Xie. Xy-sketch: On sketching data streams at web scale. In *Proceedings of the Web Conference 2021*, pages 1169–1180, 2021.
- [114] Jintao He, Jiaqi Zhu, and Qun Huang. Histsketch: A compact data structure for accurate per-key distribution monitoring. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, pages 2008–2021. IEEE, 2023.
- [115] Chuanhao Sun, Kai Xu, Gianni Antichi, and Mahesh K Marina. Netgsr: Towards efficient and reliable network monitoring with generative super resolution. *Proceedings of the ACM on Networking*, 2(CoNEXT4):1–27, 2024.
- [116] Xizheng Wang, Libin Liu, Li Chen, Dan Li, Yukai Miao, and Yu Bai. Resolving packets from counters: Enabling multi-scale network traffic super resolution via composable large traffic model. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 1541–1561, 2025.
- [117] Lu Tang, Qun Huang, and Patrick PC Lee. Mv-sketch: A fast and compact invertible sketch for heavy flow detection in network data streams. In *IEEE INFOCOM 2019-IEEE Conference on Computer Communications*, pages 2026–2034. IEEE, 2019.
- [118] Christos Douligeris and Aikaterini Mitrokotsa. Ddos attacks and defense mechanisms: classification and state-of-the-art. *Computer networks*, 44(5):643–666, 2004.
- [119] Haohui Mai, Ahmed Khurshid, Rachit Agarwal, Matthew Caesar, P Brighten Godfrey, and Samuel Talmadge King. Debugging the data plane with anteater. *ACM SIGCOMM Computer Communication Review*, 41(4):290–301, 2011.
- [120] Chuanxiong Guo, Lihua Yuan, Dong Xiang, Yingnong Dang, Ray Huang, Dave Maltz, Zhaoyi Liu, Vin Wang, Bin Pang, Hua Chen, et al. Pingmesh: A large-scale system for data center network latency measurement and analysis. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pages 139–152, 2015.
- [121] Noga Alon, Yossi Matias, and Mario Szegedy. The space complexity of approximating the frequency moments. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 20–29, 1996.
- [122] Andrew McGregor. Graph stream algorithms: a survey. *ACM SIGMOD Record*, 43(1):9–20, 2014.
- [123] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- [124] Brian DO Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.

Appendix

A Background

A.1 Variational Autoencoder

The idea behind a Variational Autoencoder (VAE) [21] is, in fact, less closely related to conventional autoencoders and more deeply rooted in variational Bayesian inference and probabilistic graphical models. In VAEs, the conditional probability distribution $p_\theta(\mathbf{x} | \mathbf{z})$ defines the generative process, playing a role analogous to the decoder function $f_\theta(\mathbf{x} | \mathbf{z})$ introduced previously. Accordingly, $p_\theta(\mathbf{x} | \mathbf{z})$ is often referred to as the probabilistic decoder. Similarly, the approximation function $q_\phi(\mathbf{z} | \mathbf{x})$ serves as the probabilistic encoder, analogous to the encoder function $g_\phi(\mathbf{z} | \mathbf{x})$.

The objective is to make the approximate posterior distribution $q_\phi(\mathbf{z} | \mathbf{x})$ as close as possible to the true posterior distribution $p_\theta(\mathbf{z} | \mathbf{x})$. This discrepancy can be quantified using the Kullback–Leibler Divergence (KL divergence). Expanding the KL divergence yields:

$$\begin{aligned}
 \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) | p_\theta(\mathbf{z} | \mathbf{x})) &= \int q_\phi(\mathbf{z} | \mathbf{x}) \log \frac{q_\phi(\mathbf{z} | \mathbf{x})}{p_\theta(\mathbf{z} | \mathbf{x})} d\mathbf{z} = \int q_\phi(\mathbf{z} | \mathbf{x}) \log \frac{q_\phi(\mathbf{z} | \mathbf{x}) p_\theta(\mathbf{x})}{p_\theta(\mathbf{z}, \mathbf{x})} d\mathbf{z} \\
 &= \int q_\phi(\mathbf{z} | \mathbf{x}) \left(\log p_\theta(\mathbf{x}) + \log \frac{q_\phi(\mathbf{z} | \mathbf{x})}{p_\theta(\mathbf{z}, \mathbf{x})} \right) d\mathbf{z} \\
 &= \log p_\theta(\mathbf{x}) + \int q_\phi(\mathbf{z} | \mathbf{x}) \log \frac{q_\phi(\mathbf{z} | \mathbf{x})}{p_\theta(\mathbf{z}, \mathbf{x})} d\mathbf{z} \\
 &= \log p_\theta(\mathbf{x}) + \int q_\phi(\mathbf{z} | \mathbf{x}) \log \frac{q_\phi(\mathbf{z} | \mathbf{x})}{p_\theta(\mathbf{x} | \mathbf{z}) p_\theta(\mathbf{z})} d\mathbf{z} \\
 &= \log p_\theta(\mathbf{x}) + \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z} | \mathbf{x})} \left[\log \frac{q_\phi(\mathbf{z} | \mathbf{x})}{p_\theta(\mathbf{z})} - \log p_\theta(\mathbf{x} | \mathbf{z}) \right] \\
 &= \log p_\theta(\mathbf{x}) + \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) | p_\theta(\mathbf{z})) - \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z} | \mathbf{x})} \log p_\theta(\mathbf{x} | \mathbf{z})
 \end{aligned} \tag{11}$$

Rearranging the terms gives:

$$\log p_\theta(\mathbf{x}) - \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) | p_\theta(\mathbf{z})) = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z} | \mathbf{x})} \log p_\theta(\mathbf{x} | \mathbf{z}) - \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) | p_\theta(\mathbf{z})).$$

The negative of this expression defines the VAE objective:

$$\mathcal{L}_{\text{VAE}}(\theta, \phi) = -\log p_\theta(\mathbf{x}) + \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) | p_\theta(\mathbf{z} | \mathbf{x})) = -\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z} | \mathbf{x})} \log p_\theta(\mathbf{x} | \mathbf{z}) - \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) | p_\theta(\mathbf{z})).$$

In variational Bayesian inference, this objective is commonly known as the evidence lower bound (ELBO), or variational lower bound. The term “lower bound” arises because KL divergence is always non-negative and thus

$$-\mathcal{L}_{\text{VAE}} = \log p_\theta(\mathbf{x}) - \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) | p_\theta(\mathbf{z} | \mathbf{x})) \leq \log p_\theta(\mathbf{x})$$

Consequently, minimizing the VAE loss is equivalent to maximizing a lower bound on the log-likelihood of the observed data, thereby improving the model’s ability to generate realistic samples.

A.2 Generative Adversarial Network

Generative Adversarial Network (GAN) [22] consists of two models: (i) A discriminator D , which estimates the probability that a given sample originates from the real dataset. Acting as a critic, it is optimized to distinguish real samples from generated ones. (ii) A generator G , which produces synthetic samples from a noise variable z (where z introduces output diversity). The generator is trained to approximate the real data distribution such that its generated samples appear increasingly realistic, ultimately encouraging the discriminator to assign them high probabilities. During training, these two models engage in an adversarial process: the generator G continuously learns to fool the discriminator, while the discriminator D simultaneously improves its ability to detect generated samples. This game drives both models to progressively enhance their capabilities.

Define p_z as the distribution of the noise input z , p_g as the generator-induced distribution over data samples x , and p_r as the distribution of real data samples. On the one hand, the discriminator D is trained to correctly classify real samples by maximizing $\mathbb{E} x \sim p_r(x) [\log D(x)]$. At the same time, for a generated sample $G(z)$ with $z \sim p_z(z)$, the discriminator is encouraged to assign a probability close to zero to the fake sample by maximizing $\mathbb{E} z \sim p_z(z) [\log(1 - D(G(z)))]$.

On the other hand, the generator G aims to produce realistic samples that can fool the discriminator. Therefore, it is trained to minimize $\mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$, thereby encouraging $D(G(z))$ to be as large as possible.

Combining these two objectives, the discriminator and generator engage in a minimax game, formulated through the following objective function:

$$\begin{aligned} \min_G \max_D \mathcal{L}(D, G) &= \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \\ &= \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{x \sim p_g(x)} [\log(1 - D(x))] \end{aligned} \quad (12)$$

Wasserstein Distance can be informally interpreted as the minimum cost required to transform one probability distribution into another, analogous to moving a pile of dirt from one shape to another. The transportation cost is defined as the amount of dirt being moved multiplied by the distance it travels. Unlike divergence measures such as KL divergence or Jensen–Shannon divergence, the Wasserstein metric provides a smoother and more meaningful measure of discrepancy between distributions, which significantly improves the stability of gradient-based optimization during training [45].

Compared with the original Generative Adversarial Network (GAN), Wasserstein Generative Adversarial Network (WGAN) introduces two major modifications: (i) After each gradient update of the critic network, the model weights are clipped to lie within a small fixed range in order to enforce the Lipschitz continuity constraint. (ii) The original GAN loss is replaced with a new objective derived from the Wasserstein distance, eliminating the logarithmic terms used in the standard adversarial loss.

Under this formulation, the discriminator no longer serves as a binary classifier that directly distinguishes real samples from generated ones. Instead, it acts as a critic that estimates the Wasserstein distance between the real and generated data distributions.

A.3 Flow-based Generative Model

The key idea behind flow-based generative models is to transform a simple base distribution into a highly complex target distribution through a sequence of invertible mappings. As samples pass through these transformations, their probability densities are updated iteratively according to the change-of-variables theorem. In this manner, Normalizing Flows (NFs) [24] preserve exact likelihood computation while greatly enhancing the expressive power of the modeled distribution. Here we define

$$(i) \mathbf{z}_{i-1} \sim p_{i-1}(\mathbf{z}_{i-1}) \quad (ii) \mathbf{z}_i = T_i(\mathbf{z}_{i-1}), \text{ thus } \mathbf{z}_{i-1} = T_i^{-1}(\mathbf{z}_i) \quad (iii) p_i(\mathbf{z}_i) = p_{i-1}(T_i^{-1}(\mathbf{z}_i)) \left| \det \frac{dT_i^{-1}}{d\mathbf{z}_i} \right|$$

Then, we rewrite the equation in terms of $\mathbf{z}_i$ so that inference can be performed directly in the base distribution space.

$$p_i(\mathbf{z}_i) = p_{i-1}(T_i^{-1}(\mathbf{z}_i)) \left| \det \frac{dT_i^{-1}}{d\mathbf{z}_i} \right| = p_{i-1}(\mathbf{z}_{i-1}) \left| \det \left(\frac{dT_i}{d\mathbf{z}_{i-1}} \right)^{-1} \right| = p_{i-1}(\mathbf{z}_{i-1}) \left| \det \frac{dT_i}{d\mathbf{z}_{i-1}} \right|^{-1}$$

Given this chain of probability density functions, the relationship between every pair of consecutive variables is explicitly defined. Therefore, we can recursively expand the density of the output variable $\mathbf{x}$, tracing it step by step back to the initial latent distribution $\mathbf{z}_0$.

$$\begin{aligned} \mathbf{x} &= \mathbf{z}_K = T_K \circ T_{K-1} \circ \dots \circ f_1(\mathbf{z}_0) \\ \log p(\mathbf{x}) &= \log \pi_K(\mathbf{z}_K) = \log \pi_{K-1}(\mathbf{z}_{K-1}) - \log \left| \det \frac{dT_K}{d\mathbf{z}_{K-1}} \right| \\ &= \log \pi_{K-2}(\mathbf{z}_{K-2}) - \log \left| \det \frac{dT_{K-1}}{d\mathbf{z}_{K-2}} \right| - \log \left| \det \frac{dT_K}{d\mathbf{z}_{K-1}} \right| \\ &= \dots = \log \pi_0(\mathbf{z}_0) - \sum_{i=1}^K \log \left| \det \frac{dT_i}{d\mathbf{z}_{i-1}} \right| \end{aligned} \quad (13)$$

The trajectory defined by the sequence of random variables $\mathbf{z}_i = T_i(\mathbf{z}_{i-1})$ is referred to as the flow, while the entire sequence of intermediate distributions π_i constitutes a normalizing flow. To make the density transformation computationally feasible, each transformation function T_i must satisfy two key properties: (i) it must be easily invertible, and (ii) the determinant of its Jacobian must be efficient to compute.

With normalizing flows, the exact log-likelihood of an observed sample $\log p(\mathbf{x})$ becomes tractable. Consequently, flow-based generative models can be trained directly through maximum likelihood estimation. Given a training dataset $\mathcal{D}$, the objective function is defined as the negative log-likelihood: $\mathcal{L}(\mathcal{D}) = -\frac{1}{\|\mathcal{D}\|} \sum_{\mathbf{x} \in \mathcal{D}} \log p(\mathbf{x})$.

A.4 Diffusion Model

Diffusion model is a class of deep generative models that parameterize the encoding (forward) and decoding (reverse) processes via a diffusion process. The key idea behind them is to add small amounts of noise gradually to a data point for transitions from data manifold to noisy manifolds, then train the underlying neural network to transport from the pure Gaussian to the clean area through inverting the diffusion process.

Suppose $p_0(\mathbf{x}_0)$ be the d -dimensional data distribution. We consider the general class of score-based diffusion models in a continuous form that can be described with the following Ornstein-Uhlenbeck stochastic differential equation (SDE) [123]

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t) dt + g(t) d\mathbf{w} \quad (14)$$

where $\mathbf{w}$ denotes the standard Brownian motion, $\mathbf{f} : \mathbb{R}^d \times \mathbb{R} \rightarrow \mathbb{R}^d$, $g : \mathbb{R} \rightarrow \mathbb{R}$ is the linear drift function and scalar diffusion coefficient, respectively. The SDE results in a series of marginal distributions $\{p_t(\mathbf{x}_t)\}$ where $t \in [0, T]$, so that $\mathbf{x}_T \sim \mathcal{N}(0, \sigma_T^2)$ with some constants $\sigma_T > 0$.

It is known that the density evolution process can be reversed by another SDE with the Anderson's theorem [124]

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t) \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)] dt + g(t) d\bar{\mathbf{w}} \quad (15)$$

where $\bar{\mathbf{w}}$ is a Brownian motion running backward in time from T to 0 . Then diffusion models approximate this reverse process by learning the score function $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)$ through a score-matching loss:

$$\mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_t | \mathbf{x}_0} \left[\lambda_t \|s_\theta(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)\|_2^2 \right] \quad (16)$$

where λ_t is a positive weight coefficient, and t is uniformly sampled over $[0, T]$. With a learned score function $s_\theta(\mathbf{x}_t, t)$, one can then solve the reverse SDE through

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t) s_\theta(\mathbf{x}_t, t)] dt + g(t) d\bar{\mathbf{w}} \quad (17)$$

by first sampling from the prior Gaussian $p_T(\mathbf{x}_T)$.

In this work, we focus on Denoising Diffusion Probabilistic Models [23] which is equivalent to the variance preserving form of the SDE (VP-SDE). To be more specific, we have the forward and reverse SDEs as the continuous version of the diffusion process in DDPM with the choice of $\mathbf{f}(\mathbf{x}, t) = -\beta(t) \mathbf{x}/2$ and $g(t) = \sqrt{\beta(t)}$. The corresponding forward SDE can be formulated as the following:

$$d\mathbf{x} = -\frac{\beta(t)}{2} \mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}. \quad (18)$$

In particular, the forward process of DDPMs gradually corrupts original data $\mathbf{x}_0 \in \mathbb{R}^d$ via a fixed Markov chain $\mathbf{x}_0, \dots, \mathbf{x}_T$ with each variable in $\mathbb{R}^d$ as follows:

$$\begin{cases} \mathbf{x}_t | \mathbf{x}_{t-1} \sim \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I}), \\ \mathbf{x}_t | \mathbf{x}_0 \sim \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \\ \mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \end{cases} \quad (19)$$

with $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, $\alpha_t := 1 - \beta_t$ where $\beta_t \in (0, 1)$ is a variance at diffusion step t , scaling factor $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$.

Starting from the Gaussian noise $\mathbf{x}_T$, we can run the reverse process parametrized by the model $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \mu(\mathbf{x}_t, t, \theta), \Sigma(\mathbf{x}_t, t, \theta))$ to get $\mathbf{x}_0$. The diffusion model is trained to maximize the marginal likelihood of the data $\mathbb{E}_{\mathbf{x}_0} [\log p_\theta(\mathbf{x}_0)]$, and we can write the variational lower bound with KLD as follows:

$$\mathcal{L}_{vlb} := \underbrace{-\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)}_{\mathcal{L}_0} + \underbrace{\mathcal{D}_{KL}(p(\mathbf{x}_T | \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{\mathcal{L}_T} + \sum_{t=2}^T \underbrace{\mathcal{D}_{KL}(p(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{\mathcal{L}_{t-1}} \quad (20)$$

The main objective is a sum of independent terms $\mathcal{L}_{t-1}$. There are many different ways to parameterize the posterior mean $\mu(\mathbf{x}_t, t, \theta)$, and the most obvious option is to predict $\mu(\mathbf{x}_t, t, \theta)$ directly:

$$\mathcal{L}_o := \mathbb{E}_{t, \mathbf{x}_0} \left[\frac{1}{2\Sigma^2(\mathbf{x}_t, t, \theta)} \|\hat{\mu}(\mathbf{x}_t, \mathbf{x}_0) - \mu(\mathbf{x}_t, t, \theta)\|^2 \right], \quad (21)$$

where $\Sigma(\mathbf{x}_t, t, \theta)$ is often set to pre-defined time dependent constants, and $\hat{\mu}(\mathbf{x}_t, \mathbf{x}_0)$ is the mean of the posterior $p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0)$ which are defined as follows:

$$\hat{\mu}(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t. \quad (22)$$

Alternatively, the network could also predict $\mathbf{x}_0$ or ϵ using (19) and (22). In this work, we train our model to reconstruct input $\mathbf{x}_0$ itself combined with a reweighted loss function:

$$\mathcal{L}_{simple} = \mathbb{E}_{t, \epsilon, \mathbf{x}_0} \left[\|\mathbf{x}_0 - \hat{\mathbf{x}}_0(\mathbf{x}_t, t, \theta)\|^2 \right]. \quad (23)$$

The connection between the score function and the prediction in DDPMs can be formulated approximately as: $s_\theta(\mathbf{x}_t, t) \approx (\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \hat{\mathbf{x}}_0(\mathbf{x}_t, t, \theta)) / (1 - \bar{\alpha}_t)$ [109]. Data generation through denoising depends on the score function and can be seen as noise conditional score-based generation. Then the reverse process $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ in DDPMs can be written as follows:

$$\mathbf{x}_{t-1} = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \hat{\mathbf{x}}_0(\mathbf{x}_t, t, \theta) + \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t \mathbf{z}_t. \quad (24)$$

B Useful Lemmas

Lemma 1. Let $\mathbf{u}, \mathbf{v} \in \mathbb{C}^N$ be vectors such that $\|\mathbf{u}\|_0 \leq s$ and $\|\mathbf{v}\|_0 \leq t$. Assume that their supports are disjoint, i.e., $\text{supp}(\mathbf{u}) \cap \text{supp}(\mathbf{v}) = \emptyset$. Then $|\langle \mathbf{A}\mathbf{u}, \mathbf{A}\mathbf{v} \rangle| \leq \delta_{s+t} \|\mathbf{u}\|_2 \|\mathbf{v}\|_2$, where δ_{s+t} is the isometry constant of matrix $\mathbf{A}$.

Proof. Let $S := \text{supp}(\mathbf{u}) \cup \text{supp}(\mathbf{v})$, so that $|S| \leq s + t$. Denote by $\mathbf{u}_S, \mathbf{v}_S \in \mathbb{C}^{|S|}$ the restrictions of $\mathbf{u}$ and $\mathbf{v}$ to the index set S , and by $\mathbf{A}_S$ the submatrix of $\mathbf{A}$ consisting of the columns indexed by S . Then $\mathbf{A}\mathbf{u} = \mathbf{A}_S \mathbf{u}_S$ and $\mathbf{A}\mathbf{v} = \mathbf{A}_S \mathbf{v}_S$. Since $\text{supp}(\mathbf{u}) \cap \text{supp}(\mathbf{v}) = \emptyset$, we have $\langle \mathbf{u}_S, \mathbf{v}_S \rangle = 0$. Therefore,

$$|\langle \mathbf{A}\mathbf{u}, \mathbf{A}\mathbf{v} \rangle| = |\langle \mathbf{A}_S \mathbf{u}_S, \mathbf{A}_S \mathbf{v}_S \rangle - \langle \mathbf{u}_S, \mathbf{v}_S \rangle| = |\langle (\mathbf{A}_S^* \mathbf{A}_S - \mathbf{I}) \mathbf{u}_S, \mathbf{v}_S \rangle|.$$

Applying the Cauchy–Schwarz inequality yields

$$|\langle (\mathbf{A}_S^* \mathbf{A}_S - \mathbf{I}) \mathbf{u}_S, \mathbf{v}_S \rangle| \leq \|\mathbf{A}_S^* \mathbf{A}_S - \mathbf{I}\|_{2 \rightarrow 2} \|\mathbf{u}_S\|_2 \|\mathbf{v}_S\|_2.$$

By the RIP of order $s + t$, i.e., Definition 4, we have $\|\mathbf{A}_S^* \mathbf{A}_S - \mathbf{I}\|_{2 \rightarrow 2} \leq \delta_{s+t}$. Finally, noting that $\|\mathbf{u}_S\|_2 = \|\mathbf{u}\|_2$ and $\|\mathbf{v}_S\|_2 = \|\mathbf{v}\|_2$, we obtain $|\langle \mathbf{A}\mathbf{u}, \mathbf{A}\mathbf{v} \rangle| \leq \delta_{s+t} \|\mathbf{u}\|_2 \|\mathbf{v}\|_2$, which completes the proof. $\square$

Lemma 2. For any $x \in (0, 1)$, we have that $\sqrt{x+1} < 1 + (\sqrt{2} + 1)x$.

Proof. Let $g(x) = (\sqrt{2} + 1)x - \sqrt{x+1} + 1$. Then, $g'(x) = \sqrt{2} + 1 - \frac{1}{2\sqrt{x+1}}$. Also, $g(0) = 0$, $g(1) = 2$, and $g'(x) > 0$ for any $x \in (0, 1)$. Therefore, $\sqrt{x+1} < 1 + (\sqrt{2} + 1)x, \forall x \in (0, 1)$. $\square$

Lemma 3. Let $\mathbf{x} \in \mathbb{R}^N$ be an s -sparse vector, and $\mathbf{A} \in \mathbb{R}^{m \times N}$ be a random dense matrix drawn from the Gaussian matrix ensemble. Let $\varepsilon \in (0, 1)$. If

$$\frac{m^2}{m+1} \geq 2s \left(\sqrt{\ln(2.34N/s)} + \sqrt{\frac{\ln(\varepsilon^{-1})}{s}} \right)^2 \quad (25)$$

then with probability at least $1 - \varepsilon$, $\mathbf{x}$ is the unique minimizer of $\|\mathbf{z}\|_1$ subject to $\mathbf{A}\mathbf{x} = \mathbf{A}\mathbf{z}$.

Proof. This lemma is a direct application of Theorem 9.16 from page 258 of [14]. $\square$

Lemma 4. Let $\mathcal{F}$ be a unit ball in a universal reproducing kernel Hilbert space $\mathcal{H}$, defined on the compact metric space $\mathcal{X}$, with associated continuous kernel $k(\cdot, \cdot)$. Then $\text{MMD}[\mathcal{F}, p, q] = 0$ if and only if $p = q$.

Proof. The original proof can be found in [59] (see its Theorem 5). $\square$

C Visualizations

C.1 Learned Generative Flow

We would like to clarify that the role of z in our framework is not to encode semantically disentangled features, but rather to serve as an auxiliary latent variable that completes the missing information in f conditioned on the sketch b .

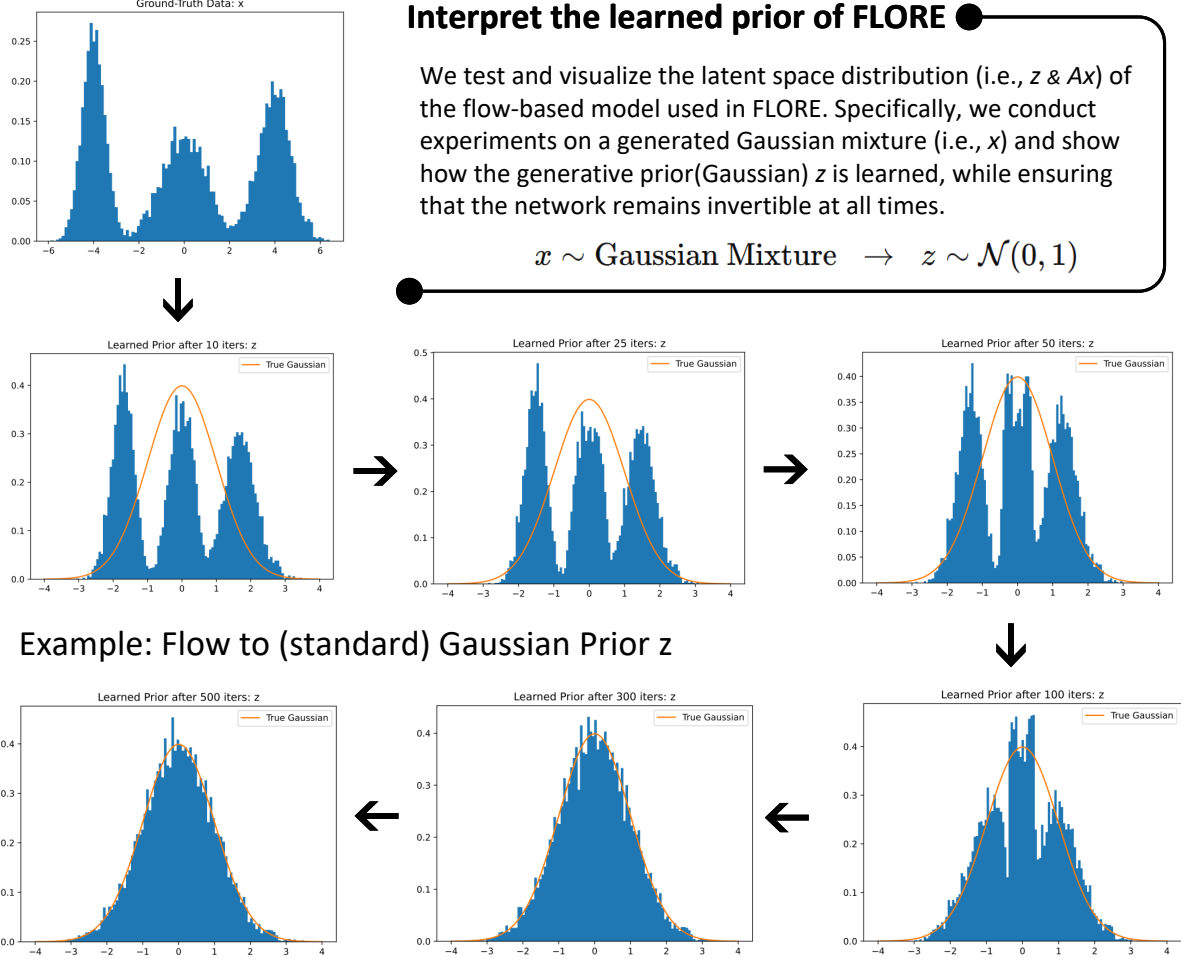


Figure 25: Visualizations of the learned flow-based mapping during training.

As such, our objective is aligned with faithful posterior modeling, rather than interpretable or controllable factorization of latent space. Also note that, unlike VAEs, flow-based models do not enforce clear semantic disentanglement on z . Nevertheless, we still conduct a toy experiment to better characterize the flow between latent z and original f , by following [this URL](#). Specifically, we visualize how the distribution of z from f evolves during training on a synthetic Gaussian mixture dataset, as shown in Figure 25. The figure demonstrates that the flow successfully models the generative prior with a meaningful mapping from GT to z .

C.2 Recovery (distributional) Results

To gain insights into the behaviors of sketches, we next visualize and compare the recovery results from each sketch viewpoint (i.e., FLORE, CM, CS, NZE, LCM, and LCS), which can also be regarded as a reflection of how well the distribution is perceived by them. The resulting visualizations under different distributions and space budgets are shown from Figure 26 to Figure 28, where key-wise frequencies are aggregated by their nearest 100 elements. Similar to the visualizations in the previous section, here the more the shaded areas overlap and the smaller the area, the better the results. First of all, the output of all sketches becomes more realistic as the memory budget increases. We also observe that all non-CS methods significantly overestimate the true frequencies of most elements because they lack a global perspective. Meanwhile, due to the need to satisfy both consistency and sparsity constraints, the total area of NZE using compressive sensing is close to that of the true distribution; however, it often compensates for high-frequency items by underestimating the majority of values or otherwise. In comparison, FLORE consistently achieves the highest fidelity in both curve shape and overall pattern under identical settings. Although FLORE partially overestimates the true values of some elements under the guidance of the CM enhanced by EM refinements, it requires only 128KB to become nearly indistinguishable from the ground-truth pattern, achieving an almost perfect fit. This aligns with our quantitative

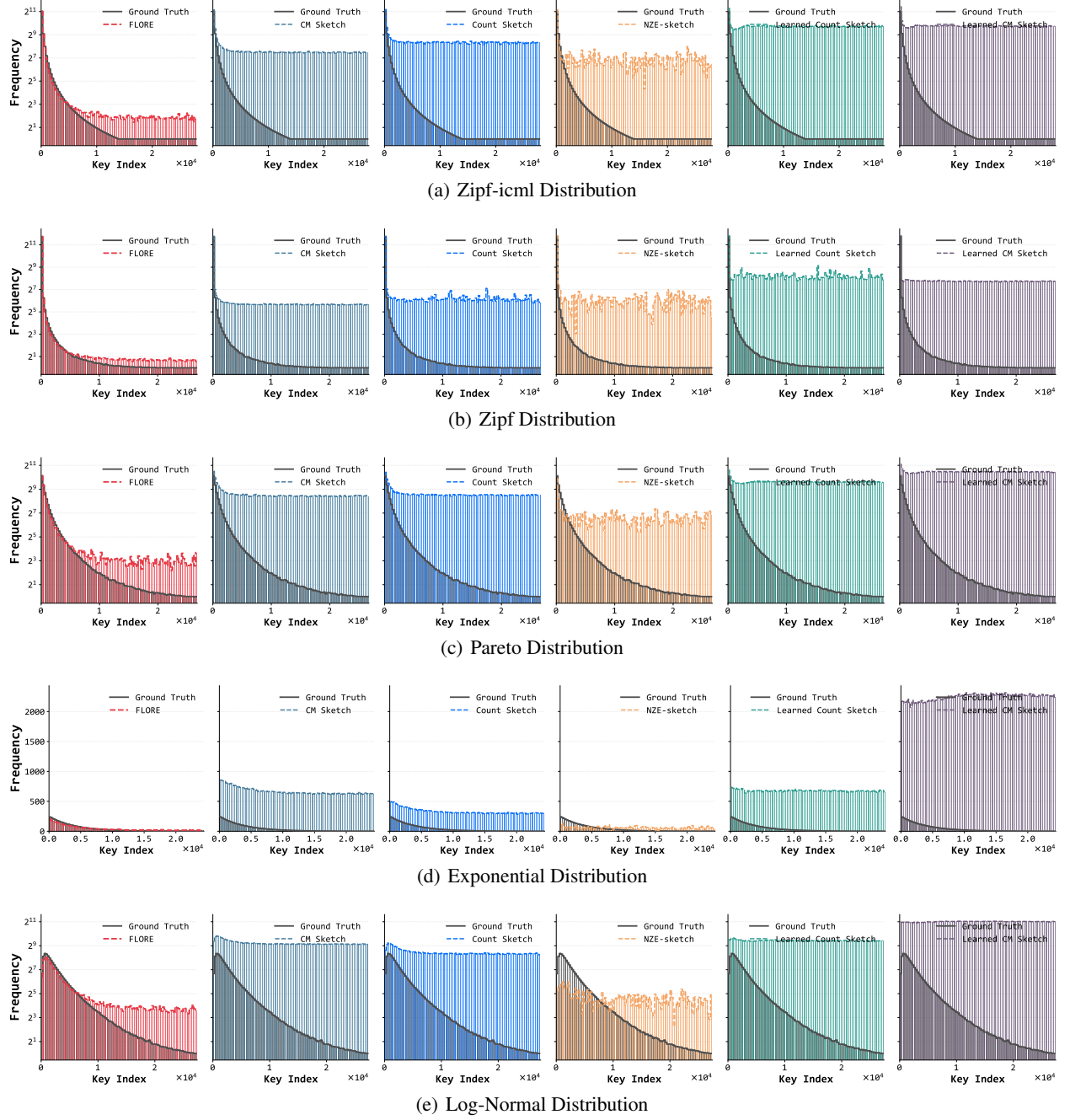


Figure 26: Visualizations of sketch recovery under a fixed 32KB memory budget. From left to right, we present the recovery results of FLORE, CM, CS, NZE, LCM, and LCS, compared against the ground-truth sample distribution. To enhance readability, every 100 consecutive keys are aggregated in the x-axis.

distribution estimation results (WMRE) and can be attributed to the powerful distribution approximation capability of the generative modeling paradigm.

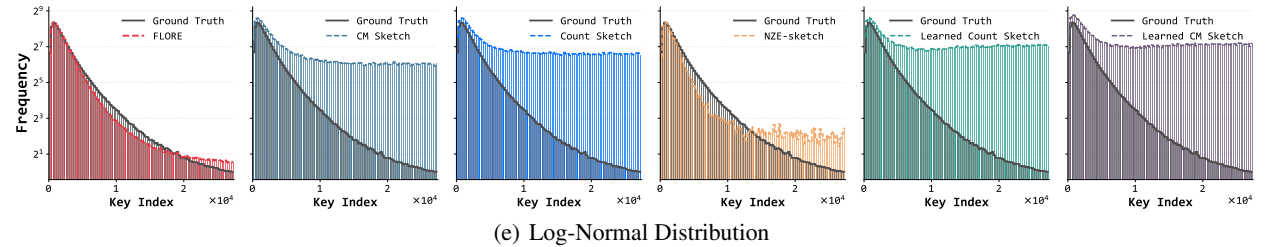
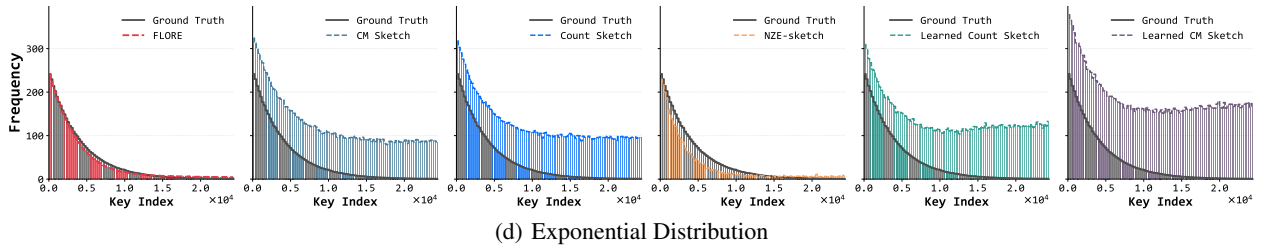
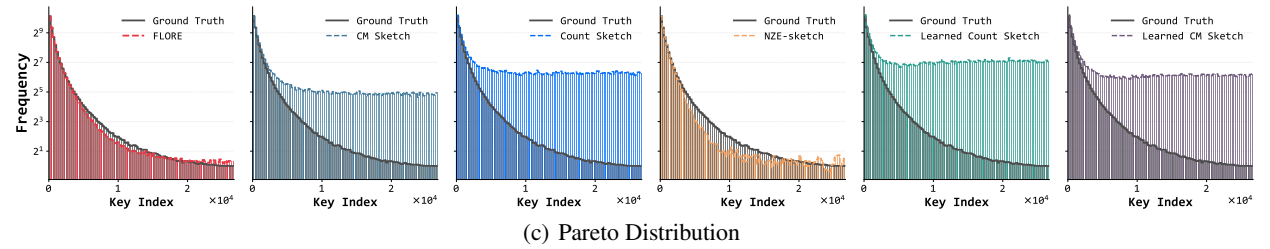
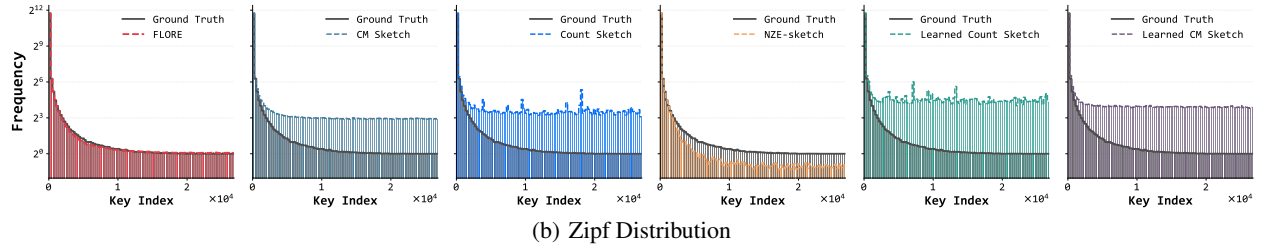
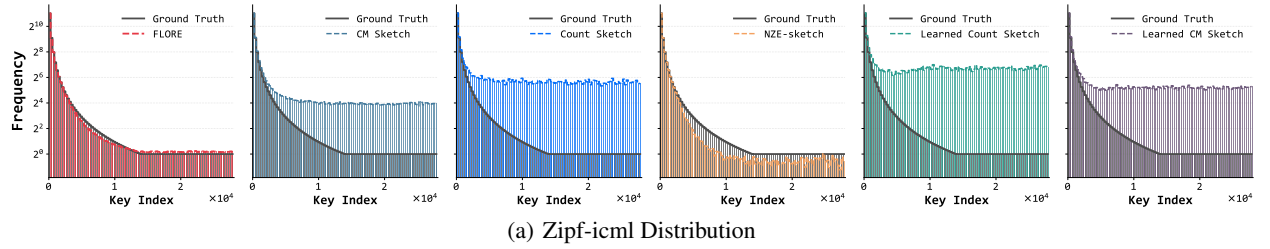


Figure 27: Visualizations of sketch recovery under a fixed *128KB* memory budget. From left to right, we present the recovery results of FLORE, CM, CS, NZE, LCM, and LCS, compared against the ground-truth sample distribution. To enhance readability, every 100 consecutive keys are aggregated in the x-axis.

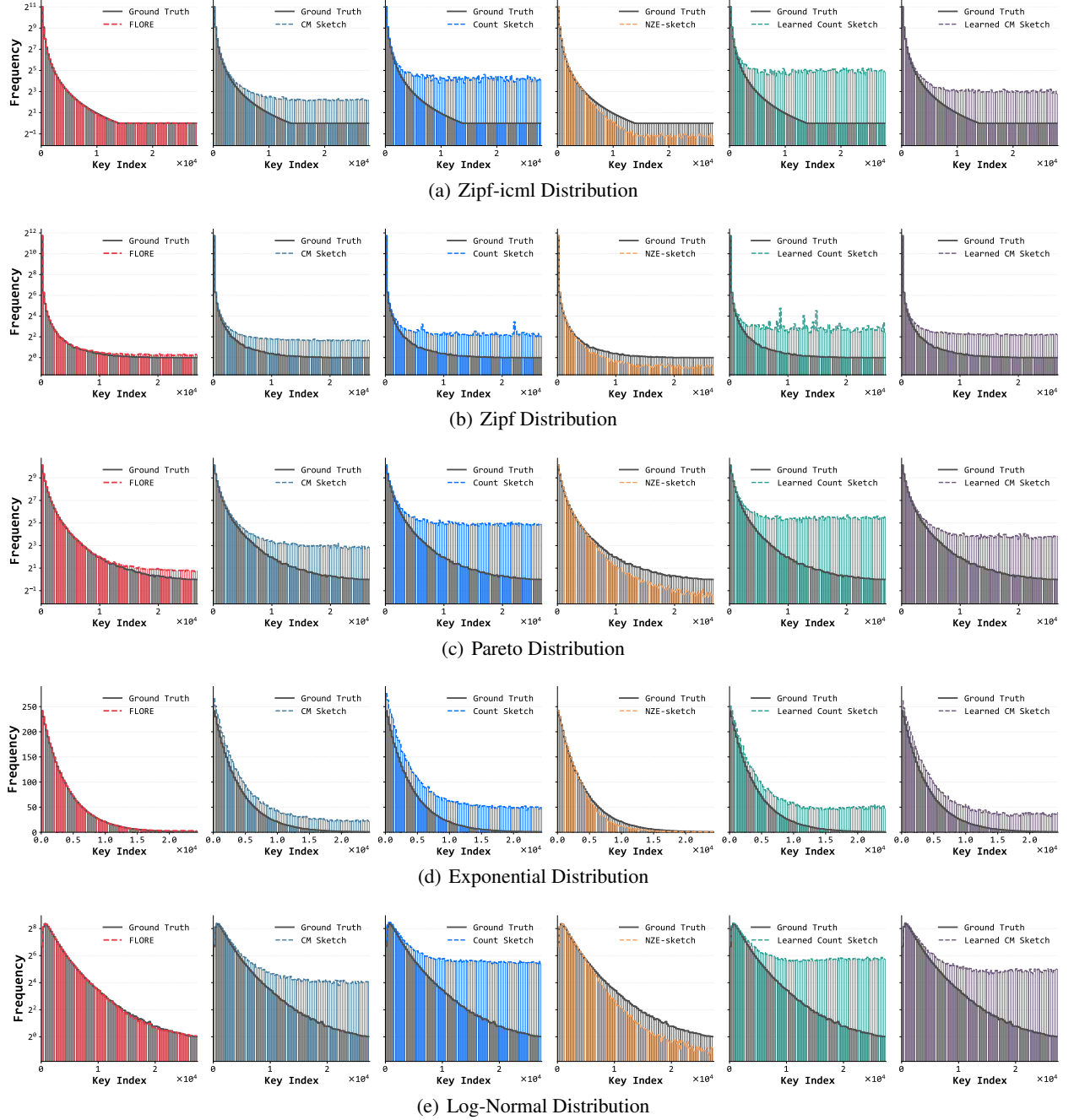


Figure 28: Visualizations of sketch recovery under a fixed 256KB memory budget. From left to right, we present the recovery results of FLORE, CM, CS, NZE, LCM, and LCS, compared against the ground-truth sample distribution. To enhance readability, every 100 consecutive keys are aggregated in the x-axis.